Real Estate Management

Release 8.0

jankstar

Oct 12, 2026

CONTENTS

1 Structure **3**

2 Contents **5**

2.1 Module Dependencies . . . 6
2.2 Installation . . . 6
2.3 Configuration . . . 7
2.4 Access Control . . . 10
2.5 Data Model . . . 13
2.6 Property Management . . . 13
2.7 Contract Management . . . 17
2.8 Rent Adjustments . . . 22
2.9 Graduated Rent . . . 24
2.10 Index Rent . . . 25
2.11 Comparative Rent (Rent Survey) . . . 27
2.12 Operating Cost Settlement . . . 32
2.13 CO2 Cost Allocation (CO2KostAufG) . . . 38
2.14 BVED External Billing Interface . . . 41
2.15 Option Rate (Input VAT Deduction) . . . 50
2.16 Extensions to Core Modules . . . 52
2.17 Tasks, Processes and Handover Reports . . . 55
2.18 Recurring Inspections . . . 59
2.19 Wizards . . . 63
2.20 Reports . . . 65
2.21 Accounting / WoWi . . . 67
2.22 Source Layout . . . 67
2.23 Running Tests . . . 69
2.24 Demo data scripts . . . 70
2.25 Release notes . . . 72

A Tryton ERP module for real estate management. Packaged as `jankstar_real_estate`, targeting Tryton 8.0.0 and Python 3.9–3.13.

Covers property management, lease and sales contracts, tenant/owner party roles, operating cost settlement, CO2 cost allocation under the German CO2KostAufG, and a German "Kontenrahmen der Wohnungswirtschaft" (WoWi) accounting chart.

CHAPTER

ONE

STRUCTURE

- **Master data** — real-estate objects in a tree (property building apartment/commercial unit/parking, equipment such as meters), addresses, measurements (areas, rooms, meter readings), occupancy, and party roles per object; meter reading sheets for the collective entry of the meter readings of a property, building or unit (meters with unit, tenant and previous reading, printable list, follow-up sheet).
- **Contracts** — contract types, contract parties with roles and validity periods, items (rented objects) and terms (recurring charges such as rent and operating cost advances); terms generate a planned cash flow that is booked as invoices via the *Create Contract Moves* wizard or a cron task (one invoice per contract and posting date; term types flagged as *Separate Move*, e.g. the rent deposit, get an invoice of their own).
- **Rent adjustments** — per contract term: graduated rent (§ 557a BGB) generated in advance as terms; index rent (§ 557b BGB) with price index series and import (CSV or GENESIS-Online API); comparative rent (§§ 558-558b BGB) from the rent survey with cap (20 %/15 %), request for consent, consent/partial consent/refusal and deadline tasks - both processed by the adjustment run (workflow with process, protocol, letters, receipts, execution); modernisation and operating cost procedures as headers so far.
- **Rent survey** — rent surveys (*Mietspiegel*, table or regression method) with versions, classification features and classes, table cells, feature groups and features, CSV import and consistency check; classes and features on property, building and rental unit (derived from the year of construction or measurements, automatic energy features); direct calculation of the local comparative rent (§ 558 BGB) with protocol, checks and mandatory acceptance; check of the rent of a new letting on the draft contract against the comparative rent (+10 % in a tight housing market, § 556d BGB, else +20 %) with optional adjustment of the terms.
- **Operating cost settlement** — billing units and settlement units allocate supplier invoice costs to contracts (by measurement, consumption, external billing, …), including CO2 cost allocation (CO2KostAufG) and the BVED interface to external metering providers.
- **Tasks, processes and handover reports** — tasks (follow-ups) on contracts, objects, billing units, rent adjustments, inspections and other records (tab *Tasks and Processes* with *Open* / *History*), created manually, by the module, by scheduled task rules or by process steps, with reminders and escalation; responsibility by group, user or the party role on the object (e.g. property administrator, caretaker - resolved to the linked user via the employee). Processes from templates with work instructions: each step is a task (planned until due, then open), completed by hand, by a condition or by executing its action; conditional steps; reopen and reactivate. Delivered templates: move-out, move-in, change of tenant (started by the contract type), index rent adjustment, vacancy / reletting on an object, and the inspection processes annual walkthrough and smoke detector check. Handover reports (move-in, pre-inspection, move-out) with checklists, keys and meter readings (created as meter readings when done) that complete the matching process steps.

- **Recurring inspections** — inspection types with interval, checklists and process template, inspection plans per property or building with due date rules (fixed rhythm or from execution, preferred month); inspections created ahead by a scheduled task, with lines per rental unit or equipment, typed results, validation, approval and an archived inspection report; tenant notice and letters; defects with deadlines, follow-up tasks and carry-over to the next inspection; access attempts with a second date for units without access; undo of the workflow steps; the process of the inspection with its steps and progress as first tab *Workflow*; default types annual walkthrough and smoke detector check with checklists and process templates; equipment kinds on equipment objects; menu *Master Data › Inspections*.
- **Accounting** — German WoWi chart of accounts, company-specific real-estate accounting configuration, input VAT option rate, and real-estate fields on invoices and journal lines.

Setup: master data and per-company settings (real estate accounting, scheduled tasks, task and process configuration) are described in *Configuration*, the entries in `trytond.conf` (GENESIS-Online token, e-mail, bus) and the required cron process in *Installation*.

Demo data: the scripts under `tests/` create properties with buildings, units, meters, property administrator and caretaker, smoke detectors and the inspection plans (annual walkthrough, smoke detector check), contracts with graduated and index rents, billing units, supplier invoices and payments, the complete Berlin rent survey 2026 with a comparative rent calculation - see *Demo data scripts*.

Full documentation is split into per-topic pages under `doc/` — see *Contents* below and *Source Layout* for the file-to-model mapping.

ISO_4217 Currency https://de.wikipedia.org/wiki/ISO_4217
ISO_3166 Country Codes https://www.laenderdaten.de/kuerzel/iso_3166-1.aspx

This module was developed with the support of Claude Code — all requirements, specifications, tests, corrections, and optimizations were carried out by human contributors.

CHAPTER

TWO

CONTENTS

- Module Dependencies
- Installation (incl. trytond.conf)
- Configuration
- Access Control
- Data Model
  - Property Management
  - Contract Management
  - Rent Adjustments
    - Graduated Rent
    - Index Rent
    - Comparative Rent (Rent Survey)
  - Operating Cost Settlement
    - CO2 Cost Allocation (CO2KostAufG)
    - BVED External Billing Interface
  - Option Rate (Input VAT Deduction)
  - Tasks, Processes and Handover Reports
  - Recurring Inspections
  - Extensions to Core Modules
- Wizards
- Reports
- Accounting / WoWi
- Source Layout
- Running Tests
  - Demo data scripts
- Release notes

2.1 Module Dependencies

`ir`, `res`, `company`, `party`, `product`, `currency`, `country`, `account`, `account_invoice`, `account_deposit`, `account_payment_clearing`, `account_tax_non_deductible`, `bank`

`bank` is used only by the BVED export's A-/M-Satz bank-field lookup (`party.bank_accounts_used` → IBAN → classic Kontonummer/Bankleitzahl) — see BVED External Billing Interface.

2.2 Installation

```
pip install -e .           # development install
pip install -e '.[test]'   # with test dependencies
```

The module entry point is declared in `setup.py`:

```
[trytond.modules]
real_estate = trytond.modules.real_estate
```

Link or install the package inside the Tryton modules directory so that `trytond-admin -u real_estate` can find it.

2.2.1 trytond.conf

Entries in the server configuration used by the module (all optional):

```
[real_estate]
# GENESIS-Online (Destatis) API token for the import of price index
# values (button "Fetch Values" and scheduled task price_index_import);
# without a token only the CSV import is available
genesis_token = <API token from GENESIS-Online, menu Webservice (API)>
#genesis_url = https://genesis.destatis.de/genesisWS/rest/2020/
#genesis_timeout = 60

[email]
# SMTP server and sender for task reminders by e-mail (task types with
# "E-Mail"); without it the error is only logged
uri = smtp+tls://user:password@smtp.example.com:587
from = tryton@example.com

[bus]
# live display of task notifications in the client
allow_subscribe = True
```

2.2.2 Processes

Besides the server (`trytond`) the scheduled tasks need the cron process (`trytond-cron`): a single `ir.cron` entry *Real Estate Daily Tasks* dispatches the per-company scheduled tasks (see Configuration, `real_estate.cron_task`). After `trytond-admin -u real_estate` restart server, worker and cron - the cron process otherwise keeps running the old code.

2.3 Configuration

The following master data must be set up before the module can be used:

real_estate.object_party.role
Partner roles in combination with the property type (`base_object.type`), e.g. *Tenant*, *Owner*, *Administrator*.

real_estate.measurement.type
Named measurements linked to a unit of measure, e.g. *Living Area (m²)*, *Number of Rooms*, *Gross Floor Area (m²)*.

real_estate.contract.type
Contract types defining invoice direction (in/out), default journal, tax defaults, contract number prefix, and whether occupancy exclusivity is enforced. Group *Processes*: process templates started automatically when a contract is set to running (`start_process_template`), terminated (`termination_process_template`) or its partner is changed (`partner_change_process_template`) - see Tasks, Processes and Handover Reports.

real_estate.contract.term.type
Term type definitions with default rhythm (monthly / quarterly / …), default quantity source (measurement type), default account, and whether terms of this type are posted as a separate move (own invoice, optional own move description).

real_estate.price_index / real_estate.index_cap_rule
Price index series and values (incl. GENESIS-Online import) and cap rules for index rents - see Index Rent.

real_estate.use_class
Dynamic use-class catalogue replacing the former static selection field. Each record carries two boolean flags:

has_basement_nr
Show/hide the *Basement Number* field on rental objects.

has_parking_nr
Show/hide the *Parking Number* field on rental objects.

Six default records are loaded at module installation: *Apartment* (has_basement_nr), *Office* (has_basement_nr), *Retail* (has_basement_nr), *Warehouse* (has_basement_nr), *Parking* (has_parking_nr), *Garage* (has_parking_nr). Additional classes can be created without changing code.

real_estate.cost_category_group / real_estate.cost_type
Cost categories (e.g. *Heating*, *Water*) and individual cost types used to structure operating cost settlements. Default values for German BetrKV (§ 2) are loaded at module installation.

real_estate.re_accounting (re_accounting.py)
Standalone, company-scoped real-estate accounting configuration, referenced one-directionally from `company.company.re_accounting` (`company.py`; optional — a company without a linked `re_accounting` record gets none of the automation below).

re_account_allocation_by_owner
Vacancy cost account (debit and credit side of vacancy postings).

re_journal_billing
Journal used for direct GL postings in operating cost settlements.

re_payment_term_billing
: Default payment term for operating cost settlement invoices, used when the contract itself has none set.

receipt_days (default 3)
: Days from the declaration date of an index rent adjustment to the expected receipt by the tenant - only a preview of the effective date, the actual receipt is captured separately.

deposit_task_months (default 6)
: Months after the end of a contract until the move-out process step *Settle deposit* is due.

co2_landlord_share_commercial
: Default CO2 cost landlord share (%, 0–100) for commercial properties, which are not covered by the residential 10-tier distribution model — see *CO2 Cost Allocation (CO2KostAufG)* under *Operating Cost Settlement* below. Shown right before the *Cron Tasks* table on the form.

cron_tasks
: One2Many to `real_estate.cron_task` — the company's scheduled task configuration, see below.

real_estate.cron_task (cron_task.py)
: Per-`re_accounting` (i.e. per-company) row configuring one recurring operation: `task` (selection, currently `update_contract_status` / `update_contract_cash_flow` / `book_contract_cash_flow` / `update_option_rate` / `price_index_import` (GENESIS-Online, see above) / `task_notify` (task reminders and escalation) / `task_rules` (task rules and process conditions)), `valid_from`, `valid_until`, `interval_days`, `interval_months`, `schedule_day_of_month`, `horizon_months_ahead`, `invoice_state`, `future_contracts_horizon_days`, `last_run` (updated by the dispatcher), `active`. Task names are deliberately rhythm-agnostic (no "daily"/"rolling"/… in the name) — how often each task actually runs is a per-row configuration choice made by the user, not implied by the task itself; see *Scheduling and parameters* below. None of `interval_days`/`interval_months`/`schedule_day_of_month` is individually required, but `validate()` rejects a row that has none of the three set. A unique SQL constraint prevents duplicate rows for the same `(re_accounting, task)` pair. No rows are seeded automatically — a company gets no automatic behaviour until at least one row is added under the *Cron Tasks* tab of its `real_estate.re_accounting` record.

: **Dispatch.** A single `ir.cron` entry ("Real Estate Daily Tasks", method `real_estate.contract|cron_daily` — registered via a small `ir.cron.method` selection extension in `ir.py`, since that field is otherwise a fixed core list) runs daily and calls `Contract.cron_daily()` (`contract_core.py`). It iterates all active `cron_task` rows and, for each one that is due (see *Scheduling* below), calls the matching `Contract._cron_<task>(re_accounting, task)` classmethod and updates `last_run`. A future recurring operation only needs a new `_cron_<code>` classmethod plus a new `get_tasks()` option — no new `ir.cron` entry.

: **Scheduling and parameters.** Each `cron_task` row carries its own parameters, read by its handler from the `task` record passed in, and its own rhythm - e.g. `update_contract_status` daily (`interval_days = 1`), `update_contract_cash_flow` every six months (`interval_months = 6`), `book_contract_cash_flow` and `update_option_rate` monthly (`schedule_day_of_month` set, e.g. 15 and 1 respectively).

: `valid_from`/`valid_until` (both optional) are hard lower/upper bounds checked before all three modes below - the task never runs while `today < valid_from` or `today > valid_until`. If the task has never run yet and today is already past `valid_from`, it becomes due immediately (using today, not `valid_from`, as the actual first `last_run`). E.g. setting `valid_from = 15.03.2026` on a row activated on 20.03.2026 makes it run right away on 20.03.2026, not wait until the next scheduled date. `valid_until` just stops execution once passed - the row itself, and its `last_run` history, stay in place. `validate()` rejects a row where `valid_until` is before `valid_from`.

Three scheduling modes, checked by `Contract._cron_task_is_due()` in this priority order:

1. **Day-of-month-based**: if `schedule_day_of_month` (1-31) is set, it takes priority over both interval fields — the task runs at most once a month, on or after that calendar day (capped to the last day of shorter months), guarded by comparing `last_run`'s year/month to today's so a delayed run still catches up without re-running twice in the same month.
2. **Month-interval-based**: otherwise, if `interval_months` is set, it takes priority over `interval_days`. If `valid_from` is also set, `Contract._add_months()` computes the exact recurring date `valid_from + k * interval_months` months (same day-of-month as `valid_from`, clamped to shorter months), advancing `k` past `last_run` each time - so e.g. `valid_from = 15.03.2026` with `interval_months = 1` runs on 15.03., 15.04., 15.05., … and re-aligns to the correct slot even after a delayed run, instead of drifting from whatever date the delayed run actually happened on. Without `valid_from`, falls back to a looser check: due once at least that many calendar months (year/month difference, not exact days) have passed since `last_run` - a convenience for longer rhythms (e.g. 6 for half-yearly) without day-of-month precision.
3. **Day-interval-based** (default): otherwise, due once `today - last_run >= interval_days` (or `last_run` is unset).

`_cron_update_contract_status`
: Auto-terminates `running` contracts of the company whose `end_date` has passed and which have no active termination yet (`termination_date` unset): sets `state = 'terminated'`, `termination_date = end_date`, `terminated_by_type = 'expired'`, and a `contract.log` entry — reusing the existing `terminated` state rather than adding a new one, so all state-dependent logic (occupancy, `create_moves`, item validation) keeps working unchanged. See `get_effective_end_date()` (`termination_date` if set and earlier than `end_date`, else `end_date`) which already governs occupancy for both `running` and `terminated` contracts regardless of whether this task has run yet.

`_cron_update_contract_cash_flow`
: **Cash-flow recalculation only** (`action='re_calc'`): calls `Contract.call_create_moves()` for running/terminated contracts of the company, and for not-yet-started contracts whose `start_date` is within `today + future_contracts_horizon_days` (the row's own field, default 60) — already-running/-ended contracts are always included regardless of this value; it only decides how far ahead of their actual start not-yet-started contracts get pulled in early. For every contract included, `Term.re_calc()` (`contract_term.py`) recalculates that term's cash flow up to a **fixed** horizon of exactly 1 year from today (`_re_calc_year = 1`, not related to and not affected by `future_contracts_horizon_days`). It does **not** create invoices — booking is a separate, explicitly scheduled step, see `_cron_book_contract_cash_flow` below.

`_cron_book_contract_cash_flow`
: Books (creates invoices for) all due terms, using `action='re_calc_and_create'` so it is self-sufficient regardless of whether `_cron_update_contract_cash_flow`'s horizon already covers the target date. The horizon is the end of the month that is `horizon_months_ahead` months after the run date — e.g. with `schedule_day_of_month = 15` and `horizon_months_ahead = 1`, a run on 15.07 books everything due up to 31.08. The row's own `invoice_state` (`draft`/`posted`, default `draft`) is passed through to `call_create_moves`/`_create_moves` and determines whether the created invoices are posted immediately or left as drafts for manual review. Note: each invoice's own `invoice_date`/ `accounting_date` is *not* forced to this month-end horizon — it is the individual term's own computed `document_date` (see `_create_moves`, `inv_date = invoice_date or document_date` with `invoice_date=None` here), which may differ per term/rhythm.

`_cron_update_option_rate`
Recomputes and, where the rate actually changed, books new option rates via `OptionRate.process_update()` (`option_rate.py` — the same logic as the manual `real_estate.option_rate_update.wizard`, see Wizards) for every property of every company using this `re_accounting` configuration, as of today's date. Intended to run monthly with `schedule_day_of_month = 1`, which also determines the `effective_date` used by `process_update` (the 1st of the current month). Per-record results (created/updated/unchanged/ skipped counts, plus one detail line per processed object) go to the Python logger, not `contract.log`, since this task is not tied to individual contracts.

Tasks and processes (menu *Configuration › Tasks and Processes*)
Task types, task rules (delivered inactive), process templates and handover checklists - see Tasks, Processes and Handover Reports. Per user: preference `task_email` (task reminders by e-mail, default on).

Server configuration (`trytond.conf`)
See Installation, section *trytond.conf*.

2.4 Access Control

Seven user groups are provided. Access is additive — a user may belong to multiple groups.

`group_real_estate_admin` — Real Estate Administration
Full CRUD on all module models. Intended for system administrators and property managers with unrestricted access. The built-in `admin` user is automatically assigned to this group at installation.

`group_real_estate_object` — Real Estate Object
Full CRUD on property and object data (`base_object`, `address`, `measurement`, `meter_reading`, `object_party`, `occupancy`). Read-only access to contract, billing, and configuration models. Intended for facility managers who maintain the property master data but do not manage contracts or run settlements.

Exception: also has full CRUD on `real_estate.option_rate` (in addition to `group_real_estate_admin`), since option rates are maintained together with the property/object master data via the *Update Option Rates* wizard — see Option Rate. The `group_real_estate_contract` and `group_real_estate_billing` groups remain read-only on it.

`group_real_estate_contract` — Real Estate Contract
Full CRUD on contract data (`contract`, `contract.term`, `contract.item`, `contract.log`, `contract.term.cash_flow`, `contract.term.tax`, `account_contract`). Read-only access to property, billing, and configuration models. Intended for property managers / letting agents.

Exception: also has full CRUD on `real_estate.contract.term.adjustment` (in addition to `group_real_estate_admin`) — see the *Contract Term Adjustment* wizard in Wizards. The `group_real_estate_object` and `group_real_estate_billing` groups remain read-only on it.

`group_real_estate_billing` — Real Estate Billing
Full CRUD on all operating cost and settlement models (`billing_unit`, `billing_unit.log`, `billing_unit.moves`, `settlement_unit`, `settlement_result`, `cost_share`, `cost_category_group`, `cost_type`, `account.configuration.real_estate`), and on the BVED external billing models (`bved.service_provider`, `bved.provider_assignment`, `bved.object_number`, `bved.object_number.m_satz_line`, `bved.export`, `bved.import`, `bved.import.line` — see *BVED External Billing Interface* below). Read-only access to property and contract models. Intended for the operating cost accountant who runs the annual settlement but does not modify contracts or property master data.

Exception: the group_real_estate_contract group also has read-only access to all six BVED models (shared master/exchange data, not contract-specific); group_real_estate_object has none.

group_real_estate_view — Real Estate View (German: *Immobilien Anzeigen*)
Read-only (perm_read=True, perm_write/perm_create/ perm_delete=False) on every module model that has explicit ir.model.access records — i.e. every model listed in the permission matrix below, without exception. Defined in its own file, group_real_estate_view.xml (one res.group record plus one ir.model.access record per model, generated by enumerating every <record model="ir.model.access"> across the module's XML files via an XML parser — not a plain-text/regex search, which would also match commented-out ir.model.access blocks such as the disabled real_estate.address.format one in address.xml). Intended for users who need full visibility across the module (e.g. auditors, read-only reporting accounts) without being able to change anything, independent of which of the other four functional groups they would otherwise need for write access. Deliberately scoped to the module's own real_estate.* models only — it does not touch access to core models the module merely extends with extra fields (party.party, account.invoice, account.move.line, company.company), since those already have their own, separate access control.

group_account_view — Accounting View (German: *Rechnungswesen Anzeigen*)
Read-only counterpart to core's own account.group_account ("Accounting" / German *Rechnungswesen*) group — same ten models (account.aged_balance, account.general_ledger.account, account.general_ledger.account.party, account.invoice, account.invoice.line, account.invoice.payment.mean.rule, account.journal.period, account.move, account.move.reconciliation, account.move.template) that group_account itself has explicit ir.model.access records for (found by querying a live database's ir.model.access table for that group — group_account's own permissions are additive with its child group_account_admin's via res.group.parent, so account.account/account.period/account.fiscalyear/ account.tax etc. are only reachable through that child group, not directly on group_account, and are intentionally excluded here). All ten downgraded to read-only. Defined in group_account_view.xml — kept as a module of its own file, not merged into group_real_estate_view.xml, since it grants access on core accounting models rather than this module's own; added here (rather than patched into core's account module, which this module must not modify) because real_estate already depends on account. Independent of group_real_estate_view — a user typically needs both to see the full picture (real-estate objects/contracts plus the accounting moves/invoices behind them) without being able to change either.

group_account_view.xml also links the group to the top-level *Rechnungswesen* menu (account.menu_account) via menu_account_group_account_view (ir.ui.menu–res.group), since that menu is only shown to explicitly linked groups.

group_party_access — Party Access (German: *Partner Zugriff*)
Party editing is a separately-assigned privilege: only members of group_party_access may create, modify, or delete parties; every *other* role — including this module's own group_real_estate_admin/object/contract/billing and both "view" groups above — has read-only access (so party names still display wherever a party is referenced: contracts, invoices, lists…). Defined in group_party_access.xml: a _default (no-group) ir.model.access row (perm_read=True, the rest False) plus one full-CRUD row for group_party_access itself. The built-in admin user is auto-assigned to this group at installation (res.user–res.group), matching how core's own group_account/group_account_admin and this module's own group_real_estate_admin already do, so the out-of-the-box admin account is not locked out.

> **Warning**
>
> Groups of optional modules (`sale`, `purchase`, …) are not included. Users of such modules who need to create or edit parties must be added to `group_party_access` by an administrator (Administration User the user's *Groups* field).

Permission matrix (C = CRUD · R = read · — = no access). The `view` group is not shown as its own column since its value is R for every single row without exception:

Model	admin	object	contract	billing
`real_estate.address`	C	C	R	R
`real_estate.base_object`	C	C	R	R
`real_estate.base_object.occupancy`	C	C	C	R
`real_estate.option_rate`	C	C	R	R
`real_estate.meter_reading`	C	C	R	R
`real_estate.measurement.type`	C	R	R	R
`real_estate.use_class`	C	R	R	R
`real_estate.measurement`	C	C	R	R
`real_estate.object_party`	C	C	R	R
`real_estate.object_party.role`	C	R	R	R
`real_estate.contract`	C	R	C	R
`real_estate.contract.item`	C	R	C	R
`real_estate.contract.term`	C	R	C	R
`real_estate.contract.term.tax`	C	R	C	R
`real_estate.contract.log`	C	R	C	R
`real_estate.contract.account_contract`	C	R	C	R
`real_estate.contract.term.cash_flow`	C	R	C	R
`real_estate.contract.term.adjustment`	C	R	C	R
`real_estate.account_contract`	C	R	R	R
`real_estate.contract.type`	C	R	R	R
`real_estate.contract.term.type`	C	R	R	R
`real_estate.re_accounting`	C	R	R	R
`real_estate.cron_task`	C	R	R	R
`real_estate.co2_emission_share`	C	R	R	R
`real_estate.contract.type.tax`	C	—	R	R
`real_estate.cost_category_group`	C	R	R	C
`real_estate.cost_type`	C	R	R	C
`real_estate.co2_kostaufg`	C	R	R	C
`real_estate.co2_kostaufg.consumption`	C	R	R	C
`real_estate.billing_unit`	C	R	R	C
`real_estate.billing_unit.log`	C	R	R	C
`real_estate.billing_unit.moves`	C	R	R	C
`real_estate.settlement_unit`	C	R	R	C
`real_estate.cost_share`	C	C	C	C
`real_estate.settlement_result`	C	C	C	C
`real_estate.bved.service_provider`	C	—	R	C
`real_estate.bved.provider_assignment`	C	—	R	C
`real_estate.bved.object_number`	C	—	R	C
`real_estate.bved.object_number.m_satz_line`	C	—	R	C
`real_estate.bved.export`	C	—	R	C

continues on next page

Table 1 – continued from previous page

Model	admin	object	contract	billing
real_estate.bved.import	C	—	R	C
real_estate.bved.import.line	C	—	R	C
account.configuration.real_estate	C	R	R	R

2.5 Data Model

The data model is split into the following areas:

- Property Management
- Contract Management
- Rent Adjustments
  - Graduated Rent
  - Index Rent
  - Comparative Rent (Rent Survey)
- Operating Cost Settlement
  - CO2 Cost Allocation (CO2KostAufG)
  - BVED External Billing Interface
- Option Rate (Input VAT Deduction)
- Tasks, Processes and Handover Reports
- Recurring Inspections, Duty to maintain safe premises
- Extensions to Core Modules

2.6 Property Management

real_estate.address (address.py)
Structured address with granular fields (street_name, building_number, unit_number, floor_number, room_number, post_box) plus an unstructured fallback text field. Supports both formats and is linked to base_object records.

real_estate.base_object (base_object.py)
Central entity for every real estate asset.

Types: property · building · object · land · equipment

Workflow: Draft Active Closed

Key features:

- Parent–child tree (tree() mixin), e.g. Property Building Apartment
- sequence unique per type and parent (constraint sequence_unique); when parent and type are set and the sequence is empty, the next free one below the parent is proposed (step 10, on_change_with_sequence). The object number is the parent's object number plus / and the sequence.
- One2Many relations to Address, ObjectParty, Measurement, and BillingUnit

- History tracking via `BaseObjectOccupancy` (tenant occupancy periods)
- `occupancy_state` (Function field, `type = 'object'` only): rented / vacant / under negotiation as of the `occupancy_date` context key (defaults to today), read from `BaseObjectOccupancy`. Added as a column to the object search's default list view (`base_object_list_simple.xml`, priority 10 — the one actually used for a Many2One's pick dialog, not the higher-priority-number `base_object_tree.xml`) so that picking an object for a contract item shows its status as of *that item's own* `valid_from` (`ContractItem.objects`' own `context={'occupancy_date': Eval('valid_from')}`), not just today.
- Meter readings (`MeterReading`) linked to equipment objects
- `billing_as` / `collective_billing` flags to control how operating cost billing is aggregated at property level
- `next_billing_start_date` (function field, property only): the earliest `start_date` among all non-`billed` billing units of the property. Drives the *ready for billing* checks and the green line-color highlight in the billing unit list view.
- Buttons `compute_value_shares`, `compute_settlement_result_property`, and `ready_for_billing_property` delegate bulk settlement actions to all billing units of the property that share `next_billing_start_date`. `billing_property` opens the `real_estate.billing_unit.wizard` instead of billing directly.
- `call_billing` / `do_billing` (classmethods) run the actual billing for one or more properties, optionally in the background queue (`execute_in_queue`). `do_billing` requires every billing unit that would be billed to already be in state `ready_for_billing`; with `collective_billing` all billing units sharing the same start date must be included together, otherwise a `ValidationError` is raised.
- `bved_provider_assignments` (function-less One2Many, `readonly=True`, visible only for `type in ('property', 'building')`) shows the `real_estate.bved.provider_assignment` records anchored on this object. The *BVED Provider Assignments* notebook page itself stays hidden until at least one such assignment actually exists — it is a read-only overview, not a way to create or edit assignments; those are always maintained on the dedicated *BVED Provider-Liegenschaft* screen (`real_estate.bved.provider_assignment`, see below).

Rental object fields (visible only for `type = 'object'`):

`type_of_use`
: Economic use: `residential`, `commercial`, `property` (owner-occupied), or `internal`. Drives the contract-type filter.

`use_class`
: Many2One to `real_estate.use_class`. Controls which additional fields (`basement_nr` / `parking_nr`) appear on the form.

Meter fields (visible only for `type = 'equipment'`, `e_type = 'meters'`):

`meter_no_decimals`
: Boolean, default `True`. When set, meter readings are validated and rounded to integer values. The estimate-consumption wizard and `simulate_estimate` respect this flag and round accordingly.

Context classes for list views (`BaseObjectEquipmentContext`, `BaseObjectOccupancyContext`, `MeterReadingContext`) provide filterable search panels. Selection fields shared with the underlying model (e.g. `e_type`, `state`) are derived dynamically so their labels stay in sync automatically.

real_estate.object_party.role (object_party.py)
Configurable role definitions per object type (e.g. *Tenant* only for type `object`, *Owner* for all types).

real_estate.object_party (object_party.py)
Links a `party.party` record to a `BaseObject` with a typed role and an optional validity period.

real_estate.measurement.type (measurement.py)
Named measurement type linked to a `product.uom` unit.

Supports a **group hierarchy**: a type with `is_group = True` acts as a container grouping one or more child types via the `parent` Many2One field. Multi-level hierarchies are supported — groups can themselves have a parent group (e.g. *Gross Floor Area* → *Net Floor Area* → *Living Area*).

Constraints enforced at save time:

- All child types must share the same unit as their parent group.
- Circular references (a group referencing one of its own descendants as parent) are rejected.
- Only **leaf** (non-group) types can be assigned to individual `real_estate.measurement` records on `BaseObject` instances.

When a group type is referenced in `ContractTermType.m_type` or in `SettlementUnit.m_type`, the system automatically expands the group to all descendant leaf type IDs at query time, so measurements of any child type are included in the calculation.

get_effective_ids(m_type)
Classmethod resolving `m_type` to the flat list of leaf type ids to search: itself if not a group, else all descendant leaf ids (recursively, for nested groups).

real_estate.measurement (measurement.py)
Associates a numeric value and measurement type with a `BaseObject` for a given validity period.

get_total_value(base_object_id, m_type, as_of_date=None)
The single, hierarchy-aware entry point for "the value of `m_type` on `base_object_id` as of `as_of_date`" — every computation in the module that needs a measurement value (as opposed to just listing raw rows for display) goes through this classmethod rather than querying `real_estate.measurement` directly. Resolves `m_type` via `MeasurementType.get_effective_ids()` and, for **each** effective leaf id independently, looks up the object's own latest row with `valid_from <= as_of_date` (or the single latest row ever, if `as_of_date` is `None`) — then **sums** whatever was found. This matters for a "Summenbemessung" (group type) where a single object carries values under **several sibling leaf types** at once (e.g. a mixed-use object with both a residential and a commercial area entry): the object contributes all of them, not just whichever happens to be the most recently dated row. Returns `None` if `m_type` is falsy, resolves to no effective ids, or none of them has any matching row at all (true "not recorded", as opposed to a recorded value of zero) — callers use this to distinguish "no data" from a real zero. Used by `real_estate.bved.provider_assignment` (L-Satz *Gesamtfläche*), `BillingUnit.on_change_with_co2_total_area`, `SettlementUnit.compute_value_shares` (`allocation_by_measurement`), `ContractTerm._sum_measurements`, and `OptionRate._measurement_value` (dynamic option-rate weighting).

company / property (Function fields)
Display/search-only, derived from `base_object` — `company` is simply `base_object.company`; `property` is `base_object` itself if its own `type = 'property'`, else `base_object.property` (the stored ancestor-property reference every `base_object` already carries). Used by the *Measurements* list below to show and filter by company/Wirtschaftseinheit without a join in the view itself. `search_property` matches either a row whose own `property` field equals the given value, or a property-type row whose own `id`

does (so filtering by a given property also finds that property's own direct measurements, not just its descendants').

***Measurements* list (`act_measurement_tree` in `measurement.xml`)**
Menu *Master Data*, between *Rental Object* and *Equipment*. A plain, filterable browse list over `real_estate.measurement` (columns: Company, Property, Object, From, Measurement Type, Value — summed — , Symbol), **not** a wizard and **not** hierarchy-aware the way `get_total_value()` is — it lists raw rows matching a simple filter, with no de-duplication to "the one currently effective row per (object, type)". The filter panel (`real_estate.measurement.context`, via the action's `context_model`/`context_domain` — same mechanism as `real_estate.meter_reading.context` above) offers:

- `company` (required) — restricts to `base_object.company`.
- `property` — restricts the `base_object` picklist to that property's own subtree (via `child_of`) and, if `base_object` itself is left empty, restricts the list the same way.
- `base_object` — if set, takes precedence over `property` and restricts the list to this object and **all** of its descendants (`('base_object', 'child_of', [base_object], 'parent')` — inclusive of the object itself).
- `m_type` — exact match, if set.
- `date` ("Stichtag", required, default today) — a plain `valid_from <= date` filter; deliberately **not** the latest-row-per-group logic of `get_total_value()` — every row satisfying the date cutoff is shown (and contributes to the summed Value column), including any historical rows later superseded by a newer `valid_from` on the same object/type.

`real_estate.meter_reading` (`base_object.py`)
Meter reading record linked to an equipment object of type `meters`.

Key fields: `company` (stored, auto-filled from `base_object` on change), `base_object`, `meter_id`, `reading_date`, `m_type` (`initial` / `reading` / `estimate` / `final`), `value`, `unit` (derived from the meter's `meter_unit`), `consumption` (difference to previous reading for counter meters), `comment` (free text).

Browseable via the *Meter Readings* menu entry under *Master Data*, filterable by company, property, parent object, equipment (meters only), and date range (`from_date` / `to_date`).

`real_estate.meter_reading.sheet` (`meter_reading_sheet.py`)
Meter reading sheet (*Zählerableseliste*) for the collective entry of meter readings, menu *Master Data* › *Meter Reading Sheets* and relate *Meter Reading Sheets* on an object (sheets of the object and below).

Header: `base_object` (property, building or rental unit; the `property` is derived), `reading_date`, optional selection of the meters by `equipment_kind` and `name_filter` (`ilike`, e.g. *Warmwasser*), `reader` (reading user of the readings created), `progress` (*entered* / *meters*), state *Draft* / *Done* / *Cancelled*.

Lines (`real_estate.meter_reading.sheet.line`), one per meter, in an editable list: rental unit and main tenant on the reading date (via the occupancy), meter, meter ID, previous reading (date and value, last reading before the reading date), **new value**, unit, consumption (counters), remarks, link to the meter reading created.

Buttons:

- *Load Meters* (also on create, e.g. a new sheet, a copy or a follow-up sheet): adds the meters below the object not yet listed (not deactivated, filtered) and refreshes unit, tenant and previous reading; values entered are kept.
- *Complete*: confirmable warning for meters without value; creates one meter reading (`m_type='reading'`) per line with value - an existing reading of the meter on the reading date

is linked instead (as in the handover report). The validation of the meter reading applies (same meter ID, counter value not lower).

- *Reset to Draft*: deletes the meter readings created by the sheet (`reading_created`), linked ones are kept; completing again creates them anew.
- *Follow-up Sheet* (done): new sheet with the same object and filter, reading date + 1 year, lines loaded with this reading as previous value, opened at once.

A done sheet cannot be deleted. The sheet can be the reference of tasks and processes. Reminder: task rule O01 *Record meter readings* (done as soon as every meter has a reading, e.g. by completing the sheet). Print: *Meter Reading Sheet* (see *Reports*).

Consumption estimate (`simulate_estimate` / `create_estimate`):

`simulate_estimate(base_object, per_date, meter_id=None)`
Returns `(estimated_value, consumption, r1, r2)`. Prefers **interpolation** when readings exist both before and after `per_date`: takes the closest reading on each side and interpolates linearly between them. Falls back to **extrapolation** using the last two readings within one year before `per_date` when no future reading is available. Rounds the result to the meter's UOM digit precision; if `meter_no_decimals` is set on the meter, rounds to integers.

`create_estimate(base_object, per_date, reason, meter_id=None)`
Calls `simulate_estimate` and saves a new reading with `m_type = 'estimate'`.

2.7 Contract Management

`real_estate.contract.type` (`contract_type.py`)
Template for contracts. Defines invoice direction (`in`/`out`), default taxes, accounting journal, number prefix, step sizes for item and term sequence numbers, and whether occupancy exclusivity is enforced.

`oc_mark`
Free-text label used in operating cost settlement invoice descriptions and invoice headers, e.g. `"Betriebskostenabrechnung 2025"`. Falls back to `"Operating Cost Settlement"` / `"Operating Costs"` when empty.

`adjustment_procedures`
Rent adjustment procedures allowed for contracts of this type (MultiSelection, see *Rent Adjustments* below). A procedure is only possible for a term if both the contract type and the term type allow it.

`real_estate.contract.type.tax` (`contract_type.py`)
Many2Many relation table between `ContractType` and `account.tax`.

`real_estate.contract.term.type` (`contract_type.py`)
Template for contract terms. Defines default rhythm, rhythm type (daily / weekly / monthly / quarterly / annually / one-time), rhythm start day, measurement type for quantity derivation (`m_type`), informative measurement type for a value per area only (`info_m_type`), default quantity, and default account.

`adjustment_procedures`
Rent adjustment procedures allowed for terms of this type (MultiSelection). Consistency rules: operating cost procedures only with an operating cost processing, rent procedures only without; one-time terms have no procedures.

`separate_move` / `move_description`
Separate Move: terms of this type are posted in periodic postings as an invoice of their own per contract and posting date instead of together with the other terms of the contract (e.g. rent

deposit). `move_description` (translatable, only visible with `separate_move`) is the description of that invoice; empty = the default description (contract type mark or name). The contract reference is kept on every invoice.

`real_estate.contract` (`contract_core.py`)

Main contract record.

Workflow: Draft → Running → Terminated / Cancelled

Key features:

- Links to `party.party` (contractual partner, see `real_estate.contract.party` above), `base_object` (property), and one or more `ContractItem` records
- Generates Tryton accounting moves (invoices) for all active terms via `CreateContractMoves` wizard (`call_create_moves`): one invoice per posting date for the regular terms, one per term type (and own payment term) for terms of a `separate_move` term type (`_move_group_key`, `_move_description`)
- `get_move_payment_term(term=None)` resolves the payment term of these invoices and of the cash flow due dates: the term's own `payment_term` (separate move only) → the contract's `payment_term` → the party's customer/supplier payment term (by invoice type) → the default customer payment term of `account.configuration` (no default on the supplier side)
- Cash flow tabs on the contract form: *Draft* (invoice state `draft`/`validated`), *Pending* (`posted`), *Paid* (`paid`)
- `_refresh_occupancy_for_contracts` updates occupancy records when a contract starts or is terminated; cancelled contracts are excluded from occupancy calculations automatically
- `offer_date` (*Offered on*, editable in draft) and `date_of_signature` (*Date of Signature*) on the tab *General* - a draft with an offer date is an offer (e.g. new letting); information only, the activation checks are unchanged
- *Copy Terms* (button on the tab *Terms*, draft only, wizard `real_estate.contract.copy_terms`): proposes the terms of the previous contract of the rental units of the items (latest other contract on the same units starting before, else an offer in draft; any other contract can be chosen), valid on its end (else today; for a draft its start) plus one-time terms. Only terms for an existing item with the same rental units, a term type allowed for this contract and not yet in the contract can be copied; notes for measured quantities, graduated/index rent (current amount only) and one-time terms. Marked lines are copied with the new unit price (editable, default the previous one) from the contract start, open end, new sequence and without cash flow; contract log `copy_terms`. The lines are built again on the server (the client sends only the copy flag and the new unit price)
- `add_log` appends timestamped `ContractLog` entries
- `next_item_sequence` / `next_term_sequence` auto-increment helpers
- `sequence` is only editable in `draft`; readonly once `running`/`terminated`/`cancelled`.

Contract type before type of use. `c_type` is picked first (its domain filters by the company's accounting scheme, `re_accounting`). `type_of_use` is only editable (Function field `c_type_multi_use`) once a contract type is chosen **and** it allows more than one type of use (its `types_of_use` `MultiSelection`); the selection is then restricted to the contract type's allowed values. If the contract type allows only one, `on_change_c_type` presets `type_of_use` to it (readonly).

`settlement_units` (Function field, `get_settlement_units`)

The contract's *last valid* settlement units: settlement units of the property's billing units whose objects overlap with the objects assigned to this contract via its items. Billing units are restricted

to the property's `next_billing_start_date` (the earliest non-billed billing unit start date); if the property has none set, the most recently `billed` period is used instead. The overlap is determined in one batch by `_participating_settlement_units` (billed units by their cost shares, the others by the approved objects of the property matching `reg_ex_object`) instead of the `objects` function field of every unit. Shown in the tab *Operation Costs* (participation only, no amounts), used by `get_cost_shares` and by `real_estate.contract.annex4.report` for the Anlage 4 print (see Reports) — the report intentionally reuses this field instead of re-deriving the billing unit itself.

Costs and shares are opened from the contract with the relates *Operating Costs Settlement Units* (wizard, all billing units of the property overlapping the contract period, `get_settlement_units_all_periods`), *Operating Costs Cost Shares* and *Operating Costs Settlement Results* (domain `contract`).

Tab *Tasks and Processes*: sub tabs *Open* (open tasks - '+' creates a task - and running processes) and *History* (done/cancelled tasks and processes), filtered One2Many fields `tasks_open`/`tasks_history` and `processes_open`/`processes_history`.

Fixed term / termination. `unlimited` (Boolean, default `True`) determines whether the contract has a fixed end date. Editable only in `draft`:

- `unlimited = True` (default) — `end_date` is readonly and must be empty; `on_change_unlimited` clears it automatically when toggled on.
- `unlimited = False` — `end_date` becomes required and editable (only in `draft`).
- The invariant ("unlimited contract must not have an end date", `msg_contract_unlimited_with_end_date`) is enforced by `validate_fields` only in `draft` — once `running`/`terminated` it is no longer checked, since termination legitimately sets `end_date` regardless of `unlimited` (see below).

The *Terminate* button (`real_estate.terminate_contract.wizard`, see *Wizards*) also writes the resolved `termination_date` into `end_date` — so `end_date` always reflects the contract's actual end once terminated, fixed-term or not.

The *Running* button is only shown for `draft`/`cancelled`. A **terminated** contract is reactivated via the *Revert Termination* button (`revert_termination`, transition `terminated → running`, only shown for `state = 'terminated'`). It clears the termination fields (`termination_date`, `terminated_by_type`, `receipt_of_termination_notice`, `termination_notice`, `termination_reason`) and, **only if the contract is "unlimited"**, also `end_date` — a fixed-term contract that was terminated early keeps its `end_date`.

See also the `update_contract_status` cron task (*Configuration* above) for the automatic `expired` termination of fixed-term contracts whose `end_date` has passed without an active termination.

Cancellation: the *Cancel* button transitions the contract to *Cancelled*. Before the transition:

- If any cash flow entry in state `done` (already invoiced) exists, a `ContractCancelWarning` is shown explaining that existing postings will **not** be reversed automatically. The user must confirm to proceed.
- All cash flow entries still in state `draft` are deleted.
- Occupancy records are recalculated; the cancelled contract is excluded.

`real_estate.contract.log` (`contract_core.py`)
Append-only audit log attached to a contract (event name + description).

`real_estate.contract.party.role` (`contract_party.py`)
Configurable role definitions for the "Parties" tab on a contract (e.g. *Hauptmieter* / Main Ten-

ant, *Nebenmieter* / Secondary Tenant, *Eigentümer* / Owner — shipped as default data), optionally restricted to specific contract types via the `contract_types` Many2Many (`real_estate.contract.party.role–contract.type`); left empty, a role applies to every contract type.

`mandatory`
: If set, this role must be assigned without gaps for the contract's *whole* validity period (`start_date` to its effective end date, or indefinitely if open-ended) — checked via `Contract._check_party_roles` (`validate_fields`, so on every create/write): a *warning* while the contract is still `draft` (the user may still be assembling the party list), a hard `ValidationError` otherwise.

`only_once`
: If set, this role may only be assigned once at any given point in time on a contract (no two overlapping `valid_from`/`valid_to` assignments) — same warning-in-draft/error-otherwise split as `mandatory`.

Confirmed warnings of both checks are remembered per role (not per contract): confirming a missing/overlapping role once also skips the warning for other draft contracts with the same gap.

`real_estate.contract.party` (`contract_party.py`)
: One party assignment row (`party`, `role`, `valid_from`/ `valid_to`) on a contract's "Parties" tab, plus `invoice_address` (domain-restricted to the assigned party's own addresses) and a read-only `phone_partner` (first phone contact mechanism of the party). A unique SQL constraint on `(party, contract, valid_from, role)` prevents exact duplicates.

`Contract.contractual_partner` is **derived** from whichever party currently holds the contract type's configured *Main Tenant Role* here, not set directly — `on_change_parties` keeps it in sync. It is a stored, required `Many2One` column (used by the *Kontenblatt* query `AccountContract.table_query()`).

`delete()` blocks removing the *last* assignment row for a party that still has booked (`state='done'`) cash flow entries on the contract (`msg_contract_party_delete_has_bookings`) — the party must stay traceable even if its specific role/period is being corrected, as long as some assignment row for it remains.

Change Partner (see Wizards) is the supported way to swap the main tenant on a running contract, including rebooking still-open invoices — editing the "Parties" tab directly for that purpose is still possible but only warns (`ContractPartnerChangedWarning`, confirmable) when the outgoing party already has booked entries, rather than doing any of the rebooking itself.

`real_estate.contract.item` (`contract_item.py`)
: Associates one or more rental objects with a contract for a given validity period (`label`, `valid_from`/`valid_to`).

`objects` is a `Many2Many` to `real_estate.base_object` via the relation model `real_estate.contract.item.object`; the client shows the objects with their normal list/form views (including `occupancy_state`). The object selector is restricted to objects of the **same property** as the contract, and additionally to `type = 'object'` for **occupancy contracts** (`contract.c_type.occupancy`, mirrored by the Function field `occupancy`). A unique SQL constraint on `(item, object)` prevents assigning the same object twice to the same item; the same object may be assigned to an item of another contract.

On create/write/delete, triggers occupancy refresh and re-runs BillingUnit selection and value-share calculation for the affected property.

`_check_occupancy_overlap` — only for occupancy contracts, only while the contract isn't `cancelled` — checks at save time, for every assigned object, whether this item's `valid_from`/`valid_to` range overlaps an existing `BaseObjectOccupancy` period of a *different* contract:

- Overlap with a `rented` period → `ValidationError` (`msg_occupancy_overlap`).
- Overlap with an `under_negotiation` period only → confirmable `ContractItemOccupancyWarning` (`msg_occupancy_overlap_warning`), remembered per object.

real_estate.contract.term (contract_term.py)

A recurring charge line on a contract (rent, operating cost advance, etc.).

Key fields: `term_type`, `reference_item`, `valid_from`/`valid_to`, `rhythm` + `rhythm_type` + `rhythm_start`, `quantity`, `unit`, `unit_price`, `taxes`, `payment_term` (optional, only visible for a term type with `separate_move`, Function field `term_type_separate_move`).

Graduated rent fields: `rent_adjustment` (the graduated rent the term belongs to), `graduated_step` (0 = base term, 1…n = steps), `graduated_amount` (agreed net amount of the step), `graduated_increase_percent` (information) and `graduated_locked` (Function field with searcher, see *Rent Adjustments*).

_split(term, valid_from, unit_price, quantity=None, values=None)

Common term split: ends `term` the day before `valid_from` and copies it into a follow-up term from `valid_from` on (cash flow and graduated rent membership are not copied). Refuses a date not after the term's start, after its end, inside its booked period (`get_booked_to()`) or when a follow-up term of the same type and item exists. Writes with `_skip_re_calc` - the caller recalculates the cash flow once afterwards.

Key methods:

re_calc()

Rebuilds the `CashFlow` list from existing invoice lines and projects future entries up to `_re_calc_year` years ahead.

_next_document_date()

Calculates the next invoice date based on rhythm and last posting date.

_on_change_with_next_due_date()

Applies the payment term of `Contract.get_move_payment_term()` to derive the due date.

real_estate.contract.term.tax (contract_term.py)

Many2Many relation table between `ContractTerm` and `account.tax`.

real_estate.contract.term.cash_flow (contract_term.py)

Projected or realised cash flow entry for a term.

States: `draft` (planned, no invoice line yet) · `done` (invoiced)

Invoice states (computed from the linked invoice): `draft` / `validated` — not yet posted (excluded from settlement calculations); `posted` — open receivable; `paid` — settled. Only `posted` and `paid` entries appear in the balance sheet and are used as *advance payment* in operating cost settlement.

Stores `posting_date`, `document_date`, `due_date`, and a link to the `account.invoice.line` once billed. `base_object` (function field) is derived from the linked invoice line and used to key advance payments to a (contract, object) pair during operating cost settlement. `create_moves_run_id` (format `YYYYMMDD-HHMMSS-U<userid>`) is set for all cash flow entries created in a single `CreateContractMoves` run, allowing traceability back to the wizard invocation. Supports date-pattern search (`YYYY/MM` or `YYYY/MM/DD`) in the name field.

Quantitative (contract_term.py)

Custom `fields.Numeric` subclass that carries a `unit` reference and sets `quantitative=True` in its field definition so the UI renders the associated unit symbol.

real_estate.contract.term.adjustment / real_estate.contract.rent_adjustment

Term adjustments and rent adjustments (graduated rent, index rent, …) - see Rent Adjustments.

real_estate.contract.handover (handover.py)
Handover report (*Übergabeprotokoll*) of a contract - see Tasks, Processes and Handover Reports.

2.8 Rent Adjustments

2.8.1 Overview

A rent adjustment (*Mietanpassung*) changes exactly one contract term by one procedure (`contract_type.ADJUSTMENT_PROCEDURES`):

- Graduated rent (*Staffelmiete*, § 557a BGB): all steps are generated in advance as terms.
- Index rent (*Indexmiete*, § 557b BGB): agreement on a price index; the adjustment run calculates the change, followed by approval, declaration letter, receipt and execution.
- Comparative rent (*Vergleichsmiete*, §§ 558-558b BGB): the adjustment run selects the terms, uses the accepted calculations of the rent survey, applies the cap and requests the consent of the tenant.
- **Modernisation, operating cost billing/plan, free adjustment**: header records so far.

A procedure is possible only if the contract type, the term type and the use class of at least one rental unit of the term's contract item allow it (`adjustment_procedures`; rental units without use class: all procedures except the comparative rent). By default the use class *Apartment* allows all procedures, *Office*, *Retail*, *Warehouse*, *Parking* and *Garage* all except the comparative rent and the modernisation - the comparative rent (§ 558 BGB) is only possible for apartments. At most one agreed procedure (graduated or index rent) per term.

Menus:

- *Real Estate › Contracts › Rent Adjustments*, in the order of the work: *Agreements* (the rent adjustments of all procedures), *Index Rents - Follow-up*, *Adjustment Runs* (the central entry, see *Adjustment run*), *Adjustments* (the contract term adjustments), *Comparative Rent Calculations* - all lists with the selection *Company* (required) and *Property* (optional) above the list; the adjustment runs show a run without properties (= all) for every property; columns *Property* (agreements, adjustments) and *Properties* (runs)
- *Real Estate › Configuration*: *Price Indices*, *Import Index Values*, *Index Rent Cap Rules*

On the contract: tab *Rent Adjustments*. Settings: `receipt_days` in the real estate accounting, scheduled task `price_index_import`, GENESIS token in `trytond.conf` (see Configuration and Installation). Tasks and the process templates of the adjustment run: see Tasks, Processes and Handover Reports.

2.8.2 Rent adjustment

real_estate.contract.rent_adjustment (contract_rent_adjustment.py)
Rent Adjustments (*Mietanpassungen*): one record per adjustment of one contract term, for every procedure, shown on the contract's *Rent Adjustments* tab (O2M `rent_adjustments`) and in the menu *Contracts* □ *Rent Adjustments*. A new record is created there via *New*: first the procedure, then the term.

Procedures (`contract_type.ADJUSTMENT_PROCEDURES`): `graduated_rent` (§ 557a BGB), `index_rent` (§ 557b), `comparative_rent` (§§ 558 ff.), `modernisation` (§§ 559 ff.), `operation_costs_billing` / `operation_costs_plan` (§ 560), `free_adjustment`. Graduated rent, index rent and comparative rent have full processing (see *Procedures*); modernisation, operating cost and free adjustment are header records so far.

Header fields (all procedures): `contract`, `procedure`, `term` (exactly one term - offered are terms of the contract whose contract type and term type both allow the procedure, and in state `draft`

no term locked by a graduated rent), `valid_from`, `agreement_date` (required for graduated/index rent), `written_form`, `comment`, `state` (`draft` / `generated` for a graduated rent, `draft` / `active` / `closed` for an index rent). Checks on save: term belongs to the contract, procedure allowed by contract type and term type, at most one agreed procedure (graduated or index rent) per term (§§ 557a, 557b BGB; a closed index rent no longer counts), no comparative rent for a residential term with an active index rent. Only drafts can be deleted.

2.8.3 Procedures

The procedures with their own processing are described on separate pages:

- Graduated Rent (*Staffelmiete*, § 557a BGB) - steps generated in advance as terms
- Index Rent (*Indexmiete*, § 557b BGB) - price indices, cap rules, agreement, adjustments, declaration
- Comparative Rent (Rent Survey) (*Vergleichsmiete*, §§ 558-558b BGB) - rent survey and calculation of the comparative rent, selection, cap, request for consent, consent

Modernisation, operating cost billing/plan and free adjustment are header records so far.

2.8.4 Adjustment run

`real_estate.contract.term.adjustment.run` (`adjustment_run.py`)

One process for all subsequent procedures (so far *Index Rent* and *Comparative Rent*), menu *Contracts › Rent Adjustments › Adjustment Runs* with the tabs *In Progress*, *Announced*, *Done*, *All*. A run (number `AL-<year>-<n>`) has the procedure, company, optional properties and contracts as filter, key date, planned declaration date and the parameters of the procedure (index: series, index month, include decreases; comparative rent: minimum increase in amount and percent, create calculations). Tabs *Workflow* (process of the run), *Parameters*, *Adjustments* (editable list: approval reason, manual amount with reason), *Receipts / Consents* (editable list of the announced adjustments: receipt date, dispatch method, consent), *Protocol* (dated block per step). *Print Letters* (announced/ready) prints all letters still to send in one document; the archived originals hang on the adjustment and on the contract. *Add Terms* (wizard, selected or calculated run) adds single terms with the checks of the selection.

Plausibility warnings (confirmable) when capturing receipt and consent: receipt before the declaration date or in the future, consent before the receipt - the effective date always follows the receipt.

Manual amount (both procedures): `manual_amount` with `override_reason` replaces the computed amount (`computed_amount`) - only between the current and the computed amount (A02 error, A01 hint).

Workflow: *Select* (draft selected; the procedure creates draft adjustments, every candidate gets a protocol line with the result or the cause of the skip; existing adjustments of the run stay - an adjustment cancelled in the run excludes its agreement from a new selection) *Calculate* (values and checks of the drafts) *Approve* (drafts without findings; with a warning only with an *Approval Reason*; errors and warnings without reason are cancelled and listed) *Announce* (letters of the approved adjustments, archived) the run becomes *Ready* as soon as every needed receipt is (and every consent decided; this automatic state - also back to *Announced* or on to *Done* - writes a protocol line and completes the process step) *Execute* (in the background: refused adjustments are closed at once, every consented one is executed by its own background job - an error is rolled back and written to the protocol, the adjustment stays announced; reload the run to see the result) (term split, refused adjustments are closed; the run is *Done* when no adjustment is open). *Reset to Draft* deletes the drafts, *Cancel* cancels the open adjustments and the process. All steps also work for several runs from the list.

Process: the template *Index Rent Adjustment Run* (code `adjustment_run_index_rent`) resp. *Comparative Rent Adjustment Run* (code `adjustment_run_comparative_rent`), model of the run, is started with the first selection; its six steps complete themselves with the states of the run (`run_selected` … `run_done`); the steps replace separate tasks per adjustment.

2.8.5 Term adjustments

`real_estate.contract.term.adjustment` (`contract_term.py`)

History record documenting a single old-term new-term adjustment (e.g. a rent/advance-payment change following an operating cost billing run, an operating cost plan, or a free percentage/absolute change).

Key fields: `adjustment_mode` (`percentage` / `absolute`), `term_old` (required, `ondelete='RESTRICT'`), `term_new` (`ondelete='RESTRICT'`, set once the replacement term exists), and an optional `settlement_result` link when the adjustment originates from an operating cost billing run.

Function fields mirror `company` / `property` / `contract` from `term_old`, and `valid_from` / `valid_to` / `amount` / `tax_amount` / `total_amount` from both `term_old` (suffix `_old`) and `term_new` (suffix _new), so a reviewer can compare the before/after amounts without opening either term individually.

List *Contracts › Rent Adjustments › Adjustments*.

Created by the adjustment run (see *Adjustment run*); so far for the index rent and the comparative rent - the operating cost, free adjustment and modernisation procedures will follow as procedures of the run. States `draft` · `approved` · `declared` · `done` · `refused` (comparative rent: consent refused) · `cancelled`.

2.9 Graduated Rent

Part of Rent Adjustments (common header, adjustment run, term adjustments). Procedure `graduated_rent` (*Staffelmiete*, § 557a BGB).

Graduated rent (`procedure = 'graduated_rent'`): all steps are generated in advance as terms. The procedure-specific fields are only visible (and required) for this procedure:

- Parameters: `rhythm_months` (≥ 12), `step_count` (≥ 1), `increase_mode` (`absolute` € per period, `per_area` € per m², `percent`), `increase_value`, `percent_basis` (`base` = same increase for all steps, `previous` = compound). `valid_from` is always the start of the base term.
- Area for `per_area`: the base term's informative measurement (`info_m_type`) as of its start, hierarchy-aware; fixed on generation (`generated_area`).
- Information: base amount (net, quantity × unit price), increase and increase % of the 1st step, final amount, total increase %, start of the last step, and a text preview of all steps (amount, increase and - with an informative area - the new amount per m²). Preview and generation use the same calculation `compute_amounts()`: the increase is rounded (half up) per step, every step is an exact cent amount.
- Steps tab: the step terms, booking state (`booked`, `last_booked_date`, `last_booked_step`), `predecessor` / `successor` (chain of follow-up graduated rents) and `generated_parameters` (snapshot of the last generation).

Buttons (`ir.model.button` records, restrictable per group):

Generate **(`draft → generated`)**

Checks G01-G15 (errors: procedure allowed, rhythm ≥ 12, ≥ 1 step, increase ≥ 0.01 per step, base term not booked, no follow-up term, last step before the term's/contract's end with the maximum

number of steps in the message, base term not an intermediate step of another graduated rent, contract draft/running without termination, written form for residential, area found, term type without `m_type`; confirmable warnings: written form for commercial, termination waiver over 4 years, rounding with quantity ≠ 1, step start not matching the billing rhythm). Then the base term becomes step 0 and each step is created with `ContractTerm._split()`; the last step takes over the original end of the base term (`base_valid_to_orig`). Cash flow is recalculated and the contract log gets parameters and step overview.

Regenerate
Not booked: reset and generate again. Booked: after the graduated rent warning (below), only the steps after the last booked step are recalculated from that step on; booked steps keep amount and start. Refused if the next step would start inside a booked period.

***Reset* (`generated` → `draft`, not booked)**
Deletes steps 1…n and restores the base term (original end, no membership).

***End Graduated Rent* (booked)**
After the warning, deletes the steps not yet booked; the last booked step becomes the last step (open, unlocked, `step_count` adjusted). If only the base term is booked, the graduated rent is dissolved like *Reset*.

Lock (`ContractTerm.graduated_locked`): the base term and all steps except the last one of a generated graduated rent - and the last step carrying a follow-up graduated rent - cannot change `valid_from`, `valid_to`, `term_type`, `reference_item`, rhythm fields or `taxes` (readonly in the form, muted rows in the list, note in the form), cannot be split and get no other rent adjustment (§ 557a para. 2 BGB). No term of a graduated rent can be deleted individually. Quantity and unit price can be corrected; `graduated_amount` follows. Only the graduated rent itself changes its terms (context `_graduated_rent_generate`).

After the first booking every change (regenerate, end, manual amount correction of a step) needs confirmation of `ContractGraduatedRentChangeWarning`: the graduated rent is part of the contract, a change requires an amendment agreed with the tenant. It comes before the existing warnings for booked terms, is keyed on the new parameters/values, and every confirmed change is logged in the contract log (old/new parameters, steps before/after).

Follow-up graduated rent: the last step of graduated rent A can be the base term of a new graduated rent B. The step stays a member of A and is step 0 of B only via `B.term`; while B exists (also as draft), A cannot be regenerated, reset or ended.

2.10 Index Rent

Part of Rent Adjustments (common header, adjustment run, term adjustments). Procedure `index_rent` (*Indexmiete*, § 557b BGB), processed by the adjustment run.

2.10.1 Price index and cap rules

`real_estate.price_index` / `real_estate.price_index.value`
Price index series for index rents (§ 557b BGB) with their monthly values per base year (menu *Configuration › Price Indices*). Default series `VPI-DE` (consumer price index Germany, Destatis table 61111-0002, base 2020, `residential_allowed` - the only index allowed for residential index rents) and `HVPI-DE` (manual template for commercial contracts). Values are imported with the wizard *Import Index Values* (CSV with `Month;Value` rows, month as `YYYY-MM` or `MM.YYYY`, or `Year;Month name;Value` rows of a GENESIS-Online table download; preview before import) or entered manually in the editable value list. Only `final` values of the series' current `base_year` count (`last_value_month`, `get_value()`). Maintenance is restricted to the administration group.

GENESIS-Online import (series with `source = 'destatis_genesis'` and `genesis_table`): buttons *Fetch Values* (from the year before the last value, catches revisions) and *Fetch from Base Year* on the series form (tab *Import*, administration only) and the scheduled task `price_index_import` (*Real Estate Accounting › Scheduled Tasks*, e.g. *Run on Day of Month* 20 - Destatis publishes the VPI in the middle of the following month). The web service is called by POST with the API token in the request header (`data/tablefile`, format `ffcsv`); only the index rows (unit YYYY=`100`) are imported (origin `import`), a different base year is refused, revisions of values used by declared or executed adjustments are not taken over. Result and errors are shown in *Last Import Message*; errors never abort the scheduled run. The token is configured in `trytond.conf` (not in the database):

```
[real_estate]
genesis_token = <API token from GENESIS-Online, menu Webservice (API)>
# optional
genesis_url = https://genesis.destatis.de/genesisWS/rest/2020/
genesis_timeout = 60
```

The Tryton server and the cron process need outgoing HTTPS access to `genesis.destatis.de`. A flat file CSV downloaded manually from GENESIS-Online can also be imported with *Import Index Values*.

`real_estate.index_cap_rule`
Prepared cap rule for index rents ("Mietrecht II", not in force yet; menu *Configuration › Index Rent Cap Rules*): validity of the received declaration (`valid_from`/`valid_to`), `threshold_percent` (default 3, counted in full per year), `excess_share_percent` (default 50) and `tight_market_only`. New rules are inactive by default. `apply()` computes the counted change in sections of 12 months (rest pro rata, compounded, no cap for decreases); `find()` returns the active rule for a receipt date and property. The tight housing market is marked on the property (`tight_market` with validity `tight_market_valid_from`/`tight_market_valid_to`, tab *General*).

2.10.2 Agreement and adjustments

Index rent (`procedure = 'index_rent'`, § 557b BGB): agreement, adjustment run, declaration, receipt and execution. Tab *Index*: `price_index` (residential: only series with `residential_allowed`, i.e. the VPI), `index_base_month` (first of the month) with the contract's own statement `index_base_value_contract` / `index_base_year_contract` (documentation) and the computed `index_base_value` from the current base year of the series, `threshold_type` / `threshold_value` (none / percent / index points), `effective_rule` (`statutory` - mandatory for residential - or `contract_month` with `effective_offset_months` for commercial), `apply_cap` (`auto` / `never`). Information: `current_term` (latest term of the chain of the same term type and item from `term` on, also after splits by other procedures), `current_index_month` / `current_index_value`, `last_change_date` (start of the agreement `valid_from` resp. later the last executed adjustment) and `next_possible_date` (+12 months, § 557b para. 2 BGB). `declaration_deadline` (*Declaration until*, searchable) is the follow-up date: latest date to send the declaration so that the adjustment takes effect on the next possible date (receipt in the month before last minus `receipt_days`; none for the commercial contract month rule). Menu *Contracts › Rent Adjustments › Index Rents - Follow-up* lists the active index rents with the tabs *Due* / *Next 3 Months* / *All*.

Activate (`draft → active`) checks: written form and agreement date (I01; residential error, commercial confirmable warning), statutory effective rule for residential, index allowed for residential (I02), final value of the base month in the current base year (I03), no comparative rent on the term chain and a confirmable warning for a modernisation adjustment (I10, residential), hint on the Preisklauselgesetz for commercial contracts not fixed for at least 10 years (I12). *Draft* (`active → draft`) and *Close Index Rent* (`active → closed`) change the state; in `active` index series, base month and term are readonly, threshold and comment stay editable. *Draft* is refused once an adjustment is approved, declared or done, *Close* while an adjustment is open.

Index adjustments (`real_estate.contract.term.adjustment` with `rent_adjustment`, `contract_index_rent.py`, list on the *Index* tab): created by the adjustment run (see Rent Adjustments, *Adjustment run*). Reference (`index_month_old`/`index_value_old`) is the agreement's current index (last executed adjustment, else the base month), the new value the run's index month - both from the current base year. `compute_index_adjustment` calculates the change (exact ratio, shown with 2 digits), the threshold (percent or points, on the uncapped change), the cap (`apply_cap = 'auto'`, active cap rule for the planned receipt date and the property's tight market), the new amount `planned_amount` = old amount × ratio (resp. × capped change), rounded half up to the cent - the decisive amount - and `planned_unit_price` = amount / quantity (4 digits). `planned_valid_from`: statutory first day of the month after next following `receipt_date` (before that a preview from `declaration_date` + `receipt_days` of the real estate accounting, default 3), for `contract_month` the index month + offset. Check protocol `check_state`/`check_message` (codes I04 lock period, I05 index values, I06 contract running/end, I08 rounding, I09 booked beyond the effective date, I13 decrease, I14 graduated rent lock, S01 threshold). States `draft → approved → declared → done`, `cancelled`: *Recompute* (draft), *Approve* (refused with errors, confirmable warning with findings), *Cancel* (warning if declared), *Draft* (from cancelled, at most one open adjustment per agreement). *Declare* (`approved → declared`, declaration date default today) renders the declaration `real_estate.contract.index_adjustment.letter` (`report/index_adjustment_letter_de.odt`: one letter per adjustment, addressed jointly to all main tenants, with index series and base year, reference and new index month/value, change in percent and points, cap rule if applied, old and new net rent and the difference, expected effective date) and archives the original as attachment (`letter`), a copy of the attachment also on the contract (same stored file); the values are frozen from then on. Mark: *ENTWURF* before the announcement, none while announced and the receipt is not captured (dispatch), *Zweitschrift* afterwards. *Declare* and *Execute* are buttons of the adjustment run only. Receipt date and dispatch method are entered in the adjustment form or the editable list of the tab *Receipts* of the adjustment run (declared adjustments). *Execute* (`declared → done`, for the statutory rule only with a receipt date) recalculates the effective date from the receipt, checks again (I04, I06, I09, I14, the current term must still be `term_old`) and I11 (the declared rent must not exceed the rent allowed by the cap rule on the actual receipt date), splits the term (`ContractTerm._split` with the new unit price from the effective date), recalculates the cash flow, sets `term_new`, `executed_by`/`executed_date` and logs the adjustment in the contract log (event `index_rent`). The agreement then takes the executed adjustment as reference for the next one. Index values used by a declared or executed adjustment cannot be changed or deleted.

Adjustment run with the procedure *Index Rent* (see Rent Adjustments, *Adjustment run*); declaration letter: see Reports.

2.11 Comparative Rent (Rent Survey)

Part of Rent Adjustments (common header, adjustment run, term adjustments). Procedure `comparative_rent` (*Vergleichsmiete*, §§ 558-558b BGB): the rent is increased up to the local comparative rent, which the module justifies exclusively with a rent survey (*Mietspiegel*, § 558a para. 2 no. 1 BGB). The other means of § 558a BGB (expert opinion, comparable apartments, rent database) are not supported. The page describes the rent survey with the calculation of the comparative rent first, then the rent increase procedure of the adjustment run.

The local comparative rent (*ortsübliche Vergleichsmiete*, § 558 BGB) of a residential rental unit is **calculated directly** from a rent survey (*Mietspiegel*) mapped in the system: classification by classes and features on the object tree, evaluation by the rules of the rent survey version. The result is reproducible, comes with a protocol and has to be accepted. There is no AI or external service involved.

2.11.1 Concept

All rent surveys map the statutory residential value features (kind, size, equipment, condition incl. energy, location) to a value in €/m² net rent with a range. Two methods are supported:

- **Table** (`method = 'table'`, e.g. Berlin): a table cell (classes such as location and age, plus an area range) gives lower, mean and upper value; each feature group with the rule *Majority* counts +1 / 0 / -1 (positive or negative features prevail) and moves the rent from the mean by `group_percent` (default 20 %) of the distance to the upper resp. lower value - netted (default) or separately; the result is limited to the range; groups with the rule *Additive* (special surcharges/reductions in €/m² or %) apply afterwards.
- **Regression** (`method = 'regression'`): the cell gives the base value; features add percentages and amounts; the range is the result ± the spread in %.

Classes and features are stored on the objects as **codes**, so they stay valid for the next version of the rent survey with the same codes.

2.11.2 Customizing

Menu *Configuration › Rent Surveys* (group *Real Estate Administration*; read for object, contract, billing and view group).

`real_estate.rent_survey` (`rent_survey.py`)
Rent survey (name, municipality, company) with its versions; the `current_version` is the version valid today.

`real_estate.rent_survey.version`
Version with `valid_from` (unique per survey; `valid_to` = day before the next version), survey date, kind (*Simple* / *Qualified*), method, rent definition, the living space measurement (default *Living Space*), further measurements for the protocol (default *Number of rooms*), `group_percent`, `group_netting`, spread (regression), rounding digits and `source_document` (reference/URL; the official document is attached to the version). Tabs: *General*, *Classification Features*, *Table Cells*, *Feature Groups*, *Check Result*. Buttons:

- *Check*: one class per classification feature in every cell, lower ≤ mean ≤ upper, no overlapping area ranges per class combination - the result is written to *Check Result*.
- *Import*: wizard for table cells or features from a CSV file with preview and problem list (import only without problems), optionally replacing the existing data.
- *New Version*: copies the version with classification features, classes, cells, groups, features, exclusions and class conditions (codes kept) for a new `valid_from` and opens it.

Once an accepted calculation uses the version it is **locked** (`locked`): classes, cells, features and rules are read-only; corrections by a new version.

`real_estate.rent_survey.dimension` / `…dimension.class`
Classification feature (table axis, e.g. `location`, `age`) with `level` (property, building, rental unit), `source` (*Manual*, *Year of Construction* of the building, *Measurement*) and classes with code, name, value range `value_min ≤ value < value_max` for derived classes and `manual_only` (e.g. special classes, never derived).

`real_estate.rent_survey.cell`
Table cell: code (row of the table), exactly one class per classification feature, area range `area_min ≤ living space < area_max` (empty = open), lower / mean / upper value, `qualified`, note.

`real_estate.rent_survey.group` / `…feature`
Feature group (code, name, rule *Majority* / *Additive*) with features: code (unique per version), name, wording, direction (+/–), effect (*Vote*, *Percent*, *Amount per m²*) and value, level, *Only for*

Classes (e.g. only up to construction year 2009), automatic setting by a measurement range (`auto_measurement_type`, `auto_min ≤ value < auto_max`, e.g. energy consumption value) and *Excludes* (features excluding each other).

CSV formats (separator `;`, decimal comma or point, a first line starting with `code` resp. `group_code` is the header):

```
code;dims;area_min;area_max;lower;mean;upper;qualified;note
85;location=medium,age=1950_1964;45;;5,90;7,08;9,25;1;

group_code;code;direction;effect;value;level;applies_to;auto_measurement;auto_min;auto_max
5;u_quiet;plus;vote;;property;;;;;Besonders ruhige Lage;;u_noise
S;s_minor_equipment;minus;amount;0,33;object;age=to_1918;;;;Minderausstattung;;
```

Missing feature groups are created by the feature import (*Majority* for votes, else *Additive*); `auto_measurement` is the XML id of a delivered measurement type (e.g. `measurement_energy_consumption_type`, language-independent) or the name of the measurement type.

2.11.3 Berlin rent survey 2026

The complete Berlin rent survey 2026 is provided as CSV files taken from the official documents (*Berliner Mietspiegel 2026*, *Berliner Mietspiegeltabelle 2026*):

- `tests/mietspiegel_berlin_2026_felder.csv`: all 189 table cells of the tables 9.1-9.3 (simple, medium, good location × 12 classes of construction age × area ranges, all within the qualified scope).
- `tests/mietspiegel_berlin_2026_merkmale.csv`: the guidance for the classification within the range (no. 11) with 85 features in the groups 1-5 - class conditions such as *only up to 2001* or *not for 1973-1990 East*, the energy consumption value stages as automatic, cumulative features (e.g. 90 kWh/(m²a) counts < *120* and < *100*), *either insulation or energy value* resp. *either old heating or energy value* as exclusions - plus the reduction of 0.33 €/m² for minor equipment up to 1949 (no. 9.4, group S, not qualified).

`python tests/test_rent_survey.py --database <db> [--config trytond.conf] --only-survey` creates the rent survey with the version 2026 (classification features *Wohnlage* and *Bezugsfertigkeit* with their classes; 1973-1985 West, 1986-1990 West and 1973-1990 East are manual only as they depend on the district) and imports both files; without `--only-survey` it also assigns demo data (see Demo data scripts). The data have to be checked against the original before productive use; the location of an address is taken from the street directory.

2.11.4 Data on the object tree

Tab *Rent Survey* of the object form (property always; building and apartment once the property has a rent survey). An **apartment** is a rental unit whose use class allows the procedure *Comparative Rent* (*Allowed Adjustment Procedures* of the use class, by default only *Apartment*) - parking spaces, garages and commercial units have no rent survey data, no buttons and no calculation:

- `rent_survey` (property): the rent survey of the residential rental units of the property.
- *Rent Survey Classes and Features* (`real_estate.base_object .rent_survey_value`): editable list of classes and features of this level with validity period and evidence; the selection offers the classes of the version valid today whose classification feature belongs to the level of the object or a higher one (e.g. the age class on the building or the rental unit, not on the property; the search list shows the classification feature) resp. the features of the level; stored are the codes.

- *Rent Survey Data*: remarks and reasons (e.g. reference in the street directory) - shown in the inputs of the calculation, not used for it.
- Button *Assign Features*: checklist of the features of the level; newly checked features are created without validity period, unchecked ones deleted (without period) resp. ended the day before the key date.
- Buttons *Export Matrix* / *Import Matrix* (property): CSV matrix of the property for a key date with one row per apartment resp. building (column `id`, name) and one column per classification feature assignable on that level (`class:<code>`, value = class code) and per feature of the level (code, `x` = applies); a second line starting with # holds the names. The file (UTF-8 with BOM, separator `;`) can be changed in a spreadsheet and imported again: compared with the own assignments valid on the key date, missing ones are created, removed ones deleted (without period) resp. ended the day before (with a period starting before the key date); assignments of codes not in the matrix stay untouched. Preview with the changes per object and the problems (unknown columns, objects, classes or marks); applied only without problems.
- Button *Copy from Building* (building): the same for the building - classes and features of another building of the same property.
- Button *Copy from Rental Unit* (rental unit): wizard to take over the features - optionally also the classes - of another rental unit of the same property valid on a key date (with validity period and evidence); only missing ones are added, a class of the same classification feature is kept, or with *Replace Existing* the current assignments are removed first (with a validity period: ended the day before).
- Button *Calculate Comparative Rent* (residential rental unit) and the list of its calculations.

Inheritance: a class on a lower level overrides the class of a higher level (rental unit > building > property); **features add up** over the three levels. The measurement type *Energy Consumption Value* (building, kWh/(m²a)) holds the value already converted (demand value + 20 %, + 20 kWh/(m²a) without hot water); the conversion is noted in the evidence or the rent survey data.

2.11.5 Calculation

`real_estate.rent_survey.calculation` - menu *Contracts › Rent Adjustments › Comparative Rent Calculations* (tabs *Draft*, *Calculated*, *Accepted*, *All*), tab *Rent Survey* of the rental unit. One calculation = one residential rental unit on a key date; optional references to contract, term and rent adjustment (procedure *Comparative Rent*).

Calculate:

1. version valid on the key date; living space (sum measurement) on the key date;
2. class per classification feature: manual assignment of the lowest level valid on the key date, else derived from the year of construction resp. the measurement (classes *Manual Only* are never derived);
3. table cell matching all classes and the living space;
4. features: assigned ones of the three levels valid on the key date plus automatic ones within their measurement range, minus those whose class condition does not match (listed as ignored);
5. method table resp. regression (see *Concept*), rounding;
6. protocol, snapshot of all inputs (`inputs_json`: classes with origin and level, features with source, measurements, year of construction, rent survey data), group results, checks.

Checks: B01 residential rental unit, rent survey and valid version; B02 living space; B03 one class per classification feature; B04 exactly one table cell; B05 cell outside the qualified scope (warning); B06 excluding features; B07 assigned code missing in the version (warning); B08 group without feature (note); B09 missing measurement for automatic features (warning). With an error the calculation stays *Draft* with protocol and messages; else it is *Calculated*.

Accept (mandatory step, from *Calculated*) with the accepted rent per m² (default the calculated one); a reason is required with warnings or a different rent. *Reject*. *Calculate Comparative Rent* on the rental unit and *Recalculate* calculate an open calculation (draft or calculated) of the same rental unit and key date again with the current data instead of creating another one; a new calculation is created only once the existing ones are accepted or rejected - so there is at most one open calculation per rental unit and key date. Calculated ones only accept the acceptance fields, accepted and rejected ones are read-only; accepted calculations can never be deleted (evidence, they lock the version), all others can (group *Real Estate Contract*). The comparative rent is rent per m² × living space.

Access: customizing *Real Estate Administration*; assignments on the objects *Real Estate Object*; calculating and accepting *Real Estate Contract*; reading *Real Estate View*.

The procedure *Comparative Rent* of the adjustment run (cap, periods, consent, request letter) builds on the accepted calculation: the run creates the calculations of the selected apartments on its key date, the button *Calculations* of the run opens them for the acceptance - see *Rent increase procedure* below. The calculation protocol (with translated levels) is part of the request letter.

2.11.6 New letting check

Button *Check Comparative Rent (New Letting)* (tab *Terms*) on a draft residential contract whose property has a rent survey (wizard `real_estate.contract.new_letting_check`): for every monthly rent term of exactly one apartment (term type allowing the comparative rent, valid on the contract start) the comparative rent on the contract start is calculated without storing a calculation. Limit = comparative rent + 10 % if the property is marked as tight housing market on that date (rent brake, § 556d BGB), else + 20 % as orientation (rent overcharge, § 5 WiStG), rounded down to the cent. The wizard lists living space, rent per m², comparative rent, limit, current rent, deviation and result; *Adjust* (first column) per line sets the term to the limit (rows above the limit are shown in red; up or down; for an area quantity the unit price is limit / quantity) and logs it in the contract log. The exceptions of §§ 556e, 556f BGB (previous rent, modernisation, new building) are only mentioned in the information text.

2.11.7 Rent increase procedure

Comparative rent (`procedure = 'comparative_rent'`, §§ 558-558b BGB, `contract_comparative_rent.py`): increase of the net rent up to the local comparative rent of the rent survey, only for apartments (use class), never for a term with an active index rent or during a graduated rent - after the last step the waiting period counts from its start.

Agreement: the adjustment run creates one active rent adjustment `comparative_rent` per term chain (term = first term of the chain, `valid_from` = its start) when a term is selected the first time; it keeps the history of the adjustments (lock of one year). *Close* ends it (e.g. at the end of the contract), *Draft*/*Activate* as for the index rent. Tab *Adjustments* of the agreement: current term and adjustments.

Cap (§ 558 para. 3 BGB): 20 % in three years, 15 % for a property with `reduced_cap` (*Reduced Cap 15 % (Regulation)* with validity `reduced_cap_valid_from`/`reduced_cap_valid_to`, tab *General* of the property, `BaseObject.cap_percent(date)`).

Selection (procedure `ComparativeRentProcedure`): current terms of running contracts of the filters whose contract type and term type allow the procedure, monthly rhythm (`rhythm_type = 'monthly'`, `rhythm = 1`) and use class of the rental units. Excluded with a protocol line: V01 active index rent on the chain, term locked by a graduated rent or starting after the effective date (graduated rent running; after its last step V03 applies), V02 open adjustment, V05 not exactly one apartment in the item, V06 contract ends before the effective date, V03 rent changed less than 15 months before the effective date (changes by modernisation and operating cost adjustments do not count; protocol with the earliest date), V04 last request less than one year before the declaration date. With *Create Calculations* (default) the run creates or recalculates the calculation of the rent survey of the apartment on the key date (`calculate_for`); the button *Calculations* of the run opens them for the common acceptance.

Calculation (`_comparative_compute`): effective date `t` = start of the third month after the receipt (before: declaration date + receipt days), comparative rent `M_vgl` = accepted calculation (V07: with *Create Calculations* the calculation on the key date of the run, calculated again by the run - an older accepted calculation is not used; without it the latest accepted calculation, key date at most one year before the declaration date), rent three years before `t` (`M_3J`, else the initial rent), increases not counted `E_mod` (modernisation / operating cost adjustments within the three years), cap `M_kap = M_3J × (1 + q) + E_mod`, new rent = lower of `M_vgl` and `M_kap`. Fields `calculation`, `comparative_amount`, `cap_base_amount`, `cap_excluded_amount`, `cap_percent`, `cap_amount`, `limited_by`, `calculation_text` (tab *Comparative Rent* of the adjustment). Checks: V01, V03, V04, V06 again, V07 accepted calculation (error), V08 calculation with warnings or cell outside the qualified part (warning), V09 no increase or below the minimum increase of the run (error), V10 the cap applies (hint), V11 term type with operating costs (error), V12 booked beyond the effective date (warning).

Request for consent: report `real_estate.contract.comparative_rent.letter` (`report/comparative_rent_letter_de.odt`) with the current and the new net rent, the effective date, the reasons from the calculation (rent survey, version, table cell with lower/mean/upper value, classification of the span from the calculation protocol, a deviating accepted rent with its reason), the cap and the period for consideration, plus a consent form. *Announce* archives the original at the adjustment.

Receipt and consent (tab *Receipts* / *Consents* of the run): receipt date (`consent_state = 'pending'`, task *Consent period of the rent increase ends* at `consent_deadline` = end of the second month after the receipt), then `consent_state` *Consented* / *Partially Consented* (with `consented_amount` between current and requested rent) / *Refused* with `consent_date`. `lawsuit_deadline` = three months after the consent deadline. The run is *Ready* when every decision is captured. *Execute*: consented adjustments split the term at the effective date with the requested resp. consented amount (contract log `comparative_rent`), refused ones get the state *Refused* (no split, task *Check the action for consent* 14 days before the lawsuit deadline, contract log). The rule M09 *Consent period expired* creates the same task when no decision is captured within the period.

2.12 Operating Cost Settlement

Sub-topics on separate pages:

- CO2 Cost Allocation (CO2KostAufG) - split of the CO2 cost of heating fuel between tenant and landlord
- BVED External Billing Interface - data exchange with external metering service providers

`real_estate.cost_category_group` (`billing_unit.py`)
Groups cost types for reporting, e.g. *Heating*, *Water*, *Janitorial*.

`real_estate.cost_type` (`billing_unit.py`)
Individual cost item (e.g. *Gas*, *Cold Water*) with optional `category_group`, `comment`, and `no_print` flag.

For `allocation_by_consumption` settlement units, two fields control the meter-reading tolerance window around the start/end date of each cost share:

`reading_pre_days`
Days *before* the target date within which a reading is accepted (default: 7).

`reading_post_days`
Days *after* the target date within which a reading is accepted (default: 7).

The reading closest to the target date within the window is used. If no reading is found, an error is stored on the cost share.

real_estate.billing_unit (billing_unit.py)

Annual (or period) operating cost settlement for one property.

Workflow: Draft → Approved → Selection → Value Share → Ready for Billing → Billed

external_billing

Boolean flag. When enabled, every settlement unit of the billing unit is forced to `allocation_rule = allocation_from_external_billing` (see `real_estate.settlement_unit` below) and costs are entered manually instead of being computed internally. Also reveals the *BVED* page and its `bved_provider_assignment` field — see *BVED External Billing Interface* below.

Two calculation methods:

rental_apartment

Operating cost settlement for residential tenancies under §§ 1–2 BetrKV. Costs are allocated to tenants proportionally.

WEG_billing

Annual statement for condominium owners under WEG (German condo law). Costs are allocated to co-owners by ownership share.

Key actions (buttons):

approved

Activates the billing unit.

selection

Identifies which contracts/objects are in scope for the billing period. Available in states `approved`, `selection`, `value_share`, and `ready_for_billing` (re-running from `value_share` / `ready_for_billing` revises the scope; existing settlement results are deleted after confirmation).

compute_value_shares_button

Calculates allocation shares (`CostShare`) for each settlement unit. Available in states `selection`, `value_share`, and `ready_for_billing`. If settlement results already exist for the billing unit, a confirmation warning is shown before they are deleted and value shares are recomputed. Does **not** automatically call `compute_settlement_result`. If any settlement unit has `sub_state = error` the billing unit is *not* advanced to `value_share`; the error is written to the billing unit log instead.

compute_settlement_result

Derives `SettlementResult` records per (contract, base_object) pair — actual costs, advances paid (only `posted`/`paid` invoice lines), and the resulting refund or additional receivable. Advance payments are matched to cost shares by the same (contract, object) key. If an advance-payment cash flow line has no matching cost-share group (e.g. no object on older invoices, or several terms for the same object) a fallback `SettlementResult` with `actual_costs = 0` and a full refund is created for it; the fallback count is logged. Sets affected `CostShare` records to state `error` if any cash flow entries in the period are still in `draft`/`validated` state; the error message is prefixed with `[draft]` so it can be reset on re-run once the drafts are posted or deleted. Resets a billing unit already in state `ready_for_billing` back to `value_share` before recomputing.

check_ready_for_billing

Transitions `value_share` → `ready_for_billing` after validating:

- no settlement unit has `sub_state = error`;
- every settlement unit (except `no_allocation`) has reached `sub_state = value_share`;

- at least one `approved` `SettlementResult` exists and none has `actual_costs = 0`;
- every settlement result with a non-zero advance payment has both a `term` and a `contract` (a missing term usually means several term types contributed and the billing unit's term-type filter needs narrowing);
- all advance-payment invoices are posted;
- (unless `external_billing`) the sum of settlement result actual costs matches the sum of cost share actual costs.

Any failed check raises a `ValidationError` listing the offending records. Can also be triggered per property via `BaseObject.ready_for_billing_property` for all billing units sharing `next_billing_start_date`.

`billing_wizard` / `billing`
The form button `billing_wizard` opens the `real_estate.billing_unit.wizard` (see Wizards), which calls the `billing` classmethod. `billing` creates, per settlement result, up to two invoice lines — advance-payment dissolution and actual costs — with taxes copied from the advance-payment term, then one invoice per contract. Invoices are created in state `draft` and posted immediately if the wizard's `invoice_state` is `posted`. Each invoice's `payment_term` (and therefore its due date) is taken from the wizard's `payment_term` field when set; otherwise the contract's own payment term is used, falling back to `account.configuration`'s `re_payment_term_billing`. Both `invoice_date` and `accounting_date` are set to the wizard's `invoice_date` (analogous to the `accounting_date` on contract invoices generated by `CreateContractMovesWizard`), so the posting date matches the invoice date rather than the date the wizard was run. Refuses to proceed if `sub_state` is `error` or if draft cash flow lines still exist in the settlement period; with `collective_billing` all billing units of the property sharing the same start date must already be `ready_for_billing`. Generates a `billing_run_id` (YYYYMMDD–HHMMSS–U<userid>) that is written to the billing unit and all `BillingUnitMoves` records of the same run, so every posting can be traced back to its run. Hidden when the property has `collective_billing = True`; in that case the action is triggered from the property form instead (`BaseObject.billing_property` → same wizard → `BaseObject.do_billing`).

`billing_run_id`
Char field set at the end of a successful `billing()` call, identical to the value written to all `BillingUnitMoves` of that run.

`rental_objects`
(Function, One2Many, readonly.) Distinct rental objects referenced by this billing unit's own `settlment_results` (via `settlement_result_objects()`) — the objects that actually ended up with a settlement result, as opposed to `bved_covered_object_ids()` (derived from `settlement_unit.objects` and used for BVED A-/D-Satz matching, which can include an object excluded during `selection()`, e.g. a genuine vacancy edge case). Shown as a read-only *Rental Objects* tab, right after *Settlement Results*.

`covered_rental_objects()`
Method (not a stored/Function field) used by the BVED L-Satz "Gesamtfläche" calculation — see *BVED External Billing Interface* below: returns `settlement_result_objects()` when non-empty, else falls back to the objects covered by this billing unit's own settlement units (`bved_covered_object_ids()`) for a billing unit that hasn't had "Compute Settlement Results" run yet. External callers (BVED included) always go through this one method rather than re-deriving the object set themselves.

`real_estate.billing_unit.log` (`billing_unit.py`)
Timestamped log entries attached to a billing unit.

real_estate.billing_unit.moves (billing_unit.py)

One record per invoice line created by `billing()` — a separate record for the advance-payment dissolution line and for the actual-costs line of the same settlement result (not combined into one record). Links the billing unit to the contract, settlement result, and the individual posting record:

- `moves_advanced_payment` — `account.invoice.line` that reverses the operating-cost advance (credit note line)
- `moves_actual_costs` — `account.invoice.line` for the actual costs
- `moves_alloc_by_owner` — `account.move.line` for the owner allocation journal entry

Function fields `amount`, `currency`, and `invoice` are derived from whichever of `moves_advanced_payment` / `moves_actual_costs` is set, for quick display in the tree view without opening the invoice line. For vacancy records (no `contract`, but `moves_alloc_by_owner` set), `amount` is instead computed as `debit − credit` of that `account.move.line`, `currency` from its own `currency` field, and `invoice` stays empty (move lines have no invoice reference).

`billing_run_id` ties all moves of one billing run together (same value as on the parent `BillingUnit`). Browseable via the *Billing Unit Moves* menu entry under *Operation Costs*, filtered by company, property, billing unit, contract, and date range.

real_estate.settlement_unit (settlement_unit.py)

One cost-type line within a billing unit, e.g. *Gas 2024*.

Allocation rules:

no_allocation
Entire cost stays unallocated.

allocation_by_measurement
Allocated by a measurement value, e.g. living area. `value_share` = `measurement_value × time_share / time_total` (time-weighted: a tenant occupying only part of the period receives a proportionally smaller share).

allocation_by_consumption
Allocated by meter reading consumption (HeizkostenV). `value_share` = raw consumption (no time weighting applied).

Meter readings are looked up within a tolerance window defined by `reading_pre_days` / `reading_post_days` on the cost type; the closest reading to the target date is chosen.

Vacancy handling: cost shares without a contract receive `value_share = 0` automatically — no meter reading is required. For the immediately following contract cost share, if no start reading exists within the normal window, the system falls back to the reading taken at the start of the preceding vacancy. This allows a single read-out at move-out to serve as both the vacancy baseline and the contract's opening reading.

Missing readings set the cost share to state `error` with one of:

- *"Messwert für Anfangsverbrauch nicht ermittelt"* — start reading absent
- *"Messwert für Endverbrauch nicht ermittelt"* — end reading absent

allocation_per_rental_unit
Proportional share by occupancy duration. `value_share` = `time_share / time_total` (full period = 1.0, half period = 0.5, etc.).

allocation_from_external_billing
No internal calculation; `planned_costs`/`actual_costs` are entered manually on the cost share. Automatically suggested (via `on_change_external_billing`) on every settlement unit of a billing unit when that billing unit's `external_billing` flag

is enabled (see real_estate.billing_unit below) — except units already set to allocation_via_cost_collector, which are left alone. Enforced (not just suggested) by validate_fields: for an externally billed billing unit, every settlement unit's allocation_rule must be either this or allocation_via_cost_collector.

allocation_via_cost_collector
No allocation of its own. Used to route one settlement unit's costs into another settlement unit of the *same* billing unit (its reference_settlement_unit, required for this rule and restricted to a unit that does not itself use this rule — no chaining) for allocation there instead. Also selectable when the billing unit has external_billing enabled, as an alternative to allocation_from_external_billing (e.g. several fuel suppliers, each invoiced on their own settlement unit, feeding one externally billed heating settlement unit).

A settlement unit with this rule:

- determines no rental objects of its own — objects always mirrors reference_settlement_unit.objects;
- generates no cost shares and is excluded from Contract.get_settlement_units() (so it never appears twice, once under its own cost type and once under the reference's, in e.g. the Annex 4 report);
- still tracks its own planned_costs/actual_costs (the latter from its own invoice_lines, via the normal selection_actual_costs()) — these are summed, across *all* settlement units referencing the same target, into the target's planned_costs_from_references/actual_costs_from_references (Function fields, see below) and folded into the target's own planned_costs/actual_costs when it computes its value shares.

BillingUnit.compute_value_shares_button processes all allocation_via_cost_collector units of a billing unit before any other settlement unit, so a reference target always sees its referencing units' up-to-date actual costs. BillingUnit.duplicate_next_period copies reference_settlement_unit to point at the *newly created* successor of the original target (not the old period's unit) — non-cost-collector units are always copied first so that successor already exists by the time a referencing unit is copied.

planned_costs_from_references / actual_costs_from_references
(Function fields, any settlement unit) Sum of planned_costs/ actual_costs of all settlement units of the same billing unit that reference this one via allocation_via_cost_collector. Zero when nothing references this unit.

Vacancy handling: unoccupied periods can be charged to the owner or left unallocated.

When compute_value_shares is re-run, cost shares in state error are automatically reset to selection and their error_message cleared before the new calculation starts, so corrected readings or measurements take effect immediately.

Heating cost billing split (HeizkostenV §7/§8):

heating_billing_mode
Selection: none (default) / central_heating / central_hot_water. Independent of allocation_rule/m_type above (which govern how *this* settlement unit's own costs are allocated per tenant) — this models the legally mandated split, for a centrally supplied heating or hot water system, between the consumption-based and area-based portions of the heating cost statement itself.

heating_consumption_share_percent
Numeric percentage entered by the user: the share of heating/hot water cost allocated by consumption ("Anteil nach Verbrauchsumlage"). Required and visible only when heating_billing_mode is central_heating or central_hot_water; domain-restricted

to 50–70 (statutory range, HeizkostenV §7/§8) — but only while one of those two modes is selected, so a value entered under a previous mode (or left over from an import) never blocks saving once the mode is set back to `none`.

`heating_area_share_percent` (Function field)
Computed as `100 − heating_consumption_share_percent` ("Anteil nach Flächenumlage"); read-only, same visibility as above.

`heating_area_measurement_type`
Many2One to `real_estate.measurement.type` (object-level measurement types only): which measurement to use for the area-based portion of the heating cost split. Required and visible only under the same condition as the two fields above.

`total_value_consumption` / `total_value_area`
Sum, across all of this unit's cost shares, of their `consumption_share` / `area_share` (see `real_estate.cost_share` below). Read-only, visible only when `heating_billing_mode` is `central_heating`/`central_hot_water`. Computed by `_compute_heizkostenv_split()`, called at the end of `compute_value_shares()` (both the regular per-cost-share path and both branches of `_compute_value_shares_external()`) — i.e. whenever *Compute Value Shares* is run on the billing unit.

`real_estate.cost_share` (`settlement_result.py`)
Intermediate allocation record: share of a specific cost type for one contract or object within a billing period.

States: `preparation` · `selection` · `estimated_value_share` · `value_share` · `error`

Stores `value_share` (time-weighted allocation factor — for `allocation_by_measurement` and `allocation_per_rental_unit` this already incorporates the occupancy fraction `time_share / time_total`; for `allocation_by_consumption` it holds the raw consumption value, *unless* the settlement unit is HeizkostenV-split — see `consumption_share` below), `time_share` (days), `planned_costs`, `actual_costs`. `error_message` describes the problem; entries set by `compute_settlement_result` carry a `[draft]` prefix and are automatically cleared on the next successful re-run.

`allocation_rule` / `external_billing` (Function fields, mirrored)
Mirrored from the parent `settlement_unit`. `planned_costs`/`actual_costs` are only editable (`readonly` otherwise) when `external_billing` is set, i.e. for cost shares of a settlement unit with `allocation_rule = allocation_from_external_billing`.

`heating_billing_mode` (Function field, mirrored)
Mirrored from the parent `settlement_unit`; drives the visibility of the two fields below (only visible for `central_heating`/ `central_hot_water`).

`consumption_share` / `area_share`
The two HeizkostenV split components, computed by `SettlementUnit._compute_heizkostenv_split()` (called at the end of `compute_value_shares()`) only when the parent settlement unit's `heating_billing_mode` is `central_heating`/ `central_hot_water`:

- `consumption_share` — only for `allocation_rule = allocation_by_consumption`: the raw consumption value that would otherwise sit in `value_share` is moved here instead, and `value_share` is reset to `0` (so a HeizkostenV-split consumption-based unit currently distributes no `planned_costs`/`actual_costs` via `compute_value_shares()`'s own `_distribute()` step — a deliberate, temporary state pending a follow-up that recombines `consumption_share`/`area_share` via `heating_consumption_share_percent`/`heating_area_share_percent` into the actual cost distribution).

- `area_share` — computed for *every* cost share of a HeizkostenV-split unit regardless of its `allocation_rule`: the value of the unit's own `heating_area_measurement_type` for the cost share's `base_object`, weighted by `time_share / settlement_unit.time_total` (same weighting as `allocation_by_measurement` above).

Both values are summed into the parent settlement unit's `total_value_consumption/total_value_area`.

`real_estate.settlement_result` (`settlement_result.py`)

Final settlement record per contract (or object) and billing unit.

States: `approved` · `billed`

Stores `planned_costs`, `actual_costs`, `advanced_payment` (sum of `posted`/`paid` operating-cost advance invoice lines during the period), and the derived `refund_receivable` (positive = refund to tenant, negative = additional receivable). Optionally links to the single `ContractTerm` that covered all advances when exactly one term was involved. The display name includes the record id in parentheses (e.g. `"2026-01-01 – 2026-12-31 / Mieter 1 (42)"`) to disambiguate several results for the same contract/object; the id is also searchable via the name field.

2.13 CO2 Cost Allocation (CO2KostAufG)

Part of Operating Cost Settlement.

Implements the German *CO2-Kostenaufteilungsgesetz* (CO2KostAufG), which splits the CO2 cost of heating fuel between tenant and landlord depending on the building's emission intensity (kg CO2/m²/year). Applies via an optional `co2_kostaufg` reference on one or more of a billing unit's settlement units (typically its *Heizkosten* settlement unit) — settlement units without a reference are unaffected, and a billing unit with none of its settlement units referencing one gets no *CO2 Costs* page at all.

Note

The reference and the raw consumption-row table live on `real_estate.settlement_unit`, but every *derived* figure (totals, area, per-m² emission, tenant/landlord shares) is aggregated on the parent `real_estate.billing_unit` instead — across **all** of its settlement units that carry a `co2_kostaufg` reference, not just one. This matters when a billing unit's costs are split across more than one CO2KostAufG source (e.g. two heating circuits, two fuel deliveries under separate master records): the billing unit's total consumption/emission/cost figures, and the object set used for the area basis, are the union across all of them, not just whichever settlement unit happens to be entered first.

`real_estate.co2_kostaufg` (`co2_kostaufg.py`)

Master record for one CO2KostAufG data source (e.g. one heating installation/fuel), scoped to a `property` (required, `type = 'property'`) with `company` derived from it (Function field). Referenced from one or more settlement units of the same property via their `co2_kostaufg` field (domain-restricted to that property — a settlement unit can only pick a CO2KostAufG belonging to its own property). Menu: *Real Estate* → *Operation Costs* → *CO2 Kosten verwalten*.

`real_estate.co2_kostaufg.consumption` (`co2_kostaufg.py`)

Child records under a `co2_kostaufg` (`parent`, `ondelete='CASCADE'`), one per billed/metered period: `date_from`/`date_to`, `consumption_kwh`, `co2_kg_per_kwh` (emission factor), `co2_emission_kg`, `co2_price_ct_per_kwh`, `vat_rate`, `co2_cost_net`, `co2_cost_gross`.

`co2_emission_kg` is pre-filled (`consumption_kwh` × `co2_kg_per_kwh`) only while still empty — once any value exists (auto-filled or typed by the user), later changes to `consumption_kwh`/`co2_kg_per_kwh` never overwrite it again, so it is always freely (re-)editable,

e.g. to enter the supplier's own figure. `co2_cost_net`/`co2_cost_gross` behave differently **on purpose**: they are always recomputed from `consumption_kwh`/`co2_price_ct_per_kwh`/`vat_rate` whenever any of those three change; a manual override of net/gross only sticks until the next such recompute.

`split`
Boolean. Set automatically by `SettlementUnit.compute_value_shares()` when this record was created by splitting an original record at a settlement-period boundary — see *Splitting at settlement-period boundaries* below.

Uses `DeactivableMixin` — a record replaced by a split is deactivated (`active = False`), never physically deleted.

`real_estate.co2_emission_share` (`co2_kostaufg.py`)
Configuration table for the legal 10-tier distribution model (Anlage 1 CO2KostAufG, residential rented buildings without separate sub-metering by usage type): `emission_limit` (kg CO2/m²/year — the *exclusive* upper bound of the tier), `tenant_share` / `landlord_share` (%, must sum to 100 — enforced in `validate()`). `get_share(value)` returns the first row, in `sequence` order, whose `emission_limit` is strictly greater than `value`; if `value` reaches or exceeds every configured limit, the last row (by sequence) is used as an open-ended top tier regardless of its own stored limit value. Default data for all 10 legal tiers is loaded at module installation (< 12 kg 0 % / 100 % landlord/tenant … ≥ 52 kg 95 % / 5 %). Menu: *Real Estate* *Configuration* *Emission-CO2 Verteilung*.

Fields kept on `real_estate.settlement_unit` — a minimal *CO2 Costs* page shows just these two:

`co2_kostaufg`
Many2One reference, restricted to CO2KostAufG records of the same `property` as the settlement unit.

`co2_consumption`
(Function, One2Many, readonly.) All consumption records of the referenced CO2KostAufG that overlap *this settlement unit's own* period — the raw, per-source audit table.

The boundary-splitting logic (`_split_co2_consumption()`, see *Splitting at settlement-period boundaries* below) still operates per settlement unit and its own `co2_kostaufg`/period, unaffected by the aggregation described next.

Fields added to `real_estate.billing_unit` — a *CO2 Costs* page is shown only when `co2_relevant` is true (i.e. at least one of the billing unit's settlement units has `co2_kostaufg` set); a *Non-residential building >50% commercial (§8)* flag at the top of the page then further splits which of the remaining fields are shown:

`co2_relevant`
(Function.) True if any settlement unit of this billing unit has `co2_kostaufg` set. Controls the page's own visibility.

`non_residential_flag`
Boolean, default **false**. CO2KostAufG §8: sets whether this building is treated as predominantly commercial (>50 % of usable area) — where the residential emission-per-m² tier table (Anlage 1) does not apply and a flat, configured tenant/landlord split is used instead. This is also the corresponding BVED L-Satz field 19 (Kennzeichen Nichtwohngebäude) — see *BVED External Billing Interface* below, whose provider-assignment L-Satz preview and real export both read this field directly.

- **False** (default): `co2_measurement_type`, `co2_consumptions`, the four totals, `co2_emission_per_m2`, and the residential-tier `co2_tenant_share`/`co2_landlord_share` are shown; the two commercial share fields are hidden.

- **True**: only co2_commercial_tenant_share / co2_commercial_landlord_share are shown; all of the above are hidden. (The underlying values are always computed regardless of the flag — it only controls what the form displays.)

co2_measurement_type
Many2One to real_estate.measurement.type (object-type measurements only) — which measurement to sum for the area basis below, across the union of every co2-relevant settlement unit's own objects (each object counted once even if covered by more than one such settlement unit).

co2_consumptions
(Function, One2Many, readonly.) Union of the consumption records of *every* co2_kostaufg referenced by any settlement unit of this billing unit, overlapping the billing unit's own start_date/end_date — the aggregated audit table (as opposed to each settlement unit's own single-source co2_consumption above).

co2_total_consumption / co2_total_emission / co2_total_cost_gross
(Function.) Sum of consumption_kwh / co2_emission_kg / co2_cost_gross across co2_consumptions above, each weighted by the fraction of the record's *own* date range that actually falls inside the billing unit's period. A record fully inside the period contributes 100 %; a record extending beyond either boundary is interpolated pro-rata by days, always relative to its own actual range — a record that simply doesn't reach as far as the period's end (no later data booked yet) is never extrapolated, it only ever contributes its own days.

co2_total_area
(Function.) Sum, across the union of objects covered by every co2-relevant settlement unit (deduplicated), of each object's latest real_estate.measurement value of co2_measurement_type (or its effective leaf types, if a measurement group) valid as of the billing unit's end_date.

co2_emission_per_m2
(Function.) co2_total_emission / co2_total_area, annualized via × 365 / (end_date − start_date + 1) so that billing periods shorter or longer than a calendar year still yield a correct "kg CO2/m²/year" figure.

co2_tenant_share / co2_landlord_share
(Function.) Looked up via Co2EmissionShare.get_share(co2_emission_per_m2) — the residential 10-tier split. Always computed; only shown when non_residential_flag is false.

co2_commercial_tenant_share / co2_commercial_landlord_share
(Function.) The flat split used for commercial properties (not covered by the residential tier model): the company's configured re_accounting.co2_landlord_share_commercial (landlord), and 100 % minus that value (tenant). Always computed; only shown when non_residential_flag is true.

Splitting at settlement-period boundaries. SettlementUnit.compute_value_shares() ("Compute Value Shares" button, also reachable via the billing unit's own button of the same name) calls _split_co2_consumption() first, before anything else. For each of start_date and end_date + 1 day, every active consumption record of the referenced CO2KostAufG whose own [date_from, date_to] strictly contains that boundary date is split into two new records at that date: amounts are interpolated pro-rata by day count (the second part is computed as the remainder of the first, so the two new records always sum back exactly to the original — no rounding drift); rate fields (co2_kg_per_kwh, co2_price_ct_per_kwh, vat_rate) are copied unchanged onto both. Both new records get split = True; the original is deactivated (soft-deleted via active), never physically removed. A record entirely inside the settlement period, or one that only covers part of it because no later data has been booked yet, is never split or extrapolated — only an *existing* record that actually spans across a period boundary

gets divided. Re-running is idempotent: already boundary-aligned active records are left untouched, so repeated calls never produce duplicate splits.

2.14 BVED External Billing Interface

Part of Operating Cost Settlement.

Implements the BVED / ARGE-FHW "Standard-Datenaustausch" Version 3.10 — the German fixed-width record format used to exchange operating-cost data with an external Messdienstleister (heating-cost/consumption billing service) — for billing units with `external_billing = True` (see Operating Cost Settlement). Exports the property/tenant master data plus optional fuel and cost data the provider needs to bill (A-/L-/M-/B-/K-Satz); imports the provider's response (D-Satz actual costs, plus optional E835-/E898-/P-Satz) and applies it back onto `real_estate.settlement_result`. Implemented in `bved.py` (Tryton models) and `bved_records.py` (pure-Python fixed-width (de)serialization with no Tryton dependency, unit-testable standalone).

Menu: *Real Estate* → *Operation Costs* → *Externe Abrechnung* (placed immediately before *Betriebskostenabrechnung*), bundling all models below.

`real_estate.bved.unit` (`bved.py`)
: Reference table mirroring BVED Tabelle 'E' ("Einheiten/Maßeinheiten", see `bved_records.TABLE_E`) — `code` (3-digit key, e.g. `'010'`), `description` (e.g. `'m² Wohnfläche'`), and `name` (Function, `"{code} – {description}"`, e.g. `'010 – m² Wohnfläche'`). All 29 entries from the table are loaded as default data at module installation, with `measurement_type` left unset — mapping a unit to a `real_estate.measurement.type` (e.g. `'010 – m² Wohnfläche'` → the "Living Space" measurement type) is a per-database configuration decision, made via the *BVED Units* list (menu: *Real Estate* → *Operation Costs* → *Externe Abrechnung* → *BVED Units*). Used by `real_estate.bved.provider_assignment.allocation1_unit/allocation2_unit/allocation3_unit` (below) to resolve M-Satz fields 36-41.

`real_estate.bved.service_provider` (`bved.py`)
: The Messdienstleister itself: `party`, `bved_key` (2-char code from BVED Tabelle 'U', e.g. `'40'` for ista — not enforced as a fixed selection since the table changes over time), `bved_version` (currently only `'3.10'`), `transport_type` (only `'manual'` file download/upload is actually implemented; `email`/`sftp`/ `webservice_api` are reserved for a future automated transport layer).

  M-Satz field 7 (Kennzeichen Adressfeld) and address-source rules:

  `field7_mode`
  : Selection `'1'`=Nutzer / `'2'`=Eigentümer / `'3'`=Eigentümer/Nutzer / `'4'`=Leistungsnehmer (default `'1'`, matching the field's old hardcoded value). Written unchanged into M-Satz field 7 of every record generated for this provider, and controls which address block(s) `BvedObjectNumber._m_satz_values()` actually fills:

    - `'1'/'3'` — fields 8-15 (Nutzer) from the occupying contract's own party (`entry.contract.contractual_partner`).
    - `'2'/'3'` — fields 16-23 (Eigentümer) per `owner_rule`.
    - `'4'` — fields 59-66 (Leistungsnehmer) per `tenant_rule`.

  `owner_rule`
  : Selection `'company'`=Gesellschaft ist Eigentümer / `'role'`=Eigentümer über Rolle (default `'role'`). Visible/ required only when `field7_mode` is `'2'/'3'`, but always *evaluated* (see `provider_org_rule` below) regardless of `field7_mode`. `'company'`: the Eigentümer block is filled from the company party/address (the same ones passed as `company_party`/`company_address`). `'role'`: filled via `owner_role` instead.

owner_role
: Many2One to real_estate.object_party.role — which role counts as "Eigentümer" (used when owner_rule = 'role'), resolved via BvedObjectNumber._find_party_by_role() below. Defaults to the module's built-in "Owner" role (object_party_owner_role) for new records; a provider that needs a different role (e.g. one that also covers building/land, since the built-in role only covers property/object) can override it here.

tenant_rule
: Selection 'tenant'=Mieter / 'role'=über Partner-Rolle (default 'tenant'). Visible/required only when field7_mode is '4'. 'tenant': the Leistungsnehmer block (59-66) is filled the same way as the Nutzer block (the occupying contract's own party — reproducing the module's original, unconditional "debtor mirrors tenant" behavior). 'role': filled via tenant_role instead.

tenant_role
: Many2One to real_estate.object_party.role — which role counts as "Leistungsnehmer" (used when tenant_rule = 'role'), looked up the same way as owner_role.

provider_org_rule
: Selection 'owner'=Leistungsgeber wie Eigentümer / 'role'=über Partner-Rolle (required, default 'owner'). Fills M-Satz fields 42-49 (Leistungsgeber name/address) — **independent of** field7_mode (fields 42-49 are always attempted, regardless of what field 7 selects). 'owner': reuses whichever party/address owner_rule resolved to, even if field7_mode itself isn't '2'/'3' (so a provider can have field 7 = "Nutzer" while still deriving the Leistungsgeber block from the configured Eigentümer source). 'role': filled via provider_org_role instead.

provider_org_role
: Many2One to real_estate.object_party.role — which role counts as "Leistungsgeber" (used when provider_org_rule = 'role'), looked up the same way as owner_role.

tax_id_type
: Selection sourced from party.identifier.get_types() (all globally-configured identification types, e.g. 'de_vat') — which type on the company party (Leistungsgeber, the same party used for M-Satz fields 42-49) provides M-Satz field 51 (USt-ID-Nr. oder Steuernummer). When set and the company party has a matching party.identifier, field 51 gets that identifier's code and field 50 gets tax_id_flag below; both fields stay unset if left empty or no matching identifier exists.

tax_id_flag
: Selection '1'=USt-ID-Nr. / '2'=Steuernummer (visible/ required only when tax_id_type is set; default '2', matching the field's old hardcoded value). Written into M-Satz field 50 (Kennzeichen USt-ID/Steuernummer) whenever field 51 is — the admin's own classification of what the configured tax_id_type actually represents, since Tabelle 'E' has no generic notion of "this is a USt-ID vs. a Steuernummer" built in.

tax_rate_flag
: Selection '1'=Regelsteuersatz / '2'=ermäßigt, optional (Kann-Feld). Written unchanged into M-Satz field 52 (Kennzeichen Steuersatz) of every record for this provider; left unset, field 52 stays unset too.

invoice_number_flag
: Selection '0'=keine Rechnung §14 UStG / '1'=aus Feld 54 / '2'=vom Abrechnungsunternehmen erstellt, optional (Kann-Feld). Written unchanged into M-Satz field 53 (Kennzeichen Rechnungsnummer). '1' additionally requires invoice_number_rule to fill field 54 itself.

invoice_number_rule

Selection `'contract'`=Nr. Mietvertrag / `'object'`=Nr. Mietobjekt / `'settlement_result'`=ID Settlement result. Visible/required only when `invoice_number_flag` is `'1'`. Fills M-Satz field 54 (Rechnungsnummer, max. 25 chars) per occupancy segment: `'contract'` — the occupying contract's own `contract_number`; `'object'` — this mapping's own `base_object.object_number`; `'settlement_result'` — the `id` of the `real_estate.settlement_result` matching this object (and contract, or `None` for a vacancy segment) whose `billing_unit` period overlaps `[period_start, period_end]`. Field 54 stays unset if no matching value/record is found.

`direct_debit_flag`
Selection `'0'`=keine Abbuchungserlaubnis / `'1'`=Abbuchungserlaubnis, default `'0'`. Written unchanged into M-Satz field 58 (Kennzeichen Zahlungsart) of every record for this provider.

The optional selection fields `owner_rule`, `tenant_rule`, `invoice_number_rule`, `tax_id_flag`, `tax_rate_flag`, `invoice_number_flag` and `direct_debit_flag` include a blank choice, so they can be left empty.

`real_estate.bved.provider_assignment` (`bved.py`, form label "BVED Provider-Liegenschaft")
Anchors one provider + customer number + 9-digit Liegenschaftsnummer (`external_property_number`) to a `base_object` of type `property` or `building`, with `valid_from`/`valid_to` so a provider change over time doesn't destroy history. Unique per `(base_object, provider, valid_from)` — a property/building may have several *concurrent* assignments for different providers (e.g. heating via Techem, water via ista, both on the same building). Referenced explicitly from `billing_unit.bved_provider_assignment` (never derived from the object tree), since one building can have more than one active assignment and only the user knows which one a given billing unit means.

`object_numbers`
One2Many to `real_estate.bved.object_number` (below).

`allocation1_unit` / `allocation2_unit` / `allocation3_unit`
("M-Satz Allocation Key 1/2/3") Many2One to `real_estate.bved.unit` each. Resolve M-Satz fields 36/37, 38/39, 40/41 (Schlüssel/Anteil Umlage 1-3) for every object number mapping under this assignment, via `BvedObjectNumber._allocation_shares()`: field 36/38/40 gets the unit's own `code` (Tabelle 'E'), field 37/39/41 gets the rental object's own measurement value (as of the M-Satz row's `occupancy_end`) for the unit's mapped `measurement_type` — left unset if the unit has no `measurement_type` mapped (the key is still written on its own in that case) or if the slot itself is left empty.

`tenant_change_fee_flag`
Selection `'0'`=keine Umlage / `'1'`=Umlage (required, default `'0'`). Written unchanged into M-Satz field 68 (Kennzeichen Umlage Nutzerwechselgebühr) of every record generated for this provider assignment — unlike the other M-Satz catalog settings (on `real_estate.bved.service_provider`), this one is set per assignment, since it may differ per Liegenschaft under the same provider.

L-Satz Preview page — read-only preview of how this assignment's L-Satz (Liegenschaft) record would look today, split into two field groups:

- **Automatic** — `l_period_start`/`l_period_end` (always the most recently completed calendar year), `l_provider_key`, `l_street`/ `l_country`/`l_postal_code`/`l_city` (from the property's/ building's address), `l_vat_flag` (from the covered billing units' Optionssatz: 100 % → net/fully opted, 0 % → no VAT shown, mixed → per M-Satz field 25/user), `l_weg_flag` (from `calculation_method` — Cash basis sets it, Accrual basis doesn't), `l_non_residential_flag` (true if any covered billing unit's own `non_residential_flag` is set — see CO2 Cost Allocation), `l_co2_landlord_share_percent` (from the covered heating-cost billing unit's own

co2_landlord_share/co2_commercial_landlord_share), l_total_area (computed once a measurement type is selected below).

- **To Be Entered** — gross_floor_area_measurement_type (drives l_total_area: summed, as of each billing unit's own end date, over the union of every assigned billing unit's own covered_rental_objects() — objects with a settlement result where one exists, else every object covered by a settlement unit; each object counted once even if covered by several billing units), plus six purely-manual fields with no derivable source: vacancy_risk_flag/vacancy_risk_percent, labor_share_flag, energy_improvement_flag, heat_supply_flag, heat_connection_2023_flag (various L-Satz Kann-/Muss-Felder — legal facts about the building not represented anywhere else in the data model). vacancy_risk_flag is also reused, unchanged, for M-Satz field 26 (Kennzeichen Umlageausfallwagnis) — see BvedObjectNumber._m_satz_values() below.

The preview always uses the most recently completed calendar year across every billing unit currently assigned; the real export (BvedExport._build_l_m_records()) recomputes the same underlying helpers per specific billing unit and its own period instead, so preview and actual export can differ if a billing unit's period doesn't align with a plain calendar year.

real_estate.bved.object_number (bved.py)

Maps one rental object to its internal_reference ("Ordnungsbegriff des Auftraggebers") and 4-digit external_unit_number under a provider assignment, with its own valid_from/valid_to — a provider renumbering (e.g. after a renovation) should close the old row and add a new one rather than overwrite internal_reference in place, otherwise already-imported historical D-Satz lines referencing the old number can no longer be resolved. Numbers are assigned automatically (BvedExport._ensure_object_numbers()) the first time an object is exported — scoped per assignment (not per billing unit) and counted all-time, so a closed mapping's number is never reused by a later one.

_m_satz_values(period_start, period_end, provider, company_party=None, company_address=None, warnings=None, refresh_occupancy=True)

Returns the list of M-Satz value dicts for this mapping's object over the given period — one dict per real_estate.base_object.occupancy segment overlapping it (vacancy periods included with vacancy_flag=1). Field 25 (Kennzeichen MwSt, vat_treatment_flag) is derived per segment from the object's own Optionssatz as of that segment's occupancy_end: 0 (kein Ausweis) if 0 % (or unknown), else 1 (gewerbl. Vermietung). Field 7 (address_flag) and the Nutzer/Eigentümer/Leistungsnehmer/Leistungsgeber blocks are governed by provider.field7_mode/owner_rule/ tenant_rule/provider_org_rule — see real_estate.bved.service_provider above for the full rule description. Bank account fields (55/56) and vacancy_flag keep coming unconditionally from the occupying contract's own party, independent of field7_mode. Field 51 (USt-ID-Nr. oder Steuernummer) comes from company_party's own party.identifier of the type configured on provider.tax_id_type — always the company party specifically, never the field7_mode/provider_org_rule-resolved Leistungsgeber party; field 50 (Kennzeichen USt-ID/Steuernummer) is provider.tax_id_flag whenever field 51 is set, otherwise both fields stay unset. Field 52 (Kennzeichen Steuersatz) is provider.tax_rate_flag unchanged, if set. Fields 53/54 (Kennzeichen Rechnungsnummer / Rechnungsnummer) come from provider.invoice_number_flag and, when that is '1', provider.invoice_number_rule. Field 58 (Kennzeichen Zahlungsart) is provider.direct_debit_flag unchanged (see real_estate.bved.service_provider above for all of these). Field 68 (Kennzeichen Umlage Nutzerwechselgebühr) is assignment.tenant_change_fee_flag unchanged — this one comes from the provider *assignment* (real_estate.bved. provider_assignment above), not the provider itself. Field 26 (Kennzeichen Umlageausfallwagnis) is 1 if assignment.vacancy_risk_flag (the assignment's own L-Satz field 13) is set, else 0 — reusing the L-Satz value directly rather than a separate M-Satz-specific setting. The single, shared implementation of the M-Satz record

content — used both by the real export (`BvedExport._build_l_m_records()`) and by `refresh_m_satz_preview()` below, so they can never diverge. `refresh_occupancy` (default `True`) recomputes `real_estate.base_object.occupancy` first.

`_find_party_by_role(ObjectParty, role_id, as_of_date)`
Generic role-based party lookup — shared by the Eigentümer (`owner_role`), Leistungsnehmer (`tenant_role`), and Leistungsgeber (`provider_org_role`) resolutions alike: searches `real_estate.object_party` for `role_id`, valid *exactly on* `as_of_date` (a point-in-time check — `valid_from <= as_of_date` and no `valid_to` or `valid_to >= as_of_date` — not merely overlapping the billing period), walking up from this mapping's own `base_object` through its `parent` chain (rental object → building/land → property) until a match is found or the top of the tree is reached. Returns `None` immediately if `role_id` is `None`. All three call sites in `_m_satz_values()` use `period_end` as `as_of_date` (resolved once per call, not per occupancy segment).

`_area_share_by_mode(mode, as_of_date)`
Shared by `_heating_base_share()` (`mode='central_heating'`, M-Satz fields 27/69) and `_hotwater_base_share()` (`mode='central_hot_water'`, fields 30/70). Returns `(share, key)`:

- `share` — the object's own **direct** measurement value (via `Measurement.get_total_value()`, as of `as_of_date`) for the `heating_area_measurement_type` of the (first, if several) settlement unit belonging to this mapping's object's property with `heating_billing_mode = mode`. Deliberately **not** time-weighted by the occupancy segment (unlike `cost_share.area_share`, which drives the actual cost allocation and is prorated by `time_share/time_total`) and **not** affected by the `heating_consumption_share_percent/heating_area_share_percent` split — fields 27/30 always report the object's plain Bemessung. Does not require `compute_value_shares()` to have been run at all (no cost share involved). `None` — leaving the field unset (Kann-Feld) — if no qualifying settlement unit has a `heating_area_measurement_type` set, or the object has no matching measurement recorded.
- `key` — the BVED Tabelle 'E' code (see `real_estate.bved.unit` above) of the unit whose own `measurement_type` matches that measurement type — i.e. the unit `share` is actually expressed in. `None` if none is mapped.

`_heating_base_share(as_of_date)` and `_hotwater_base_share(as_of_date)` are thin wrappers writing into fields 27/69 and 30/70 respectively (called with the M-Satz row's own `occupancy_end` as `as_of_date`) — the two are fully independent (a property can have both a central-heating and a central-hot-water settlement unit, each with its own `heating_area_measurement_type`/mapped `real_estate.bved.unit`).

`_mode_settlement_unit(mode)`
The first `real_estate.settlement_unit` of this mapping's own object's property with `heating_billing_mode = mode`, or `None` — the same lookup `_area_share_by_mode()` performs inline, factored out since `_advance_by_mode()` below needs it independently of any `heating_area_measurement_type`.

`_advance_by_mode(mode, contract, start_date, end_date)`
M-Satz fields 28/29 (Heizung Vorauszahlung Brutto/Netto, `mode='central_heating'`) or 31/32 (Warmwasser Vorauszahlung Brutto/Netto, `mode='central_hot_water'`) — **not** 30/31, which is the Warmwasser Grundanteil/Vorauszahlung-Brutto pair respectively; field 30 itself is the (already-implemented) Warmwasser Grundanteil, see `_area_share_by_mode()` above. Returns `(gross, net)`, either possibly `None`:

- Resolves the qualifying settlement unit via `_mode_settlement_unit(mode)` and, through it, its own `billing_unit` — the billing unit whose *Konditionstyp*

(`term_types_of_use`) and *Vorauszahlungen* tab (`cash_flow_lines`) define which advance-payment condition applies. `(None, None)` if no qualifying settlement unit, or it has no `billing_unit`.

- For `mode='central_hot_water'`: if the property's own central-heating settlement unit exists and shares the *same* `billing_unit` as the central-hot-water one, returns `(None, None)` unconditionally — German rental contracts normally carry a single combined "Heizkosten" advance payment covering both heating and hot water; attributing it to both fields would double it. Sharing a billing unit is the normal setup where one annual "Heizkosten" cost group contains both a central-heating and a central-hot-water settlement unit side by side (see `tests/test_billing_unit.py`'s two-Settlement-Unit "Heizkosten" billing unit).
- Otherwise, sums `total_amount` (gross) and `amount` (net) over every entry of the billing unit's own `cash_flow_lines` (already filtered there by `term_types_of_use`/invoice state, see Operating Cost Settlement) whose own `contract` matches `contract` and `base_object` matches this mapping's own object, restricted to `document_date` within `[start_date, end_date]`. `(None, None)` if nothing matches (a genuine Kann-Feld, not a zero).

`_heating_advance(contract, start_date, end_date)` and `_hotwater_advance(contract, start_date, end_date)` are thin wrappers, called from `_m_satz_values()` with the M-Satz row's own `occupancy_start`/`occupancy_end` (not the whole export period) and its resolved tenant contract (`None` for a vacancy segment, in which case both return `(None, None)` immediately).

> **Note**
>
> Fields 33-35 (Kaltwasser Grundanteil/Vorauszahlung) already exist as Kann-Felder on `real_estate.bved.object_number.m_satz_line` (`coldwater_base_share`/`coldwater_advance_gross`/`_net`) but are not populated by `_m_satz_values()` — there is no `heating_billing_mode` value for a centrally-billed cold-water system (cold water is always metered per object via `allocation_by_consumption` in this module), so `_mode_settlement_unit()`/`_advance_by_mode()` have no analogous "central cold water" case to resolve them from.

`_allocation_shares(as_of_date)`

Resolves M-Satz fields 36-41 (Schlüssel/Anteil Umlage 1-3): for each of the provider assignment's `allocation1_unit`/ `allocation2_unit`/`allocation3_unit` slots (in order), returns `(code, share)` — `code` is the `real_estate.bved.unit`'s own Tabelle 'E' code, `share` is this mapping's own `base_object`'s measurement value (via `Measurement.get_total_value()`, no time-weighting) for the unit's `measurement_type` as of `as_of_date` — or `(code, None)` if the unit has no `measurement_type` mapped, or `None` for an unset slot. Always returns exactly 3 entries. Called with each M-Satz row's own `occupancy_end` as `as_of_date`.

`m_satz_lines` / `refresh_m_satz_preview`

A stored (not Function) One2Many to `real_estate.bved.object_number.m_satz_line` (below), populated by the `refresh_m_satz_preview` button: deletes any existing preview lines for the mapping, then rebuilds them from `_m_satz_values()` for the most recently completed calendar year (same period as the provider assignment's L-Satz preview, `BvedProviderAssignment._last_full_year()`) — one line per occupancy segment. Deliberately a real, explicitly-triggered button rather than a Function field: a Function getter runs inside a *read-only* database transaction (plain `read()` RPCs), where

_m_satz_values(refresh_occupancy=True)'s delete-and-recreate inside Occupancy.refresh() would fail outright — and a true multi-row, per-field table needs real addressable child records for Tryton's list/form widgets anyway.

real_estate.bved.object_number.m_satz_line (bved.py)
One row per occupancy segment, generated only by BvedObjectNumber.refresh_m_satz_preview() — never created or edited directly (every field is readonly). Holds all 70 M-Satz fields from the BVED Nutzer/Eigentümer-Satz spec (field number prefixed in each field's label, e.g. "8. Tenant Name 1"), grouped on the form by spec section (identification, tenant, owner, occupancy period, VAT/vacancy-risk/heating-hotwater-coldwater shares, allocations, provider organization, tax/invoice/bank/payment, debtor, other flags/keys). The embedded list view shows just occupancy_start/occupancy_end/vacancy_flag/tenant_name1 for a quick overview across several tenants/vacancy periods; opening a row shows the full detail form.

Fields added to real_estate.billing_unit:

bved_provider_assignment
Which provider assignment applies (domain: same property/building subtree, validity window covering the billing unit's start date). Only visible when external_billing is set.

bved_object_numbers
(Function, One2Many.) The assignment's object numbers, restricted to objects actually covered by this billing unit (bved_covered_object_ids() — type object base objects reachable via at least one non-no_allocation settlement unit).

check_bved_provider_assignment()
Validation (run from pre_validate) rejecting a billing unit whose covered objects include one that is neither the assignment's own base_object nor a descendant of it (e.g. an object from a different building than the one referenced).

Shown on a dedicated *BVED* page, visible only when external_billing is set.

Fields added to real_estate.settlement_unit (B-Satz — fuel/stock/hot-water data, typically entered only on the heating-cost settlement unit):

bved_fuel_data
Enables the rest of the B-Satz fields below; also identifies "the" heating-cost unit representative of a billing unit's CO2 landlord share (see CO2 Cost Allocation).

bved_fuel_type (BVED Tabelle 'B'), bved_heating_value, bved_fuel_indicator (field 23, Kennzeichen Brennstoffart — Selection '0'-'9', default '0'), and bved_fuel_rule ('stock'/'meter', default 'stock') — selects which of two mutually exclusive field groups below the *B-Satz* / *Brennstoff* page shows; two further groups apply either way and are always shown.

bved_fuel_indicator distinguishes several concurrent B-Satz fuel records for the same property/period (e.g. two fuel suppliers each feeding their own externally-billed settlement unit within the same billing period). '0' if field 8 (bved_fuel_type) is left empty, else '1'-'9'. on_change_bved_fuel_type keeps it consistent automatically: clearing field 8 resets it to '0'; setting field 8 bumps it from '0'/empty to '1' (the user may raise it to '2'-'9' by hand for an additional concurrent fuel record). SettlementUnit.validate_fields enforces, whenever bved_fuel_data is set: the '0' ⇔ empty-field-8 correspondence above, and — for a non-'0' value — uniqueness across every other settlement unit of the same property (billing_unit.property) whose billing period overlaps this one's (billing_unit.start_date/end_date).

'stock' (Bestandsführung, B-Satz fields 10-17 — storable fuels like heating oil, tracked via initial/closing stock only): stock start/end date + quantity + amount (gross/net). Hidden when bved_fuel_rule = 'meter'.

WW-Anteil (B-Satz fields 18-22 — hot-water average temperature/consumption/flat-rate percentage/meter start-end) and *Versorgungszeitraum* (B-Satz fields 24-27 — up to two heating and two hot-water supply periods, start/end each) apply to both `bved_fuel_rule` values and are always shown (once `bved_fuel_data` is set) regardless of it.

`'meter'` (Zähler, B-Satz fields 28-34 — district heating or other supply metered directly, without a storable stock): `bved_meter_type` (BVED Tabelle 'G'), `bved_meter_measurement_unit` (Many2One to `real_estate.bved.unit`, BVED Tabelle 'E' — the same model used for the M-Satz allocation keys above, reused here since it already covers the full Tabelle 'E' code list), `bved_meter_number`, `bved_meter_consumption`, `bved_meter_reading_start`/`bved_meter_reading_end`. `bved_primary_energy_factor` (field 34) is shown as part of this group, per the standard's field-position grouping, even though it is a property of the fuel type (mandatory for Fernwärme) rather than of the metering method itself. Hidden when `bved_fuel_rule = 'stock'`.

All four groups are always packed into the B-Satz record regardless of `bved_fuel_rule` — the fields belonging to the currently hidden group are simply left empty by the user and packed blank/zero as usual.

> **Note**
>
> Every field label in this section (and the L-Satz Preview fields on `real_estate.bved.provider_assignment` below) is prefixed with its record field number, e.g. "10. Stock Start Date" — matching the convention already used by the M-Satz Preview line model. `bved_fuel_data` and `bved_fuel_rule` are left unnumbered since they are module-only controls, not fields of the B-Satz record itself.

K-Satz cost records (`BvedExport._build_b_k_records()`) are built not only from invoice lines booked directly on the externally-billed settlement unit, but also from every settlement unit of the same billing unit that uses `allocation_via_cost_collector` with it as `reference_settlement_unit` (e.g. several fuel suppliers, each invoiced separately, feeding one heating settlement unit) — each such invoice line keeps its own settlement unit's `type.bved_cost_key` rather than the referenced unit's, since the underlying invoices may carry a different cost-type classification. B-Satz (fuel/stock data above) is unaffected — it stays tied to the externally-billed settlement unit itself.

Export (`real_estate.bved.export`, workflow Draft Generated Sent):

`provider` + `cutoff_date` (default: 31 December of the previous year — a still-running billing period shouldn't be exported yet since its cost data isn't final) default-select the eligible `billing_units` (assigned to this provider, `external_billing = True`, `end_date <= cutoff_date`); the list stays editable. `record_types` (multi-select A/L/M/B/K, default all five) controls which files get built.

`generate`
: Validates every selected billing unit has a provider assignment matching this export's provider and a complete property address, auto-creates any missing object numbers, then builds one `.DAT` file per requested record-type group (A; L+M combined; B+K combined) and attaches them (ISO-8859-1 encoded, CRLF line endings) to the export record. Sets `export_date` and moves to `generated`.

`mark_sent`
: Moves to `sent` — the actual transmission happens outside the app (manual file handoff, per the provider's `transport_type`).

All activity (files built, warnings such as a missing German IBAN for the A-/M-Satz bank fields) is appended to `activity_log`.

Import (`real_estate.bved.import`, workflow Draft Parsed Matched Processed):

Attach the provider's response file(s) as `ir.attachment` on the import record (filename prefix determines the record type: D/E835/ E898/P), then:

`parse`
Full fresh snapshot on every click: deletes every not-yet-`applied` line and rebuilds `real_estate.bved.import.line` rows from whatever is currently attached, so re-running after attaching a corrected/additional file never leaves stale duplicates. Already-`applied` lines are never touched.

`match`
Re-evaluates every line not yet `applied` (including previously `error`/`skipped` ones, so fixing a missing object number and clicking again re-checks them too). Resolves `internal_reference` + provider to a `bved.object_number` valid on the line's period end date, then the matching `settlement_result` for that object — cross-checked against the result's own billing unit's provider assignment, so an object with two concurrent externally-billed billing units (e.g. heating via one provider, water via another) cannot be matched to the wrong result. E835-/E898-/P-Satz lines missing "Letzter Tag Nutzungszeitraum" (a Mussfeld per spec for these three types) are flagged as an error rather than guessed. A line with no data and no match is `skipped` (not an error); one with data and no match is an `error`.

`apply`
Writes matched data onto the `settlement_result`:

- **D-Satz** — lines for the same result are grouped by `d_cost_key` (BVED Tabelle 'K' cost-type code): different keys are summed (e.g. Heizung + Warmwasser), two lines sharing the same key (including both empty) are treated as a duplicate delivery and only the last is kept, with a warning. Sets `actual_costs` (re-running `on_change_actual_costs()` so `refund_receivable` updates immediately), plus a plausibility check (total − advance ≈ balance per the D-Satz's own figures; reported advance vs. the internally computed `advanced_payment` — skipped when the reported sum is exactly 0, since BVED's fixed-width numeric fields can't distinguish "blank" from "genuinely zero"). Sets `bved_state` to `validated` or `validation_error` (with the concatenated messages in `bved_check_message`). Already-`applied` D-Satz lines whose result is still `validation_error` are re-evaluated on every subsequent Apply run too, so a plausibility-check fix (e.g. a corrected advance payment) takes effect without re-parsing.
- **E898-Satz** — copies the referenced PDF attachment (matched by filename) from the import onto the settlement result.
- **E835-/P-Satz** — informational only; no settlement-result field exists to write them back to, browsable via the result's own `bved_e835_lines`/`bved_p_lines` tabs.

Afterwards, logs which of a touched billing unit's covered objects still have no D-Satz result (`bved_state` not yet `imported`/`validated`/`validation_error`) as a completeness check.

`line_summary`
(Function.) A per-record-type/state line count plus amount summary (D-Satz total, E835/P-Satz user-share subtotal) shown on the import form.

`real_estate.bved.import.line` (`bved.py`)
One row per parsed fixed-width record, typed fields for all four import record types (D/E835/E898/P)

plus raw_line/raw_data (full field dump as JSON) for audit. States: parsed → matched → applied (or skipped/error). Form notebook pages *D-Satz/E835-Satz/E898-Satz/P-Satz* show/hide automatically based on record_type (view_attributes()); the *Row* page (raw data) is always visible.

Fields added to real_estate.settlement_result:

bved_state
Selection (awaiting_import / imported / validated / validation_error) tracking the data-exchange progress independently of state (approved/billed); editable until billed, e.g. to manually clear a plausibility-check false positive.

bved_import_date / bved_check_message / bved_import_line
(readonly) — when the last D-Satz was applied, its check messages (if any), and the import line that last wrote actual_costs.

bved_matched_lines
(Function, One2Many.) Every import line matched to this result, regardless of type/state, shown as one unified, detailed table.

bved_e835_lines / bved_p_lines / bved_e835_labor_share_user_total / bved_p_user_amount_total
(Function.) Informational subsets of bved_matched_lines by record type, and their summed amounts.

Demo / test tooling. tests/test_bved_provider_response.py has no trytond/database dependency (it loads bved_records.py directly from disk by path) and simulates a provider's response from a set of exported A-/L-/M-/K-Satz files: pools K-Satz costs per property, distributes them across M-Satz units weighted by heating advance payment × occupancy-period length in days (with reproducible random jitter and a largest-remainder, cent-exact correction so the total still matches exactly), emits D-Satz plus, where applicable, E835-Satz (labor share) and a small synthetic P-Satz (Energiepreisbremse — format exercise only, the underlying support program has since ended), and one E898-Satz record per unit/period referencing a generated single-page PDF "Heizkostenabrechnung" (period, per-cost-type total vs. share, overall total/advance payment/balance) built with a dependency-free, hand-rolled minimal PDF writer:

```
python tests/test_bved_provider_response.py DTA310_....DAT DTM310_....DAT \
    DTK310_....DAT [--out-dir DIR] [--seed N]
```

2.15 Option Rate (Input VAT Deduction)

Tracks, over time, what percentage of input VAT (Vorsteuer) on purchase invoices related to a real estate object is deductible ("optiert") rather than treated as a final cost (added to the gross expense) — relevant wherever the property owner can opt for VAT liability on rent (§ 9 UStG). Applies to real_estate.base_object (property / building / land / rental object), real_estate.settlement_unit, and real_estate.billing_unit.

account.invoice.line.taxes_deductible_rate auto-fill (invoice.py)
For purchase invoice lines (invoice_type = 'in'), the core taxes_deductible_rate field (0–1 fraction of input VAT that is deductible; core already forces it to 0 when company.purchase_taxes_expense is set, via its own on_change_company) is auto-filled from the applicable option rate, divided by 100. Applies both interactively (on_change_base_object / on_change_settlement_unit / on_change_billing_unit) and to programmatic line creation (InvoiceLine.create() override) — the latter only fills the field when the caller did **not** pass taxes_deductible_rate explicitly, so an explicit value is never overridden.

The real-estate reference used is picked by InvoiceLine._option_rate_priority() in order of specificity: base_object → settlement_unit → billing_unit → the derived

property (as a base_object reference, last-resort fallback via on_change_with_property / term.property) — the first of these that is set on the line wins. OptionRate.get_current_rate_fraction(ref_field, record, date) then returns the record's latest option_rate at or before that date (taxes_date if set, else the invoice's invoice_date, else today), divided by 100; falling back to 1/0 for fix_100/fix_0 if no history record exists yet, or None (leaving the field untouched) for dynamic_measurement with no booked history.

The auto-fill is skipped entirely (field left at its core default of 1) when: the line is not a purchase line, or the company has purchase_taxes_expense set (core's own logic already governs that case). The line's contract field has no bearing on this logic and is ignored — a real-estate reference is used whenever present, even on a line that also carries a contract.

real_estate.option_rate (option_rate.py)
History-versioned option rate record, valid from a given date. Has three mutually exclusive Many2One reference fields — base_object, settlement_unit, billing_unit (all ondelete='CASCADE') — exactly one of which must be set (enforced in validate()); a unique SQL constraint prevents two records for the same reference and valid_from. option_rate is a percentage (digits=(5, 2), domain 0–100).

Fields added to real_estate.base_object, real_estate.settlement_unit, and real_estate.billing_unit (identical on all three):

option_rate_method
Required selection: fix_0 (always 0 %), fix_100 (always 100 %), or dynamic_measurement (calculated — see the wizard below). Defaults to fix_0. On a rental object (base_object with type = 'object'), changing type_of_use auto-sets it to fix_100 for commercial and fix_0 otherwise (on_change_type_of_use); dynamic_measurement is rejected by validate_fields for rental objects — only property/building/land/ settlement unit/billing unit may use it.

option_measurement_type
Many2One to real_estate.measurement.type, required only for dynamic_measurement. May reference a measurement **group**: all of its descendant leaf types are then summed per rental object (see the group hierarchy under real_estate.measurement.type above).

option_rates
One2Many (readonly) to the object's own real_estate.option_rate history.

purchase_taxes_expense
Function field mirroring the pre-existing company.purchase_taxes_expense field (added by account_invoice). When set on the object's company, all input VAT is always booked as an expense (fully gross) — equivalent to a permanent option rate of 0 % — and the *Option Rate* notebook page is hidden on the form (also always hidden for type = 'equipment').

Shown as a dedicated *Option Rate* page on the base_object, settlement_unit, and billing_unit forms (page_option_rate).

real_estate.option_rate_update.wizard (see Wizards)
Batch recalculation entry point; see the Wizards page for the full selection/calculation logic.

Selection / filter screen and standalone list. Menu: *Real Estate* → *Master Data* → *Update Option Rates*, with a child menu item *Option Rates* opening a read-only list of all real_estate.option_rate records (both list and form views are fully non-editable — creatable="0" plus readonly="1" on every field; this applies only to the standalone list, not to the option_rates tab embedded in the base_object/settlement_unit/billing_unit forms, which remains editable there). A *Selektion* context form — modelled on the *Occupancy* context pattern (real_estate.option_rate.context, class OptionRateContext) — sits above the list with three optional filter fields, base_object,

`billing_unit`, `settlement_unit`; whichever one is set narrows the list via `context_domain` (PYSON, evaluated independently per field, so any combination — or none — can be applied).

2.16 Extensions to Core Modules

`party.party` (`party.py`)
Adds `sequence` (integer sort key) and `salutation` fields.

`account.invoice` (`invoice.py`)
Adds a `contract` Many2One field so invoices can be traced back to the originating contract. Shown on the invoice form's *Other Info* tab (view extension `view/invoice_form.xml`, inherits `account_invoice.invoice_view_form`).

`_get_move_line(date, amount)`
Sets the invoice's `contract` on the receivable/payable move line as well, so the open item shown on the *Payable/Receivable Lines* list can be traced to its contract (revenue/expense lines get it via the invoice lines, see below).

`account.invoice.line` (`invoice.py`)
Adds real-estate assignment fields, gated by `assignment_control` (Selection: *All*, `contract`, `operating_costs`, `settlement_result_contract`, `settlement_result_vacant` — controls which of the fields below are visible/required for a given line):

- `contract` / `term` — the originating `real_estate.contract` / `ContractTerm`
- `base_object` — the real-estate object the line relates to
- `billing_unit` / `settlement_unit` — for operating-cost billing lines
- `property` (Function) — derived from `billing_unit`/`settlement_unit`
- `service_period_from` / `service_period_to` — billed service period
- `estg_35a` (Selection) — German §35a EStG tax-deduction category (household services / craftsmen services)
- `invoice_date`, `tax_amount`, `total_amount` (Function fields)

Indexed on `contract`, `term`, `settlement_unit`, `billing_unit`.

`account.move.line` (`invoice.py`, class `AccountMoveLine`)
Mirrors the same real-estate fields as `account.invoice.line` above (`assignment_control`, `contract`, `term`, `base_object`, `billing_unit`, `settlement_unit`, `property`) directly on the journal entry line, so postings can be traced/filtered without going through the invoice line. Extended list view shows this context for journal entries.

`effective_contract` (Function field)
Getter `on_change_with_effective_contract`, searcher `search_effective_contract`. This line's own `contract` if set, else the `contract` of whichever *other* line in the same `reconciliation` group has one. This way payment lines, which carry no `contract` of their own, are found by contract once they are reconciled with the invoice's receivable/payable line. The stored `contract` field is not changed.

***Payable/Receivable Lines* (`act_contract_move_line_payable_receivable` in `contract.xml`)**
Menu *Contracts*, between *Contract* and *Create Contract Moves*. Lists open (unreconciled) lines on receivable/payable accounts (`account.move.line`) across all parties — the same list as the core *Payable/Receivable Lines* relate action on the party form, but reachable from a menu without opening a party first.

`order`: `[('payable_receivable_date', 'ASC NULLS FIRST'), ('move', 'ASC')]` — by due date, then by move.

Tree view: `contract_move_line_view_list_payable_receivable` (`view/contract_move_line_payable_receivable_list.xml`, inherits `account.move_line_view_list_payable_receivable`):

- `party` — optional column.
- `payable_receivable_date` ("Fälligkeitsdatum" — `maturity_date` if set, else `date`) and `date` ("Buchungsdatum") — both optional.
- `amount`, `debit` ("Soll"), `credit` ("Haben") — optional.
- `payable_receivable_balance` ("Saldo") — optional.

Filter panel (`real_estate.contract.move_line_payable_receivable.context`, own context model):

- `company` (required)
- `party`, `contract`, `account` — optional, exact match; `contract` filters via `effective_contract` (see `account.move.line` above), so reconciled payment lines are found too
- `date_from`/`date_to` — restrict `account.move.line.date`
- `receivable`/`payable`/`reconciled` — checkboxes with the same defaults (`True`/`True`/`False`) and behaviour as the core party-form dialog; can be preset via the action context.

Relate button on the contract form itself (`act_contract_move_line_payable_receivable_relate`) reuses the same tree view with its own, minimal context model `real_estate.contract.move_line_payable_receivable.relate_context` (`date_from`/`date_to` and the three toggles only). The list is restricted to the open contract via the action's static `domain` `('effective_contract', 'in', Eval('active_ids'))`.

Print (`real_estate.contract.move_line_payable_receivable.report`)
Class `ContractMoveLinePayableReceivableReport` in `invoice.py`, template `report/contract_move_line_payable_receivable_de.odt`. Registered as `ir.action.report` with an `ir.action.keyword` (`keyword='form_print'`) on `account.move.line,-1` — select the desired rows in the list and use the client's *Print* action. The printed table has a fixed column set, independent of the list's column/sort settings: Datum (`payable_receivable_date` — due date if set, else booking date, as in the list), Partei, Soll, Haben, Buchungssatz (`move`), Beschreibung (`description_used`) — plus a totals row (Soll/Haben summed). The filter panel's `date_from`/`date_to` restrict the booking date (`date`).

`get_context()` adds `lines` (the selected records), `total_debit`/`total_credit` (summed), and the `format_value` helper (date/number formatting, amounts with two decimal places) also used by `ContractReport`/`ContractAnnex4Report`.

`_selection_summary()` builds a one-line German summary of the filter panel's values ("Partei: …; Vertrag: …; Konto: …; Zeitraum: … - …; Forderungen, Verbindlichkeiten; inkl. ausgeglichene Posten") from `Transaction().context`, printed near the report header (`selection_summary`); with no filter set it reads "keine Einschränkung". The action's `context` field forwards all filter panel fields into the transaction context for this purpose.

***Kontenblatt* (`act_contract_general_ledger_account_contract` in `contract.xml`)**
Menu *Contracts*, between *Payable/Receivable Lines* and *Create Contract Moves*. A standalone equivalent of the `real_estate.contract` form's own *Party Ledger* ("Kontenblatt") button (`Contract.open_party_ledger()` in `contract_core.py`, opening

act_general_ledger_account_contract_form_contract, restricted to the open contract). Same underlying model (real_estate.account_contract — per-(account, party) aggregated balance/debit/credit for the contract's own party), reachable directly from a menu without a fixed contract restriction, like core's *General Ledger - Accounts* menu entry. Default search_value [('line_count', '!=', 0)] hides accounts with no activity.

AccountContract.table_query() (contract_core.py) is restricted to the current company (context['company']) and joins at most one contract per contractual partner (the most recent one, Max(id)).

context_model: real_estate.contract.general_ledger_account_contract.context (ContractGeneralLedgerAccountContractContext in contract_core.py) — a subclass of core's GeneralLedgerAccountContext (account.general_ledger.account.context) with its fiscalyear/period/date-range/company/posted/journal fields, plus two extra filter fields party and contract, applied via the act_window's context_domain (no restriction when left empty). View: contract_general_ledger_account_contract_context_form.xml.

Tree view: contract_general_ledger_account_contract_view_list (view/contract_general_ledger_account_contract_list.xml) inherits general_ledger_account_contract_list and adds the contract column (optional="1") right after party.

Print (real_estate.contract.general_ledger_account_contract.report)
Class ContractGeneralLedgerAccountContractReport in contract_core.py, template report/contract_general_ledger_account_contract_de.odt. Registered the same way as the Payable/Receivable report above (on real_estate.account_contract, −1) — select the desired rows in the *Kontenblatt* list and use the client's *Print* action. Fixed column set: Konto, Partei, Vertrag, Anfangssaldo, Soll, Haben, Endsaldo — plus a totals row.

Same format_value helper and get_context() / _selection_summary() pattern as the Payable/Receivable report; the summary reflects the *Kontenblatt* filter fields (Partei, Vertrag, Geschäftsjahr, Periode, Zeitraum, Journal, "nur gebuchte Bewegungen"), forwarded via the action's context field.

account.general_ledger.line (invoice.py, class GeneralLedgerLine)
Adds contract, term, base_object, billing_unit, and settlement_unit (all readonly Many2One) to Tryton's standard General Ledger — Lines report (German: *Kontenblätter – Positionen*), so the individual postings for an account can be filtered/traced back to the originating real-estate contract, term, object, billing unit, or settlement unit. No override of table_query is needed: it pulls any non-Function field straight from the account.move.line table by matching field name, and account.move.line already carries these same fields (see above). Shown as optional columns in the tree view (view/general_ledger_line_list.xml, inherits account.general_ledger_line_view_list).

res.user (res.py, task.py)
Adds phone and mobile fields and the preference task_email (task reminders by e-mail, default on - see Tasks, Processes and Handover Reports).

company.company (company.py)
Adds re_accounting - the company's real estate accounting configuration (see Configuration).

ir.cron (ir.py)
Adds the method real_estate.contract|cron_daily (*Real Estate Daily Tasks*) to the fixed selection of cron methods; it dispatches the per-company scheduled tasks (real_estate.cron_task).

ir.rule (ir.py)
Adds user_id to the evaluation context of record rules and the user to their cache key, for rules that depend on the current user (task rule "assigned to me").

`ir.sequence` (`sequence.py`)
: Adds the sequence type `string timestamp` (formatted UTC timestamp `YYYY-MM-DD HH:MM:SS`).

2.17 Tasks, Processes and Handover Reports

2.17.1 Overview

- **Tasks** (*Aufgaben*): follow-ups on a record - created manually, by hooks of the module (e.g. termination, index rent), by task rules or by process steps - with reminders and escalation.
- **Task rules**: scheduled creation of tasks from a date of a record (e.g. fixed-term contract ends, meter calibration expires).
- **Processes** (*Prozesse*): a sequence of tasks from a template (move-out, move-in, change of tenant, index rent adjustment), started manually or by the contract type.
- **Handover reports** (*Übergabeprotokolle*): move-in, pre-inspection and move-out reports of a contract; they complete the matching process steps.

Menus:

- *Real Estate › Tasks and Follow-ups*: *My Tasks*, *All Tasks*, *Processes*
- *Real Estate › Contracts › Handover Reports*
- *Real Estate › Configuration › Tasks and Processes*: *Task Types*, *Task Rules*, *Process Templates*, *Handover Checklists*

The contract, the objects (property, building, land, rental object, equipment), the billing unit and the rent adjustment have a tab *Tasks and Processes* with the sub tabs *Open* (open tasks - + creates a task - and running processes) and *History* (done/cancelled tasks and processes); the lists show the reference of each task and process. What is listed:

- contract: all tasks of the contract (also of its rent adjustments and parties, stored `contract`) and its processes;
- property: all tasks and processes of the property - its objects, contracts and billing units (stored `property`; + creates a task for the property itself);
- rental object: its own tasks and processes plus, read-only, those of the contracts whose items contain it (function fields with one search each, `contract.items.objects`);
- building, land, equipment, billing unit, rent adjustment: their own tasks and processes (by `resource`).

The tab *Handover Reports* of the contract lists the handover reports.

2.17.2 Tasks

`real_estate.task.type` (`task.py`)
: Task type (menu *Configuration › Tasks and Processes › Task Types*). `code` is a selection from the fixed catalog `TASK_CODES` in `task.py` (code label, reference objects, manual or automatic only), unique and readonly once saved; hooks, rules and processes find their type by it. Types without code are own types without function in the module, their reference objects are selected in `custom_models` (empty = all, `all_models`). `resource_models` (catalog or own selection) and `manual` are shown; `for_model` (searcher only) filters the types allowed for an object. Default responsibility (`responsible_group` / `responsible_user`, optionally `responsible_role` - see *Responsibility by party role* below), `remind_days`, `remind_daily`, escalation (`escalate_days`, `escalation_group`), `recurrence_months`,

email, icon, note_on_done. Default types (task.xml, noupdate) for the uses of the module, e.g. manual, contract_unsigned, index_prepare, index_values, comparative_consent, comparative_lawsuit, billing_deadline, meter_calibration, cron_error.

real_estate.task (task.py)

Task of a record (resource: contract, contract party, rent adjustment, term adjustment, object, billing unit, price index, scheduled task, party). contract and property are stored and derived from the reference. Fields: subject, type, due_date, remind_date (default due date - remind days), responsible user and/or group (default from the type, else the creator), priority, description, result, recurrence_months, previous, origin / origin_key (rule or process, idempotence), notified_date / escalated_date (reminders, phase B), is_mine and overdue (searchable).

Responsibility by party role. With a responsible_role on the task type (or on the process template step, which takes precedence) the party holding this object party role (e.g. *Administrator*, *Caretaker*) on the objects of the reference on the **due date** becomes responsible: objects of the contract items (contract, contract party, rent and term adjustments) resp. the object itself or the property of a billing unit, then their parents up to the property - the most specific level wins, on one level the latest *valid from* (Task._role_responsible). The party is stored in responsible_party (shown, also without a user, e.g. an external caretaker). If exactly one active user is linked to an employee of the party in the company of the task (user form, field *Employees*), this user becomes the responsible user. Otherwise, for a task entered manually, the creator becomes responsible if the type has creator_responsible (*Creator Responsible on Manual Entry*); else the user of the type applies, which may be empty - the task is then visible to the group and can be taken over (*Take Over*). The group of the type is kept in every case; a task without user and group goes to the creator (Task._default_responsible). Resolved when a task is created without given responsibility and in the form when the type is chosen (on_change_task_type, also in *New Task*); open tasks are not reassigned when an assignment changes.

States open → done / cancelled: *Done* (done by/on; with a recurrence the next task n months later; optional note on the referenced record), *Cancel* (reason in *Result* required), *Reopen* (creator or administration), *Postpone* (wizard: new due date and reason, history in the description, reminder reset), *Take Over* (responsible user = current user).

Menu *Real Estate › Tasks and Follow-ups › My Tasks* (tabs *Due*, *Overdue*, *Next 30 Days*, *All Open*, *Done* with counters; list, form, calendar) and *All Tasks*. Tab *Tasks* with + on contract (there: tab *Tasks and Processes*, sub tab *Open*) (One2Many over the stored contract: all tasks of the contract, also of its rent adjustments and parties; the contract is the reference of new ones), rent adjustment, object and billing unit (One2Many over the reference resource); a property additionally lists the tasks of all its objects (read-only). Action *New Task* on these forms. The type of a task is restricted to the types allowed for the reference object (resource_model) and, for manual tasks, to manual types; tasks created by the module are marked automatic.

Visibility: company rule; the groups Object/Contract/Billing/View see tasks assigned to them (user or group) or created by them, the administration sees all. For this the record rule context is extended by user_id (ir.py).

Programming interface: Task.create_for(record, type_code, due_date, ...) (idempotent), close_for(record, type_code), cancel_for(record, type_code). Hook: executing an index adjustment closes index_prepare of the agreement; the further steps of an index adjustment are the process steps of the adjustment run. Comparative rent: the receipt date creates comparative_consent (due at the consent deadline), the decision closes it, a refusal creates comparative_lawsuit (14 days before the lawsuit deadline).

Reminders and escalation (phase B, Task.notify)

Scheduled task task_notify (*Real Estate Accounting › Scheduled Tasks*, daily): open tasks with remind_date reached notify the responsible user, else all active members of the responsible group (client notification res.notification with link to the task) - once (notified_date) or every

day with `remind_daily` on the type. Open tasks `escalate_days` after the due date notify the `escalation_group` once (`escalated_date`). With `email` on the type an e-mail is sent in addition (`ir.email`) to users with an address and `task_email` (user preference, default on); without SMTP configuration the error is only logged. For the live display of notifications in the client the server needs `[bus] allow_subscribe = True` in `trytond.conf`.

`Contract.cron_daily` runs every due scheduled task in its own transaction (`_cron_run_task`): an error rolls back only this task, is logged and creates the task `cron_error` (*Scheduled task failed*, administration) for the scheduled task, further errors are appended to its description; the other scheduled tasks continue.

2.17.3 Task rules

`real_estate.task.rule` (`task_rule.py`, phase C)

Task rule (menu *Configuration › Tasks and Processes › Task Rules*): reference object (`model`), restriction `domain` (PYSON, like an action domain), reference date from a date field of the model (also function fields) or a named method, offset in months/days, lead time (`horizon_days`), task type (only types for the object), `name_template` (`%(record)s`), `done_domain` and `on_date_change` (cancel or done when the reference date changed). The scheduled task `task_rules` (daily) and the button *Run Now* call `TaskRule.run`: per record and reference date one task (`origin` = rule, `origin_key` = record and date, marked `automatic`) once its due date is within the lead time - also a task already done or cancelled is not created again for the same date. Open tasks of the rule are done automatically when the record matches `done_domain` or no longer matches `domain`, and are cancelled (or done) with a new task when the reference date changed. Errors of a rule are logged and shown in *Last Result*.

Named methods (`TaskRule._date_<name>`): `billing_deadline` (billing units not billed: end of period + 12 months), `meter_reading_due` (properties whose meters lack a reading at the day before the next billing start, ±7 days), `index_values_stale` (last final value older than 2 months), `no_follow_up_contract` (terminated contracts without a later contract for their objects), `state_since` (date of the last state transition from `ir.model.log`), `cron_overdue` (scheduled tasks not run for their interval + 3 days).

Default rules (`task_rule.xml`, `noupdate`, all **inactive**): V02 contract not signed, V03 fixed-term contract ends, V07 contract party ends, V08 no follow-up contract, M01 prepare index rent adjustment, M05 graduated rent ends, M06 termination waiver ends, M07 index values outdated, M08 tight market regulation ends, B01 billing deadline, B02 adjust prepayments, O01 meter readings, O03 meter calibration, O04 object ends, S02 scheduled task not run, P01 inspection overdue, P02 inspection defect overdue. Active by default: M09 *Consent period expired* (comparative rent: no decision of the tenant within the period for consideration task *Check the action for consent*). Each default rule has a description (translatable, German in `locale/de.po`) that is copied into the tasks it creates.

Meter calibration (base object, tab *Meters*): `meter_calibration_date`, `meter_calibration_years`, `meter_calibration_valid_to` (proposed 31.12. of the year of the last calibration + validity, can be overwritten) and `meter_calibration_due` (ends within 3 months, searchable); optional column in the object list.

2.17.4 Processes

`real_estate.process.template` / `real_estate.process` (`process.py`, phase D)

Process templates (menu *Configuration › Tasks and Processes › Process Templates*): reference object, anchor date field (e.g. `termination_date`) and steps. A template step has a task type (reminder, escalation, responsibility) and optionally a responsible group, the due date base (start of the process, anchor date, previous step done or a method, e.g. `billing_deadline_for_contract`) with an offset - or the offset in months from the real estate accounting setting `deposit_task_months`

(default 6) - mandatory/optional, a work instruction, an **action** (report, wizard or list) and the **completion** (manual, by executing the action, or by condition: PYSON domain on the reference record or a method `deposit_paid`, `meter_readings_complete`, `billing_settled`, `handover_*_done`, `inspection_done`, `inspection_approved`), optionally with a required result, and the **creation** (always, or by a method such as `inspection_no_access` / `inspection_has_defects`: the task is created only once the condition is fulfilled; such a step never created does not block the completion).

A process (menu *Tasks and Follow-ups › Processes*, tab *Tasks and Processes* on the contract) is started with the action *Start Process* on contract, rent adjustment, object or billing unit, or automatically by the templates set on the contract type: *Process at Contract Start* (contract set to running), *Process at Termination* (termination wizard), *Process at Partner Change* (partner change wizard). While such a process is running, the contract shows it in the first tab *Workflow* (process, progress and its steps; hidden otherwise - the latest running one if there are several). **Every step is a task**: at the start the process creates one task per template step in the state **planned** (no due date, no reminder, not in *My Tasks*; fields `template_step` and `step_number`); a planned task is opened (due date, reminder, responsibility by the party role on the due date) as soon as it is due - start, anchor and method steps at once, *previous step done* steps once the previous one is done or skipped, conditional steps once their condition is fulfilled. The process form lists its tasks by step number (tab *Steps*). A task of a process shows below its reference the **process**, the **process step** and the **step number**, both opening the record; the task lists have the sortable column *Step No.*, the template step list the column *No.*. (In earlier versions the steps were separate records `real_estate.process.step`; the module update takes their data over into the tasks.) The task of a step with an action shows the button *Execute Action*: it opens the action for the reference record (and completes the step for completion *by action*). *Check State* evaluates the conditions (also daily with the scheduled task `task_rules`), *Reschedule* moves the open anchor tasks after a change of the anchor date (history), *Cancel* cancels the open and planned tasks. When all mandatory steps are done or skipped the process is done (progress "n / m done", conditional steps never opened do not count); conditional steps still planned are then cancelled as *Not required (process done)*. *Reopen* sets a done process running again (also automatically when one of its tasks is reopened) and plans these steps again; *Reactivate* sets a cancelled process running again and sets the tasks cancelled with the process back to planned (steps skipped before stay skipped). Both are noted in the history. The process of an inspection follows the inspection and cannot be cancelled or reactivated directly (see Recurring Inspections).

Default templates (`process.xml`, `noupdate`, not assigned to a contract type): *Move-out* (10 steps: confirmation with the report *Termination Confirmation*, pre-inspection, handover, meter readings, keys, deposit after `deposit_task_months`, remaining deposit and claims check optional, final operating cost billing), *Move-in* (5 steps: deposit received - `deposit_paid`, handover, meter readings, keys, direct debit mandate), *Change of Tenant in the Contract* (4 steps), *Index Rent Adjustment Run* and *Comparative Rent Adjustment Run* on the adjustment run (6 steps each, completed by the state of the run, see Rent Adjustments) and *Vacancy* / *Reletting* on an object (6 steps: meter readings and securing, inspection and repairs - both with role *Caretaker* -, new rent, advertisement, viewings and tenant selection, new lease contract; started with *Start Process* on the rental object). Every default step has a work instruction (translatable, German in `locale/de.po`); it is copied into the task in the language of the user creating it, so existing tasks keep their text.

2.17.5 Handover reports

`real_estate.contract.handover` (`handover.py`)

Handover report (*Übergabeprotokoll*, menu *Contracts › Handover Reports*, tab *Handover Reports* on the contract) of a `move_in`, `pre_inspection` or `move_out`: date and time, objects handed over, tenants present (or absent), landlord representative, other participants, check items (room, item, condition ok / normal wear / damage / missing, description, photos as attachments, remedy by tenant or landlord until a date, estimated costs - total `cost_total` as basis for the deposit settlement), keys (type, description, quantity; for a move-out the quantity of the move-in report is shown and deviations

are marked), meter readings, general condition, swept clean, agreements and the scan of the signed report (`signed_document`, copied once as attachment to the contract - same stored file).

Proposals (on creation, when choosing contract/kind, button *Apply Proposals*): the objects of the contract items, the main tenants, the check items of the checklist chosen by the contract's type of use (*Configuration › Tasks and Processes › Handover Checklists*, default *Apartment* and *Commercial unit*), the meters below the objects, for a move-out the keys of the move-in report and the open defects of the last pre-inspection.

Complete (`draft → done`) checks objects, keys (move-in/out) and one done report per contract and kind (several pre-inspections allowed), warns about missing meter readings (confirmable), creates the meter readings at the handover date (`real_estate.meter_reading`, existing readings of the day are linked), logs the report in the contract log and completes the matching process steps at once. Only the administration can reset a done report to draft. Report *Handover Report (de)* (`report/contract_handover_de.odt`) with signature fields, marked ENTWURF before completion.

Process integration: actions *Move-in / Pre-inspection / Move-out Report* (opened for the contract with `default_kind`) and the done methods `handover_move_in_done`, `handover_pre_inspection_done`, `handover_move_out_done`, used by the default templates *Move-out* (pre-inspection, handover, keys) and *Move-in* (handover, keys).

2.18 Recurring Inspections

Recurring inspections of properties, buildings, rental units and equipment: equipment kinds, inspection types, checklists and inspection plans with due date rules; inspections (protocols) with lines and results, validation, approval, archived report and the scheduled creation; conditional process steps, next access attempt, tenant notice and letters to tenants without access; defects with deadlines, follow-up tasks and escalation rules; default data and demo data.

2.18.1 Concept

An inspection type separates **where the inspection runs** (`scope`: property or building - one inspection = one notice, one report, one approval) from **where results are recorded** (`line_granularity`: none, one line per rental unit, one line per equipment below the object). The smoke detector check is one inspection per building with one line per apartment. Equipment are the existing objects of type *Equipment* with an equipment kind; quantities (e.g. number of smoke detectors) are the measurement type *Number of items*. Steps of an inspection are the steps of the process template of the type (process engine of Tasks, Processes and Handover Reports), anchored on the planned date.

2.18.2 Models

`real_estate.equipment.kind` (`inspection.py`)
Equipment kind (menu *Configuration › Inspections › Equipment Kinds*): name, code. Default kinds (`inspection.xml`, `noupdate`): smoke detector, elevator, playground, fire extinguisher, backflow preventer, heating system, smoke and heat exhaust system. Field `equipment_kind` on objects of type *Equipment* (tab *Equipment*).

`real_estate.inspection.checklist` / `.item` (`inspection.py`)
Reusable checklists (menu *Configuration › Inspections › Inspection Checklists*), one for the header and one for the lines of a type. Item: section, question, `answer_type` (OK/not OK/n.a., yes/no, number with unit and min/max, text, date, selection), mandatory, `photo_on_nok`, `create_defect_on_nok` with `default_severity`.

`real_estate.inspection.type` (`inspection.py`)
Template (menu *Configuration › Inspections › Inspection Types*, not company dependent): cate-

gory, legal basis, scope, line granularity, `equipment_kind` and `line_filter` (PYSON domain on the line objects), interval with unit and basis (fixed rhythm / from execution), `preferred_month`, `lead_time_days`, `tolerance_days`, `notice_days` (warning only), responsible party role, contractor required, unit access with attempts, approval with `approval_group`, header and line checklist, process template (model inspection), defect deadlines per severity (default 1 / 14 / 90 days).

`real_estate.inspection.plan` (`inspection.py`)

Inspection type assigned to a property or building (menu *Real Estate › Master Data › Inspections › Inspection Plans* with tabs *Due in 90 Days*, *Overdue*, *All*; tab *Inspections* on property and building): interval settings defaulted from the type, responsible user (overrides the role), contractor, `owner_maintains`, `last_done_date`, `next_due_date` (first one defaulted: today + interval, with the preferred month), `overdue` (due date + tolerance passed). One active plan per type and object; a building is required for scope building. With a building the property is always the one of the building (proposed when the building is chosen, set when saving - also for plans created in the tab *Inspections* of a building).

Due dates (`next_due_date()`): fixed rhythm = previous due date + interval, repeated until after the execution date; from execution = execution date + interval. With a preferred month the date is moved to the first of that month in the same year, or of the following year if that is not after the execution date. Example (yearly, fixed): due 15.04.2026, done 28.04.2026 → 15.04.2027 (from execution: 28.04.2027).

`real_estate.inspection` (`inspection.py`)

Inspection (protocol) of a plan (menu *Real Estate › Master Data › Inspections › Inspections* with tabs *Open*, *To Approve*, *Overdue*, *Approved*, *All*; also on the plan, on property/building in tab *Inspections* and - as reference with the stored `property` - in tab *Tasks and Processes* of the property). Number from the sequence *Inspection* (PR–<year>–0001).

Created as draft by the button *Create Inspection* on the plan or by the scheduled task `inspection_plans` (*Real Estate Accounting › Scheduled Tasks*, daily: plans whose `next_due_date` minus `lead_time_days` is reached and that have no open inspection). Lines are created from scope, granularity, equipment kind and line filter; a rental unit vacant on the due date (occupancy) gets the access `vacant`; the expected quantity comes from the measurement *Number of items* of the equipment. Header and line results are created from the checklists as snapshot (section, question, answer type, limits). *Rebuild Lines* (draft only, with confirmation) creates the lines again from the current equipment, equipment kind and line filter - e.g. after an equipment kind was set or the filter was changed.

States: *Schedule* (`draft → scheduled`, planned date required, confirmable warning below `notice_days`) starts the process of the type with the inspection as reference and the planned date as anchor (steps with negative offsets before it); *Back to Draft* cancels it; the first answered result or *Start* → `in_progress`; *Done* checks mandatory answers, photos (attachment) for "not OK" items with `photo_on_nok`, a reason for lines not inspected, the contractor and external report (`contractor_required`), warns about lines not inspected after fewer access attempts than required, sets the done date and the overall result (OK or OK with defects, editable), updates `last_done_date` and `next_due_date` of the plan and - without required approval - approves at once; *Reopen* (`done → in_progress`); *Approve* (only members of the approval group) archives the report *Inspection Report (de)* (`report/inspection_protocol_de.odt`) as attachment (`report`) and locks inspection, lines and results; *Cancel* needs a reason and cancels the process. Process steps can be completed by the conditions `inspection_done` and `inspection_approved`. Only draft or cancelled inspections can be deleted; corrections of an approved one by a new inspection (`previous`).

Undo - every step can be taken back: *Back to Draft* (scheduled, cancels the process), *Back to Scheduled* (in progress, entries kept), *Reopen* (done; sets the plan back - next due date of this inspection, last execution of the previous one - and reopens the process step completed by `inspection_done`), *Take Back Approval* (approved → done; approval group of the type or real

estate administration; the archived report stays as attachment marked "(withdrawn)", the step completed by `inspection_approved` is reopened), *Reactivate* (cancelled draft; lines and entries kept, scheduling starts a new process). The inspection leads its process: the process of an inspection cannot be cancelled or reactivated directly - cancelling the inspection or setting it back to draft cancels the process, reactivating and scheduling the inspection starts a new one.

Results, access, quantities, visit dates and remarks of the lines are entered only while the inspection is *scheduled* or *in progress* - read-only in the draft, when done (*Reopen* to change), approved or cancelled (also checked on the server). *Schedule* creates the defects of "not OK" answers stored before.

`real_estate.inspection.line` / `.result` (`inspection.py`)

Line per rental unit or equipment: tenant (main tenant of the occupancy on the visit/planned/due date), access (accessed, no access, refused, vacant, not required), visit date, attempt, quantities expected / checked / OK / replaced, derived result (defect if an item is not OK or fewer OK than checked, not inspected without access), remarks, results. Result: typed value by answer type (OK/not OK/n.a., yes/no, number with limits, text, date, selection), comment, `is_nok`. Rental units and equipment show their lines in tab *Inspections* (history).

Open Items OK - on a line (form and list) and on the inspection (header and all lines with access): answers the items not yet answered with OK (OK/not OK/n.a.) resp. Yes (yes/no); answered items, also "not OK", and other answer types stay unchanged. On a line empty quantities checked/OK are filled with the expected quantity and an empty visit date with today. Lines without access, vacant lines and approved or cancelled inspections are skipped; the buttons are shown only while the inspection is scheduled or in progress.

Conditional steps, notice, letters

A step of the process template can be created only once a condition is fulfilled (`create_condition` = *By Method*, `create_method`): `inspection_no_access` (a line has no access or access refused) and `inspection_has_defects` (a "not OK" result of an item that creates a defect with severity major or critical). Changes of the access of a line or of a result re-evaluate the process at once; a conditional step never created does not block the completion of the process and is transparent for following *previous done* steps.

Work in the inspection, tasks as worklist: the tab *Workflow* is the first tab of the inspection as soon as it has a process (from *Schedule* on). It shows the process (the running one, else the last started), its state and progress (e.g. "2 / 5 done") and lists its steps (tasks) with number, state, due date and responsible. The steps close themselves from the work in the inspection: *Inspection date (attempt 1)* by `inspection_attempt1_done` (every line to visit has a visit date), *attempt 2* by `inspection_attempt2_done` (the lines of the next attempt have a visit date again), *Complete the report* and *Approve* by done / approved; notice and letters complete their step also when created directly on the inspection (`Task.complete_by_action`). Only steps outside the system (e.g. commission the contractor) are set done by hand.

Create Tenant Notice (action on the inspection, usable as step action with completion *By Action*): archives the notice *Tenant Notice (de)* (`report/inspection_notice_de.odt`, one page per building: inspection type, legal basis, date window, time slots and contact - field `time_slots` -, contractor, request for access or key deposit, reply line) as attachment (`notice`, `notice_date` as proof) and prints it. *Create Letters No Access* archives and prints one letter per line without access to the tenant (*Letter No Access (de)*, `report/inspection_access_letter_de.odt`) and sets `letter_date` of the lines; each letter is also archived on its own at the tenant's contract (occupancy on the inspection date). *Next Attempt* (in progress) increases the attempt of the inspection and of the lines without access.

`real_estate.inspection.defect` (`inspection.py`)

Defect (menu *Real Estate › Master Data › Inspections › Defects* with tabs *Open*, *Overdue*, *To Verify*, *All*; tab *Defects* of the inspection; open defects in tab *Inspections* of property, building, rental unit and

equipment): object (building, unit or equipment), description, severity (critical, major, minor, note), detection date, deadline (detection date plus `defect_days_<severity>` of the inspection type, none for notes), responsible (contractor), cost category (unknown, maintenance, operating costs), follow-up task, fixed / verified date.

A "not OK" answer of an item with `create_defect_on_nok` creates the defect at once (severity from the item, description from section, question and comment); a withdrawn answer cancels the still open defect. Defects can also be entered by hand. From severity minor a follow-up task (type `inspection_defect`, default role *Caretaker*) is created; *Fixed* sets it done, *Cancel* cancels it.

States `open` → `in_progress` → `fixed` → `verified` (*Not Fixed*: back to in progress), `cancelled` (*Reopen*). Defects not yet verified are listed in the next inspection of the plan (*Open Defects of Previous Inspections*) and verified there (`verified_in`). Defects of an approved inspection keep only state, dates, responsible, deadline, cost category and follow-up task editable. The overall result of an inspection with a critical defect is *Not OK*; the report lists the defects and the open defects of previous inspections. The process condition `inspection_has_defects` also considers recorded defects of severity major or critical.

Escalation by the default task rules (inactive) *P01 Inspection overdue* (date method `inspection_overdue`: due date plus tolerance) and *P02 Inspection defect overdue* (`defect_overdue`: deadline), task types `inspection_overdue` / `inspection_defect_overdue` (role *Administrator*). Overviews: inspection plans (tabs *Due in 90 Days*, *Overdue*) and defects (tabs *Open*, *Overdue*, *To Verify*).

2.18.3 Default data

Delivered in `inspection.xml` (`noupdate`; changes in the database are kept by later updates):

Annual Walkthrough (code `walkthrough`)
: Building, no lines, yearly fixed rhythm with preferred month April, lead time 30 days, role *Administrator*, approval by *Real Estate Administration*, legal basis § 823 BGB. Header checklist *Annual Walkthrough - Building* (21 items in the sections exterior and roof, entrance, staircase, basement and technical rooms, outdoor areas, notices, remarks; safety items create defects - escape routes, fire doors and combustible storage as critical). Process *Annual Walkthrough*: arrange the date with the caretaker (-21 days), walkthrough (done with the inspection), approve the report (+7, done with the approval), commission repairs (+7, only with major/critical defects, optional).

Smoke Detector Check (code `smoke_detector`)
: Building, one line per residential unit with a smoke detector equipment, yearly, lead time 60 days, notice 14 days, role *Caretaker*, unit access with 2 attempts, approval by *Real Estate Object*, legal basis state building code / DIN 14676-1. Line checklist *Smoke Detectors - per Apartment* (required rooms equipped, mounting, function test, smoke inlets free, earliest replacement date, remarks). Process *Smoke Detector Check*: commission the contractor (-42), notice in the building (-14, action *Create Tenant Notice*), inspection date attempt 1 (0), letter to tenants without access (+7, action *Create Letters No Access*, only without access), attempt 2 (+21, only without access), complete the report (+28), approve the report (+35).

The intervals are common practice and have to be checked per federal state and contract. Further inspection types (elevator, electrical installation, drinking water, fire extinguishers, smoke and heat exhaust, playground, trees, backflow preventer, roof gutters, heating, winter service, vacancy check) are created as own types.

Demo data: `tests/test_immo.py` creates a smoke detector equipment per apartment (measurement *Number of items* = number of rooms) and per building the plans *Annual Walkthrough* (due 01.04.2026) and *Smoke Detector Check* (due 15.05.2026) - see Demo data scripts.

2.19 Wizards

real_estate.contract.create_moves.wizard (contract_wizard.py, class CreateContractMovesWizard)

Batch invoice generation wizard. Supports three actions:

create
: Generate invoices up to a given date. All `ContractTermCashFlow` entries created in this run share the same `create_moves_run_id` (YYYYMMDD-HHMMSS-U<userid>). One invoice per contract and posting date; terms of a term type with `separate_move` get an invoice of their own per term type (see *Contract Management*).

re_calc
: Rebuild cash flow projections without creating invoices.

re_calc_and_create
: Combine both steps.

Can be filtered by property and/or contract list. Optional queue execution via `execute_in_queue`.

real_estate.billing_unit.wizard (billing_unit_wizard.py, class BillingUnitWizard)

Batch billing wizard, opened via the `billing_wizard` button on the billing unit form or the `billing_property` button on the property form.

Start — `date` (default: last day of the current month), `company`, `invoice_date` (default: today; auto-set to the first of `date`'s month when `date` lies in the past), `invoice_state` (`draft` / `posted` — invoices are posted immediately when `posted`), `payment_term` (default: `account.configuration`'s `re_payment_term_billing`; written to every invoice created by the run and therefore drives its due date — takes priority over the contract's own payment term), `execute_in_queue` (default `True`), optional `propertys` / `billing_units` filters (pre-filled from the active record when launched from a property or billing unit form/list).

Confirm — read-only summary of the matching billing unit count and the selected filters before processing.

Process (`transition_do_billing`) — calls `BaseObject.call_billing` for the resolved properties, which invokes `BaseObject.do_billing` either directly or via the background queue. Only billing units already in state `ready_for_billing` are billed; with `collective_billing` all billing units of a property sharing the same start date must be included, otherwise a `ValidationError` is raised.

Result — reports how many billing units were queued or processed.

real_estate.terminate_contract.wizard (contract_wizard.py)

Sets a contract to *Terminated*, records `terminated_by`, `receipt_of_termination_notice`, `termination_reason`, and calculates `termination_date` from the notice period (3 / 6 / 9 / 12 months to end of month); also writes the resolved `termination_date` into the contract's `end_date` (see *Fixed term / termination* under `real_estate.contract` above). `terminated_by_type` is `tenant` or `landlord` for a manual termination via this wizard; see the `update_contract_status` cron task above for the automatic `expired` case (fixed-term contracts with no active termination), and the *Revert Termination* button to undo a termination.

real_estate.change_contract_partner.wizard (contract_wizard.py, class ChangeContractPartnerWizard)

Reassigns a contract's main tenant as of a chosen `change_date` (domain- and server-side restricted to the contract's own validity period), rebooking still-open invoices to the new party rather than just swapping a reference.

Start — `contract` (readonly), `current_party` (Function), `new_party` (domain excludes the current party), `change_date` (required, domain restricted to `[contract's start_date,`

effective end date] via two Function fields — a plain Many2One reference field cannot use _parent_<field> PySON the way a true parent-child nested form can, so contract_start_date/ contract_end_date mirror those dates onto the wizard's own start state instead).

Process (Contract.execute_change_partner):

- Rejects reassigning to the same party (msg_change_partner_same_party) and a change_date outside the contract's validity (msg_change_partner_date_out_of_range).
- Warns (confirmable, ContractPartnerChangeDraftInvoicesWarning) if the outgoing party still has draft invoices on this contract — those are never picked up by the rebooking below (only posted, still-open items are), so they would otherwise silently stay with the old party.
- Finds still-open (posted, unreconciled) invoices via the cash flow (ContractTermCashFlow, filtered by contract — deliberately not restricted to a specific term/condition, so rent *and* operating cost/heating advances on the contract are all rebooked together, matching what a real partner change should do). For each: a credit note closes the old party's open item (same mechanism as BillingUnit.cancel_units), reconciled against it when balanced; a new invoice with lines copied 1:1 (same account/amount/taxes/ contract/term/base_object — only party and accounting_date differ) reopens the same charge under the new party.
- Ends the old party's open *Main Tenant Role* assignment the day before change_date; optionally starts a *Secondary Tenant Role* assignment for the old party (if configured on the contract type); starts a new *Main Tenant Role* assignment for the new party (invoice_address pre-filled from the new party's own default invoice address).
- Re-runs the cash flow plan (call_create_moves with action='re_calc', synchronous, invoice-less) for this one contract afterwards, so still-draft plan entries immediately reflect the new party instead of waiting for the next scheduled create_moves run to happen to process this contract.

Button *Partner wechseln* on the contract form, "Parties" tab.

real_estate.estimate_consumption.wizard (base_object.py, class EstimateConsumptionWizard)

Two-step wizard launched from the *Meters* tab of any approved meter object. Estimates the meter reading for a chosen cut-off date and books it as an estimate reading.

Step 1 – Start: displays the meter description, unit and factor, pre-fills meter_id from the last reading, and prompts for per_date (default: today) and a free-text reason.

Step 2 – Result: calls simulate_estimate and shows the two basis readings (reading1 / reading2), the derived consumption, and the editable estimated_value. The user can adjust the value before booking.

Book: saves a new MeterReading with m_type = 'estimate'. The value is rounded to the meter's UOM digit precision (integer if meter_no_decimals is set) before saving.

real_estate.option_rate_update.wizard (option_rate_wizard.py, class OptionRateUpdateWizard)

Recomputes and books option rates (see Option Rate) for a selection of base objects, billing units, and/or settlement units, as of a cut-off date.

Start — base_objects (Many2Many, restricted to property/building/land/rental object; an approved property/building/land is expanded to its building/land/rental-object descendants, and an approved property additionally cascades to its billing units and their settlement units), billing_units (expanded to their settlement units), settlement_units, cutoff_date (default: today — booked option rates are always dated on the first of this month).

Confirm — read-only counts of the directly selected base objects / billing units / settlement units, and the total number of objects that will actually be processed after expansion (`n_expanded`).

Process (`OptionRate.process_update`) — for every expanded object:

- Objects with `option_rate_method` `fix_0` / `fix_100` are booked at 0 % / 100 % directly, without looking at any sub-objects.
- `dynamic_measurement` objects are calculated from their approved rental objects: for a rental object, itself; for a property/building/ land, its approved rental-object descendants regardless of how many buildings/land lie in between (a non-approved intermediate object excludes its whole subtree — both from booking and as a calculation basis); for a settlement unit, its existing `objects` function field (i.e. whatever it already resolves to via its own allocation rule/regex — read-only here, never modified by this wizard); for a billing unit, the union of `objects` across all its settlement units that are not `draft`/`billed`, deduplicated. Each contributing rental object is weighted by its measurement value (leaf types under `option_measurement_type` summed if it is a group) at the cut-off date, and contributes its own current option rate (its latest `option_rate` record at or before the cut-off date, else its own `fix_0`/`fix_100` default). The new rate is the resulting weighted average, rounded to 2 decimals.
- Base objects not in state `approved`, and billing/settlement units in state `draft` or `billed`, are excluded entirely.
- A new dated `option_rate` record is only written when the computed rate differs from the current one; a record already dated on the cut-off month's first day is updated in place rather than duplicated.

Result — counts of created / updated / unchanged / skipped records, plus a per-record detail log.

Menu: *Real Estate* → *Master Data* → *Update Option Rates*.

The former wizard *Adjustment of Contract Terms* is replaced by the adjustment run (see Rent Adjustments, *Adjustment run*).

`real_estate.contract.term.adjustment.run.add` (`contract_comparative_rent.py`)
Add Terms of an adjustment run (button on a selected or calculated run): the selected terms are processed by the procedure's selection with all its checks; added and excluded terms are logged in the protocol of the run.

`real_estate.contract.new_letting_check` (`contract_comparative_rent.py`)
Check Comparative Rent (New Letting) on a draft residential contract with a rent survey: compares the rent terms with the comparative rent on the contract start (+10 % in a tight housing market, else +20 %) and sets them to the limit if wanted - see Comparative Rent (Rent Survey), *New letting check*.

`real_estate.contract.copy_terms` (`contract_wizard.py`)
Copy Terms on a draft contract: choose the source contract (default: the previous contract of the rental units), then mark the proposed terms and adjust the unit price - see Contract Management.

2.20 Reports

ODT templates (OpenDocument Text) are located in `report/` and rendered via Genshi/relatorio; `template_extension` on the `ir.action.report` record is `odt`. Values are inserted with `text:span py:content="..."` rather than literal `${...}` interpolation (which relatorio escapes in ODT text), and control flow (`py:if`/`py:for`/`py:choose`) is written as attributes on the surrounding ODF elements. Older `.html` versions of these templates remain in `report/` for reference only and are not registered.

Several records in one document (letters, protocols, meter reading sheets): every record starts on a new page and the page numbering restarts at 1 - the first paragraph of a record uses the paragraph style `PBreak` (break before page, master page *Standard*, page number 1), chosen with

two paragraphs and `py:if="index"` / `py:if="not (index)"`; expressions inside ODF attributes (`text:style-name="${...}"`) are not evaluated. The master page has a footer *Seite N*.

Templates are maintained as ODF XML (`content.xml`/`styles.xml`), not by re-saving the `.odt` in an office suite (which drops the `py:` attributes). Context helpers exposed to templates must not start with an underscore and templates must not contain `import` statements (Genshi sandbox); the shared `format_value` helper and `Decimal` are provided via `get_context()`.

`real_estate.contract.report` (`contract_report.py`)
: General contract report. Templates: `contract_en.odt`, `contract_letter_de.odt`.

`real_estate.contract.annex4.report` (`contract_report.py`)
: Annex 4 – Betriebskostenaufstellung. Template: `anlage4_contract_de.odt`. Renders grouped operating cost positions with BetrKV paragraph references and allocation method labels, sourced from the contract's own `settlement_units` field (see `real_estate.contract` below) rather than re-deriving the billing unit independently. Allocation labels are translatable messages (`real_estate.msg_allocation_*` in `message.xml`).

`real_estate.contract.termination_confirmation.report` (`contract_report.py`)
: Termination confirmation letter ("Kündigungsbestätigung"). Template: `contract_termination_confirmation_de.odt`. Registered exactly like the Contract Letter/Annex 4 reports above — an `ir.action.report` plus an `ir.action.keyword` with `keyword='form_print'` on `real_estate.contract,-1` — so it appears in the standard Print toolbar for every contract, not behind a dedicated button. "Only available once a termination is recorded" is therefore enforced entirely inside `get_context()`: it checks `record.state == 'terminated'` for every selected record and raises `msg_contract_termination_confirmation_not_terminated` (a clear, translated error) otherwise, rather than by hiding the menu entry itself. Resolves the translated labels for `terminated_by_type`/`termination_notice` (both `Selection` fields) via `Contract.fields_get()` rather than exposing the raw storage keys to the template.

`real_estate.contract.index_adjustment.letter` (`contract_report.py`)
: Declaration of an index rent adjustment (§ 557b para. 3 BGB) on `real_estate.contract.term.adjustment`. Template: `index_adjustment_letter_de.odt` - one letter per selected adjustment (page break), addressed jointly to all main tenants (`contract.main_tenant_party_ids`, invoice address). The button *Declare* renders it with `data['original']` and archives the original as attachment of the adjustment and of the contract (copy sharing the stored file). Mark: *ENTWURF* before the announcement, none while announced and the receipt is not captured (dispatch), *Zweitschrift* afterwards. Index values are formatted with one decimal (`format_index`).

`real_estate.contract.comparative_rent.letter` (`contract_report.py`)
: Request for consent to a rent increase up to the local comparative rent (§§ 558, 558a BGB) on `real_estate.contract.term.adjustment`. Template: `comparative_rent_letter_de.odt` - one letter per adjustment to all main tenants (as the index declaration) with current and new net rent, increase, effective date, reasons (rent survey, version and kind, table cell, classification from the calculation protocol, deviating accepted rent with reason), cap with its basis, period for consideration (`consent_until`) and a consent form. *Announce* of the adjustment run archives the original.

`real_estate.contract.term.adjustment.run.letters` (`contract_report.py`)
: *Letters to Send* on the adjustment run (button *Print Letters*, state announced/ready): all announced adjustments without captured receipt in one document, rendered by the letter report of the procedure without mark; error if none is left.

`real_estate.contract.handover.report` (`contract_report.py`)
: Handover report on `real_estate.contract.handover`, template `contract_handover_de.odt`: header with landlord, tenants, rented object, kind and date, tables of check items, keys (move-out: expected quantity) and meter readings, general condition, agreements and signature fields; ENTWURF until the report is done.

`real_estate.meter_reading.sheet.report` (`meter_reading_sheet.py`)
: Meter reading sheet to take along, template `meter_reading_sheet_de.odt` (landscape): object, reading date, reader, selection and a table per meter with rental unit, tenant, meter, meter ID, previous reading, new value (empty until entered), unit and remarks, signature line; ENTWURF until the sheet is done.

`real_estate.base_object.report` (`base_object.py`)
: Fact sheet for a property or object. Template: `fact_sheet.odt`.

2.21 Accounting / WoWi

The `wowi/` directory contains a German "Kontenrahmen der Wohnungswirtschaft" (WoWi, housing-industry chart of accounts) as XML templates loaded at module installation:

- Account types and accounts
- Tax groups, tax templates, tax code templates, tax code line templates
- Tax rule templates

https://www.intex-publishing.de/pdf/Kontenrahmen_Wohnungswirtschaft.pdf https://www.intex-publishing.de/pdf/Kontenrahmen_Immobilienwirtschaft.pdf

2.22 Source Layout

```
real_estate/
├── __init__.py             # Pool registration of all models, wizards and␣
↪reports
├── tryton.cfg              # module version, dependencies, XML data files
├── setup.py                # packaging (jankstar_real_estate,␣
↪trytond.modules entry point)
├── address.py              # real_estate.address
├── adjustment_run.py       # real_estate.contract.term.adjustment.run␣
↪(adjustment
│                           #   run, procedure registry, manual amount,␣
↪consent)
├── base_object.py          # real_estate.base_object, occupancy, meter␣
↪readings
├── billing_unit.py         # real_estate.billing_unit, billing_unit.moves,
│                           #   billing_unit.log, cost_type,␣
↪cost_category_group
├── billing_unit_wizard.py  # real_estate.billing_unit.wizard (batch billing)
├── bved.py                 # BVED Tryton models: service_provider,␣
↪provider_assignment,
│                           #   object_number, export, import(.line)
├── bved_records.py         # BVED fixed-width record (de)serialization, no␣
↪Tryton dependency
├── co2_kostaufg.py         # real_estate.co2_kostaufg(.consumption),␣
↪co2_emission_share
├── company.py              # extension to company.company (re_accounting␣
↪link)
```

(continues on next page)

(continued from previous page)

```
├── contract_comparative_rent.py  # comparative rent procedure (§ 558 BGB),
│                             #   cap, consent, wizard Add Terms
├── contract_core.py          # real_estate.contract, contract.log, account␣
↪views,
│                             #   cron_daily dispatcher + cron task handlers
├── contract_index_rent.py    # index rent agreement and adjustments (§ 557b␣
↪BGB)
├── contract_item.py          # real_estate.contract.item, contract.item.object
├── contract_party.py         # real_estate.contract.party, contract.party.role
│                             #   (+ role-contract.type relation)
├── contract_term.py          # real_estate.contract.term, term.tax, cash_flow,
│                             #   term.adjustment, Quantitative
├── contract_type.py          # real_estate.contract.type, term.type
├── contract_rent_adjustment.py  # real_estate.contract.rent_adjustment (rent
│                             #   adjustments, graduated rent generation)
├── contract_report.py        # ContractReport, ContractAnnex4Report,
│                             #   ContractTerminationConfirmationReport,
│                             #   index declaration, comparative rent request
├── contract_wizard.py        # wizards: TerminateContract,␣
↪CreateContractMoves,
│                             #   CancelPeriodBooking, ContractRunning,
│                             #   ChangeContractPartner
├── cron_task.py              # real_estate.cron_task (per-company scheduled␣
↪task config)
├── handover.py               # real_estate.contract.handover(.line, .key, .
↪meter),
│                             #   handover.checklist(.line); process step␣
↪conditions
├── inspection.py             # real_estate.inspection(.line, .result, .
↪defect),
│                             #   inspection.type, .plan, .checklist(.item),
│                             #   equipment.kind; notice / access letter␣
↪wizards
│                             #   and reports, inspection report
├── invoice.py                # extensions to account.invoice / invoice.line /
│                             #   move.line / general_ledger.line,␣
↪receivable/payable
│                             #   list context + report
├── ir.py                     # extension to ir.cron (registers the cron_daily␣
↪method)
├── measurement.py            # real_estate.measurement.type, measurement
├── meter_reading_sheet.py    # real_estate.meter_reading.sheet(.line) and its␣
↪report
├── object_party.py           # real_estate.object_party, object_party.role
├── option_rate.py            # real_estate.option_rate, option_rate.context
├── option_rate_wizard.py     # real_estate.option_rate_update.wizard
├── party.py                  # extension to party.party
├── price_index.py            # real_estate.price_index(.value), index_cap_rule
```

(continues on next page)

(continued from previous page)

```
├── process.py                # real_estate.process.template(.step), process,
│                             #   wizards Start Process / Execute Action,␣
↪process
│                             #   start hooks (termination, partner change)
├── re_accounting.py          # real_estate.re_accounting (company-scoped RE␣
↪config)
├── rent_survey.py            # real_estate.rent_survey(.version, .
↪dimension(.class),
│                             #   .cell, .group, .feature, .calculation),
│                             #   base_object.rent_survey_value,␣
↪import/features wizards
├── res.py                    # extension to res.user
├── settlement_result.py      # real_estate.settlement_result, cost_share
├── sequence.py               # extension to ir.sequence ("string timestamp"␣
↪type)
├── settlement_unit.py        # real_estate.settlement_unit
├── task.py                   # real_estate.task.type, task (follow-ups),␣
↪wizards
│                             #   Postpone / Create Task, task tabs on records
├── task_rule.py              # real_estate.task.rule (scheduled task rules)
├── report/                   # ODT report templates (rendered via␣
↪Genshi/relatorio)
├── view/                     # XML form and tree view definitions
├── wowi/                     # German WoWi accounting templates
├── tests/                    # unit tests (test_module.py) and demo data␣
↪scripts
├── doc/                      # this documentation (Sphinx)
└── locale/                   # Translations (de.po)
```

2.23 Running Tests

Warning

`tests/` currently has no `unittest.TestCase`-based tests — `test_module.py` (the standard Tryton `ModuleTestCase` boilerplate) has been removed and not replaced. `tox.ini` still defines a real test matrix (`envlist = {py39,py310,py311,py312,py313}-{sqlite,postgresql}`) and the commands below run without error, but `unittest/xmlrunner` discovery currently finds **zero** test cases in every environment and exits **0 (success)** regardless — i.e. `tox` reports a false-positive pass, not "no tests configured". There is no CI pipeline in this repo currently invoking it automatically, but running it manually and reading "OK" is misleading until a real test case is added back.

```
# Full test matrix (sqlite + postgresql, py39–py313) — currently a
# false-positive pass, see warning above
tox

# Single environment
```

(continues on next page)

(continued from previous page)

```
tox -e py311-sqlite

# Direct run
export TRYTOND_DATABASE_URI=sqlite://
export DB_NAME=:memory:
coverage run --omit=*/tests/* -m xmlrunner discover -s tests
coverage report
```

See Demo data scripts for the one-shot proteus import scripts under `tests/`.

2.24 Demo data scripts

Everything in `tests/` is instead a one-shot **proteus import script** that generates demo data against a running trytond server. These are run manually — `python tests/<script>.py --database <db> [--config trytond.conf]` — not via tox/xmlrunner, in the following order (each step builds on the previous one's data):

`tests/test_immo.py`
: No prerequisites. Creates two properties (*Musterstraße 1-4* and *Musterstraße 5-8*), each with four buildings. Every building has one ground-floor retail unit (`type_of_use='commercial'`, 140 m²) and four apartments (`type_of_use='residential'`, 57/83 m² alternating) — 16 apartments, 4 retail units, and 20 water meters per property in total — plus one land parcel with four parking spaces. Meter names follow the pattern `Wasser Zähler NN` / `Wasser Zähler EHNN`, each with an initial and a consumption reading. Beforehand it creates the parties *Verwalter 1* and *Hausmeister 1*, each with an employee of the company (existing ones are reused), and assigns them to both properties as object parties with the roles *Administrator* and *Caretaker* (valid from 01.01.2025) - the basis of the task responsibility by party role (see Tasks, Processes and Handover Reports). Linking the employees to users is left to the administrator; the script prints a hint at the end. Each apartment gets a smoke detector equipment (equipment kind *Smoke Detector*, measurement *Number of items* = number of rooms) and each building the inspection plans *Annual Walkthrough* (due 01.04.2026) and *Smoke Detector Check* (due 15.05.2026) if the default inspection types exist. Looks up `real_estate.use_class`, measurement types and object party roles by `sequence`, equipment kind and inspection types by `code` (language-independent):

```
python tests/test_immo.py --database <db> [--config trytond.conf]
```

`tests/test_contracts.py`
: Requires `test_immo.py`. Creates residential lease contracts for 15 of 16 apartments per property (one left vacant), each with rent, operating cost, and heating terms; three contracts per property are terminated, two of them with a follow-up contract. The four parking spaces per property are split between existing apartment contracts (as an extra contract item) and new standalone parking contracts. Also creates commercial lease contracts for the retail units — one combined contract for all four retail units on *Musterstraße 1-4*, one contract per unit on *Musterstraße 5-8*. Terms are absolute amounts (price per m² × area, quantity 1). Up to three running apartment contracts per property get a graduated rent (rent adjustment with procedure `graduated_rent` on the rent term: 12-month rhythm, 5 steps, +30 € / +0.40 €/m² / +3 %), whose steps are generated; the procedure is allowed on the residential contract type and the rent term type first if missing. Up to two further running apartment contracts per property (without graduated rent) get an active index rent (procedure `index_rent`: price index `VPI-DE`, base month 01/2025, statutory effective date, no threshold, cap automatic); missing VPI values of the current base year are first taken over from `tests/61111-0002_de.csv` (GENESIS-Online table download, parsed like the CSV import wizard) - without the file or the base month value no index rents are created. The adjustment run itself is not executed:

```
python tests/test_contracts.py --database <db> [--config trytond.conf]
```

tests/test_billing_unit.py

Requires `test_immo.py`. Creates two billing units per property: *Kalte Betriebskosten* (measurement- and consumption-based settlement units, vacancy allocated to the owner) and *Heizkosten* (`external_billing=True`, settlement units use `allocation_from_external_billing`). Billing units are left in state `draft` — anything that requires a later state (e.g. `test_invoices.py`'s settlement-unit lookup) needs the workflow advanced manually first:

```
python tests/test_billing_unit.py --database <db> [--config trytond.conf]
```

tests/test_kreditor.py

No prerequisites. Creates 11 supplier/creditor parties (property tax office, building insurer, utilities, cleaning, caretaker, gardening, chimney sweep, heating maintenance), each with an invoice/delivery address:

```
python tests/test_kreditor.py --database <db> [--config trytond.conf]
```

tests/test_invoices.py

Requires `test_kreditor.py`, `test_immo.py`, and `test_billing_unit.py` (with billing units advanced past `draft` so settlement units can be looked up). Creates the corresponding purchase invoices — one-off and recurring — for each property, linking each line to its `settlement_unit` where a match is found. Invoices are only saved, not posted (remain in state `draft`):

```
python tests/test_invoices.py --database <db> [--config trytond.conf]
```

tests/test_rent_survey.py

Creates the rent survey *Berliner Mietspiegel* with the complete version *Berliner Mietspiegel 2026* (valid from 28.05.2026, qualified, table method, 20 % per group, netted): classification features *Wohnlage* (manual) and *Bezugsfertigkeit* (from the year of construction; the classes 1973-1990 West/East manual only) with their classes, all 189 table cells and the 86 features of the guidance (no. 11) and the reduction for minor equipment (no. 9.4), imported from `tests/mietspiegel_berlin_2026_felder.csv` and `tests/mietspiegel_berlin_2026_merkmale.csv` via the import wizard. With `--only-survey` only the rent survey is created (no prerequisites). Otherwise it also requires `test_immo.py` and assigns it to *Musterstraße 1-4*: location medium on the property; year of construction 1958 (if empty), *Besonders ruhige Lage*, bike room, insulation and an energy consumption value of 130 kWh/(m²a) on every building; features of the groups 1-3 on *Wohnung 01*; then calculates its comparative rent (key date today, at the earliest 28.05.2026). Aborts if the rent survey already exists:

```
python tests/test_rent_survey.py --database <db> [--config trytond.conf]␣
↪[--only-survey] [--company <name>]
```

tests/test_payment.py

Requires `test_contracts.py` and at least one run of the `CreateContractMoves` wizard so posted tenant invoices exist. Books one payment receipt per tenant/commercial-tenant party (debit account 1800 Bank / credit the receivable account taken from that party's open lines) and reconciles the open items:

```
python tests/test_payment.py --database <db> [--config trytond.conf]
```

tests/migrate_run_rents.py

Test migration for the comparative rent: sets the rent terms of the contracts of an adjustment run

(default `AL-2026-0001`; a run without adjustments takes the candidates of its filters, without active index rents) to about 6.00 EUR/m² (`--rent`, ± `--spread` 0.20 fixed per contract number, idempotent) from the contract start - area terms get the price per m², absolute terms price × living space. Only the open rent term is changed; posted invoices stay unchanged (test data), the booked-values warnings are confirmed. `--recalculate` calculates the run again, `--dry-run` only lists the new rents:

```
python tests/migrate_run_rents.py --database <db> [--config␣
↪trytond.conf] [--run AL-2026-0001] [--rent 6.00] [--spread 0.20]␣
↪[--recalculate] [--dry-run]
```

2.25 Release notes